Python for computational Problem Solving · Unit_04 · UNIT 4: Testing ,Debugging and Object Oriented Programming

DRAFT — UE26CS151A · Unit_04 — UNIT 4: Testing ,Debugging and Object Oriented Programming · Complete Unit Master

Source-complete searchable master · transcript, slide text, page visuals and archive references

Hugging Face PESU_Materials source tree, official units.json catalogue, transcript sidecars, slide text/PDF/PPT sources and the selected First_Year_8enterprise archive; see the provenance datasets in the release ZIP. Generated 18 October 2026.

· How to use this pack

Search this single unit master by lesson, timestamp, slide text or source page. Read the transcript in timestamp order, then compare extracted slide text with its page-faithful source figure. Formula candidates and archive question mappings are explicitly flagged for review. Topics include the applicable parts: overview, key terms, formulas, figures, flowcharts, important points, worked examples, past-paper questions and a recap. IDs are shared with the separate documents: E = formula, G = graph, F = flowchart, W = worked example. Each ID is numbered by topic, so E4.2 is the second formula of topic 4.

Source text	Verbatim or OCR-derived text, kept separate from editorial notes.
Candidate	Automated topic/source association requiring manual confirmation.
Low ASR confidence	Transcript segment requires audio verification.
DRAFT	Human review remains incomplete.

Contents

1. 1 · UE26CS151A · Unit_04: UNIT 4: Testing ,Debugging and Object Oriented Programming
2. 2 · Pytest, doctest, pdb
3. 3 · Pytest, doctest, pdb
4. 4 · Python Classes and Objects
5. 5 · Inheritance
6. 6 · Exceptions
7. 7 · Source slide deck: Virtual Environments
8. 8 · Source slide deck: UE24CS151A PCPS Unit 4 SindhuRPai Class 8 9 OOP
9. 9 · Source slide deck: UE24CS151A PCPS Unit 4 SindhuRPai Class 11 Polymorphism
10. 10 · Source slide deck: UE24CS151A PCPS Unit 4 SindhuRPai Class 12 Iterators
11. Appendix A · Symbol glossary
12. Appendix B · Sources, coverage and gaps
13. Appendix C · Revision and retrieval practice

M1

Unit_04 · UNIT 4: Testing ,Debugging and Object Oriented Programming

1 UE26CS151A · Unit_04: UNIT 4: Testing ,Debugging and Object Oriented Programming

This unit master organizes source material for Python for computational Problem Solving, Unit_04, officially titled UNIT 4: Testing ,Debugging and Object Oriented Programming. It includes 5 video transcripts and 7 source slide decks, with page-level text and rendered images where available. This draft preserves source wording and page order; it does not claim a complete human subject review or full audiovisual review.
EV0000001

SHORT-NOTE RECAP

Official catalogue entry: UE26CS151A, Unit_04 (UNIT 4: Testing ,Debugging and Object Oriented Programming). Bucket folder present: yes. Videos: 5; slide decks: 7.
EV0000001

2 Pytest, doctest, pdb

This lesson is indexed from the source video titled Pytest, doctest, pdb. Its complete timestamped transcript is preserved below in segment order. The original video path and content hash are recorded in datasets/lecture-corpus.json. Transcript text is source wording; it has not been silently corrected or rewritten. A candidate slide deck is grouped here according to the current matching score; the exact path and page locators are listed in the source evidence. EV0000002 · EV0000003

Graphs and diagrams

Figure G000001. UE24CS151A PCPS Unit 3 Sindhu Class 17 18 Pytest doctest pdb · source slide page 1.

EV0000103

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

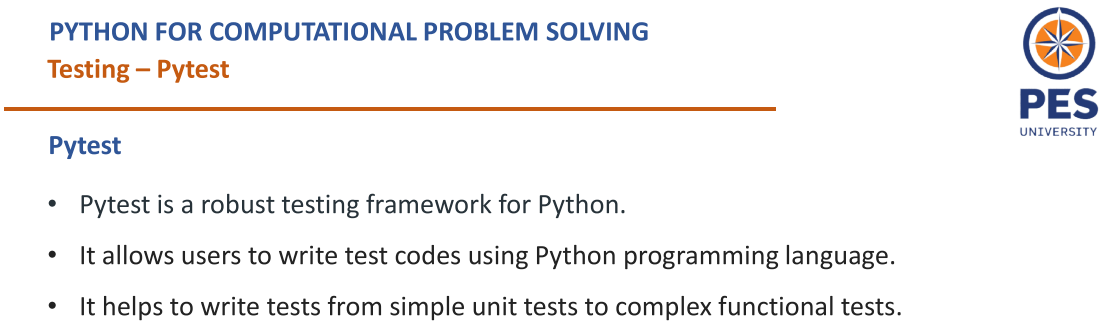


Figure G000002. UE24CS151A PCPS Unit 3 Sindhu Class 17 18 Pytest doctest pdb · source slide page 2.

EV0000106

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

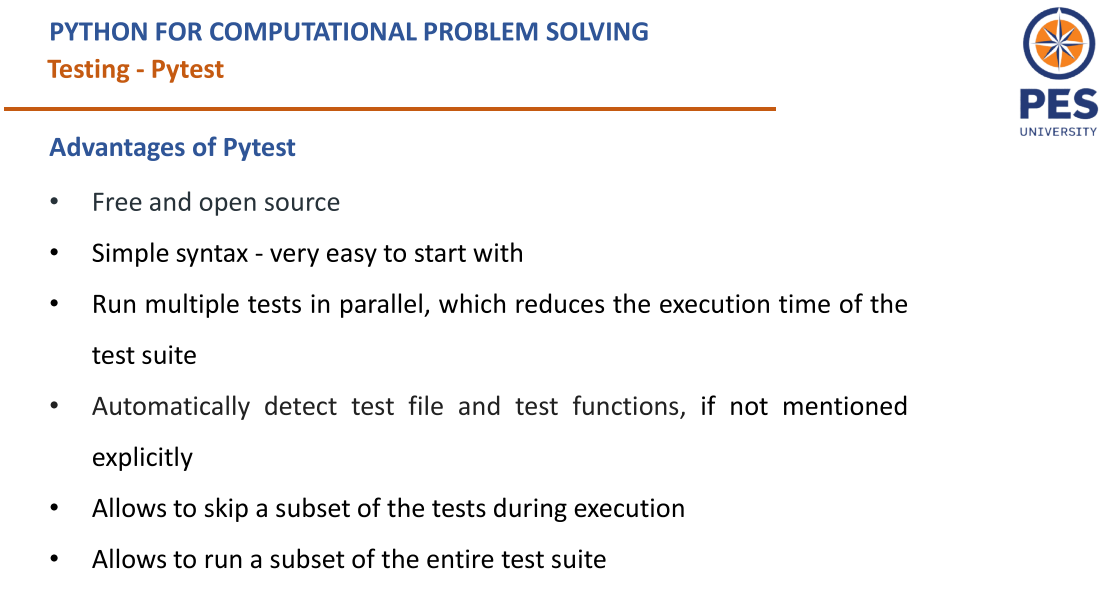


Figure G000003. UE24CS151A PCPS Unit 3 Sindhu Class 17 18 Pytest doctest pdb · source slide page 3.

EV0000109

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

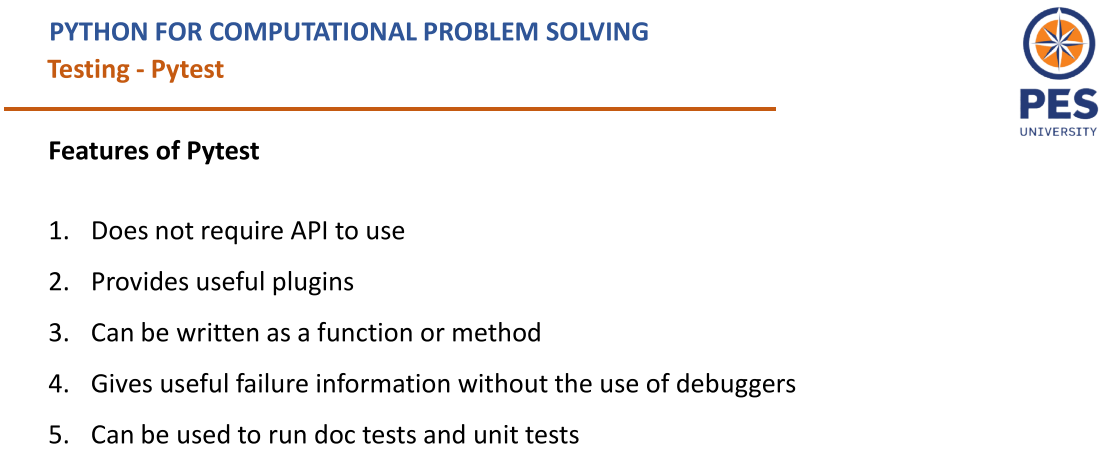


Figure G000004. UE24CS151A PCPS Unit 3 Sindhu Class 17 18 Pytest doctest pdb · source slide page 4.

EV0000112

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

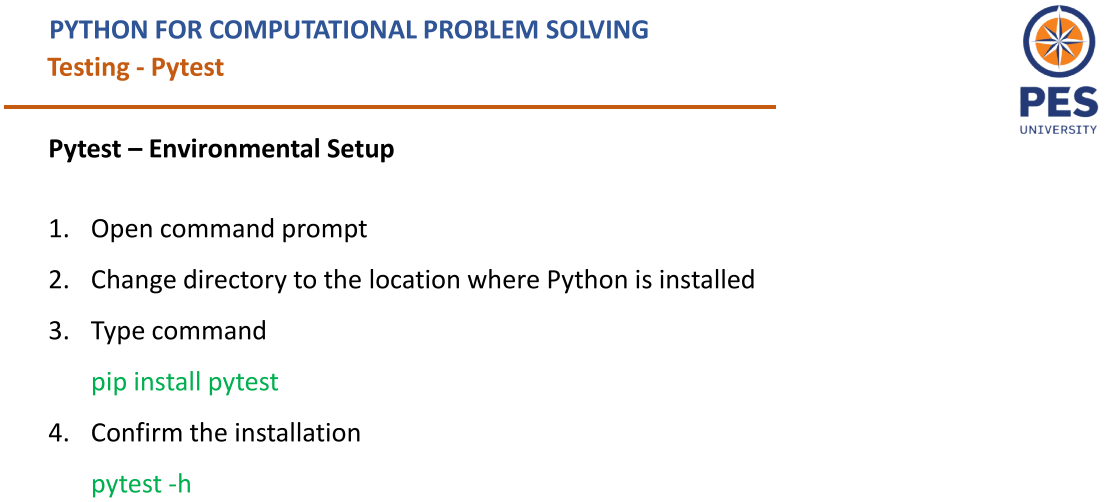


Figure G000005. UE24CS151A PCPS Unit 3 Sindhu Class 17 18 Pytest doctest pdb · source slide page 5.

EV0000115

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

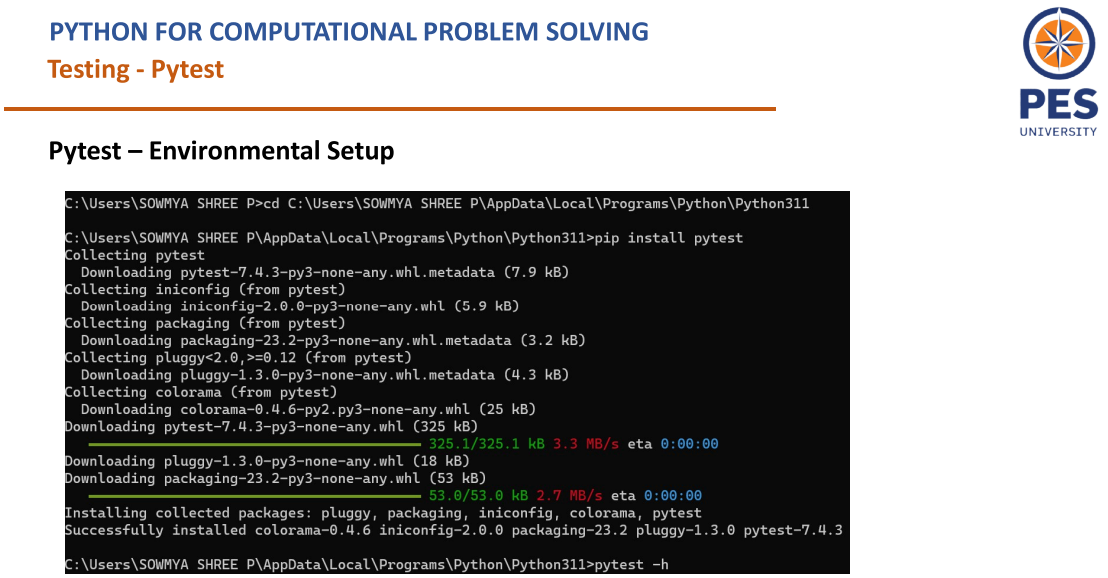


Figure G000006. UE24CS151A PCPS Unit 3 Sindhu Class 17 18 Pytest doctest pdb · source slide page 6.

EV0000118

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

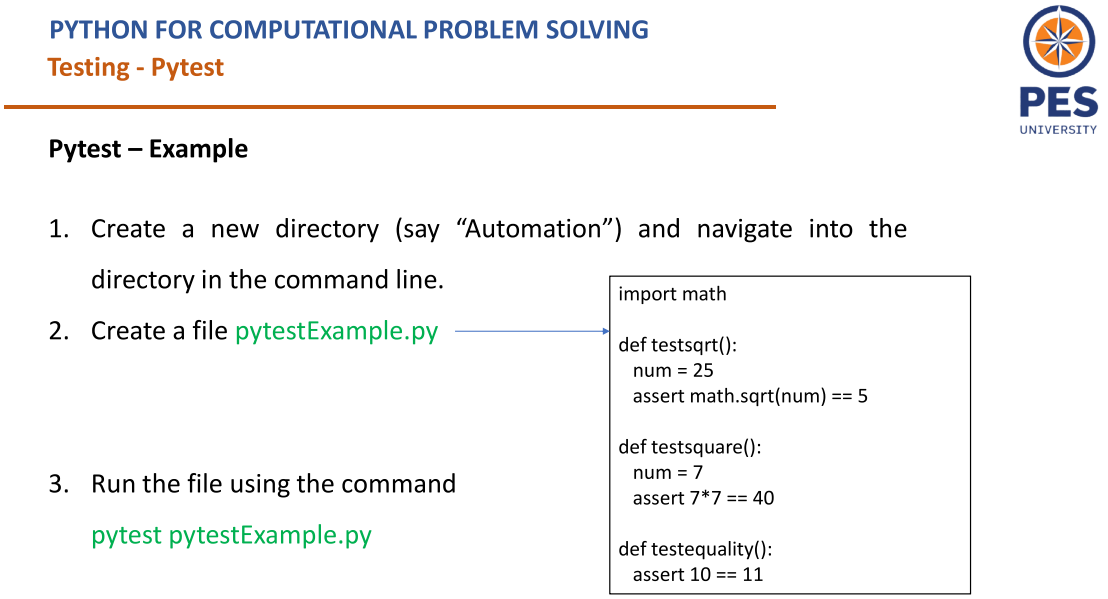


Figure G000007. UE24CS151A PCPS Unit 3 Sindhu Class 17 18 Pytest doctest pdb · source slide page 7.

EV0000121

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Testing - Pytest

PES UNIVERSITY

Pytest – Example (Output)

```
C:\Users\SOWMYA SHREE P\AppData\Local\Programs\Python\Python311\Automation>pytest pytestExample.py
============================================= test session starts =============================================
platform win32 -- Python 3.11.5, pytest-7.4.3, pluggy-1.3.0
rootdir: C:\Users\SOWMYA SHREE P\AppData\Local\Programs\Python\Python311\Automation
collected 3 items

pytestExample.py .FF

================================================== FAILURES ===================================================
_____________________________________________ testsquare ______________________________________________

    def testsquare():
        num = 7
>       assert 7*7 == 40
E       assert (7 * 7) == 40

pytestExample.py:9: AssertionError
_____________________________________________ testequality _____________________________________________

    def testequality():
>       assert 10 == 11
E       assert 10 == 11

pytestExample.py:12: AssertionError
=========================================== short test summary info ===========================================
FAILED pytestExample.py::testsquare - assert (7 * 7) == 40
FAILED pytestExample.py::testequality - assert 10 == 11
========================================= 2 failed, 1 passed in 0.06s =========================================

C:\Users\SOWMYA SHREE P\AppData\Local\Programs\Python\Python311\Automation>
```

7

Figure G000008. UE24CS151A PCPS Unit 3 Sindhu Class 17 18 Pytest doctest pdb · source slide page 8.

EV0000124

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

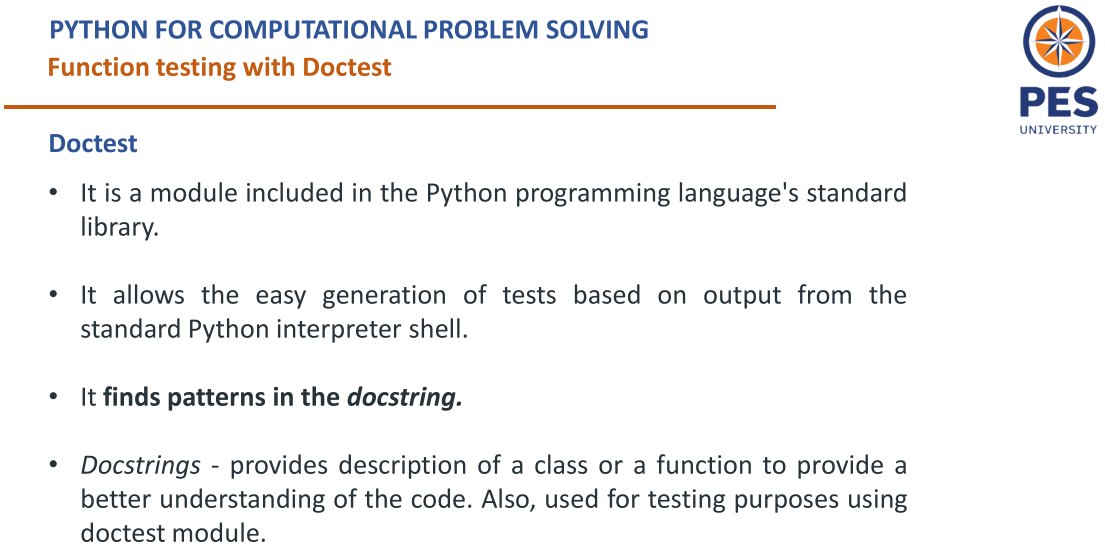


Figure G000009. UE24CS151A PCPS Unit 3 Sindhu Class 17 18 Pytest doctest pdb · source slide page 9.

EV0000127

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Function testing with Doctest

PES UNIVERSITY

Need for Doctest

- To check that a module's docstrings are up-to-date (to ensure code still work as documented).
- To perform regression testing (verifying the changes made to the code will not impact the existing functionalities of the Software).

Figure G000010. UE24CS151A PCPS Unit 3 Sindhu Class 17 18 Pytest doctest pdb · source slide page 10.

EV0000130

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

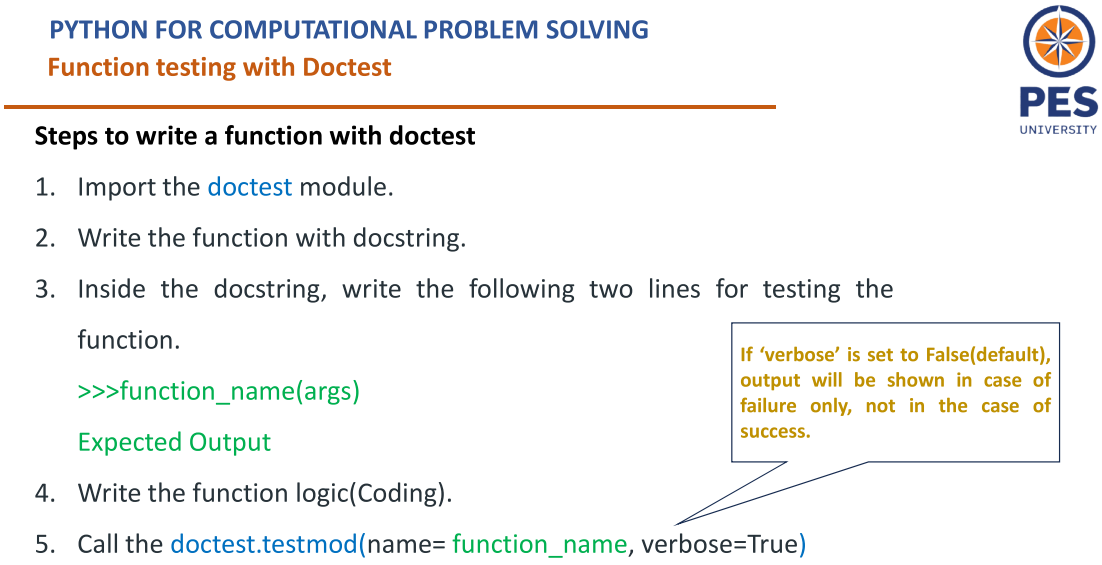


Figure G000011. UE24CS151A PCPS Unit 3 Sindhu Class 17 18 Pytest doctest pdb · source slide page 11.

EV0000133

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

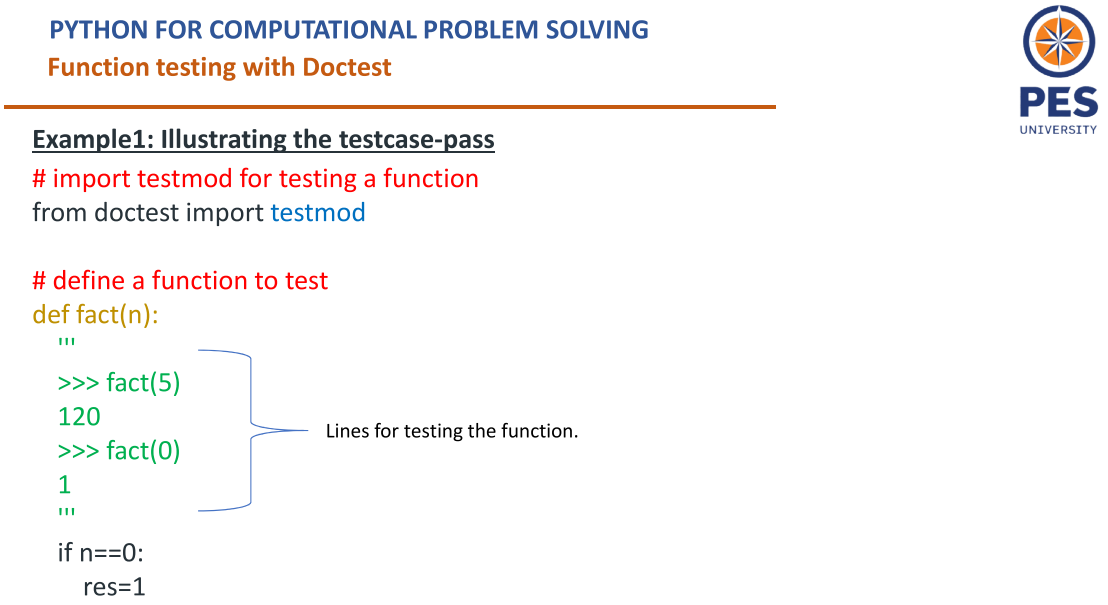


Figure G000012. UE24CS151A PCPS Unit 3 Sindhu Class 17 18 Pytest doctest pdb · source slide page 12.

EV0000136

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

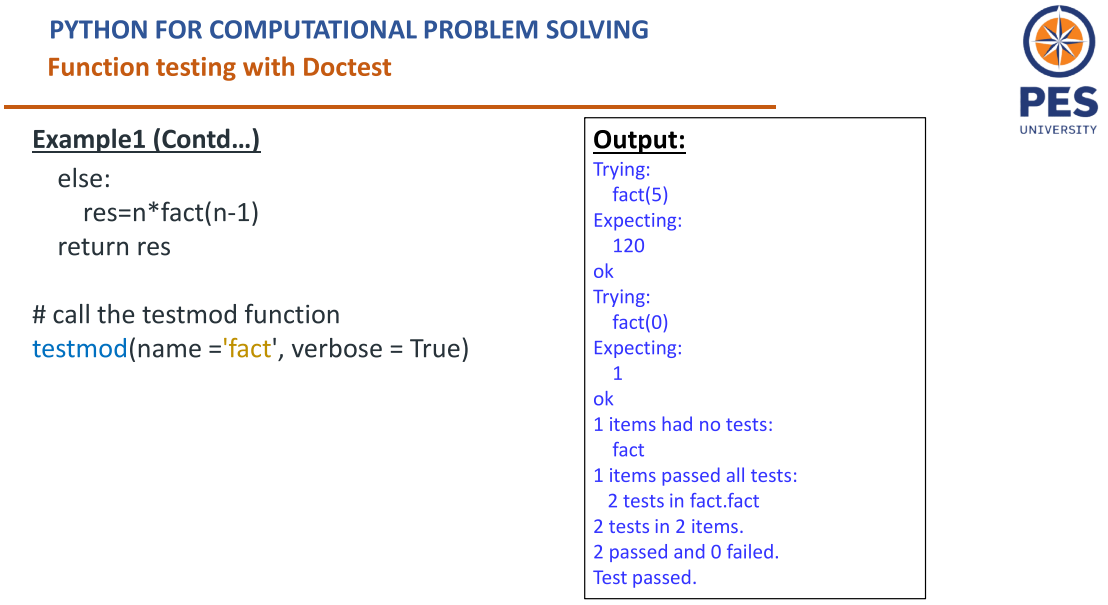


Figure G000013. UE24CS151A PCPS Unit 3 Sindhu Class 17 18 Pytest doctest pdb · source slide page 13.

EV0000139

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

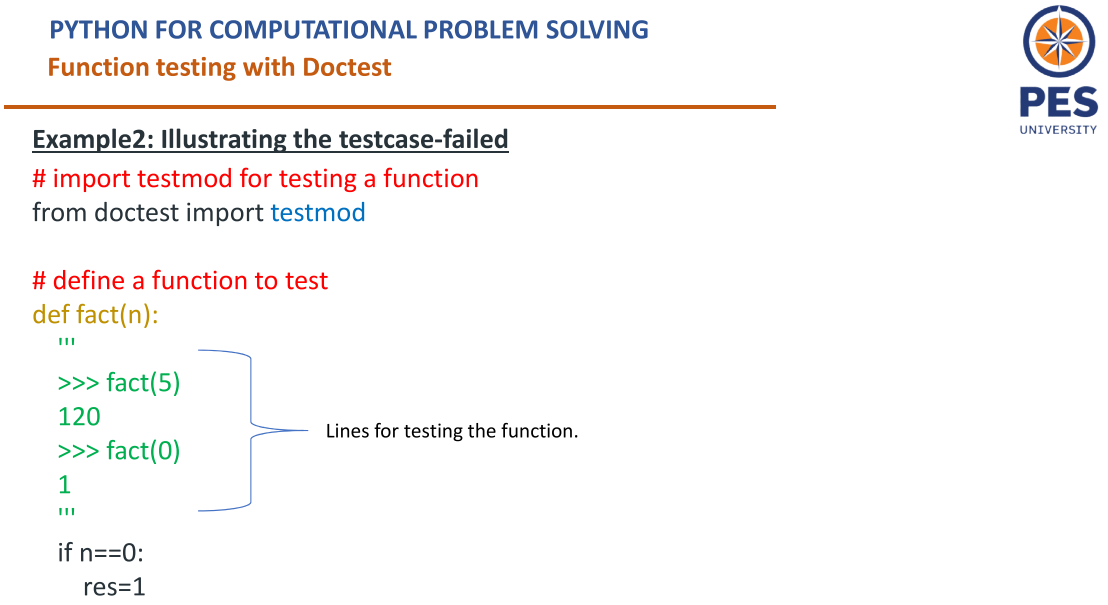


Figure G000014. UE24CS151A PCPS Unit 3 Sindhu Class 17 18 Pytest doctest pdb · source slide page 14.

EV0000142

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

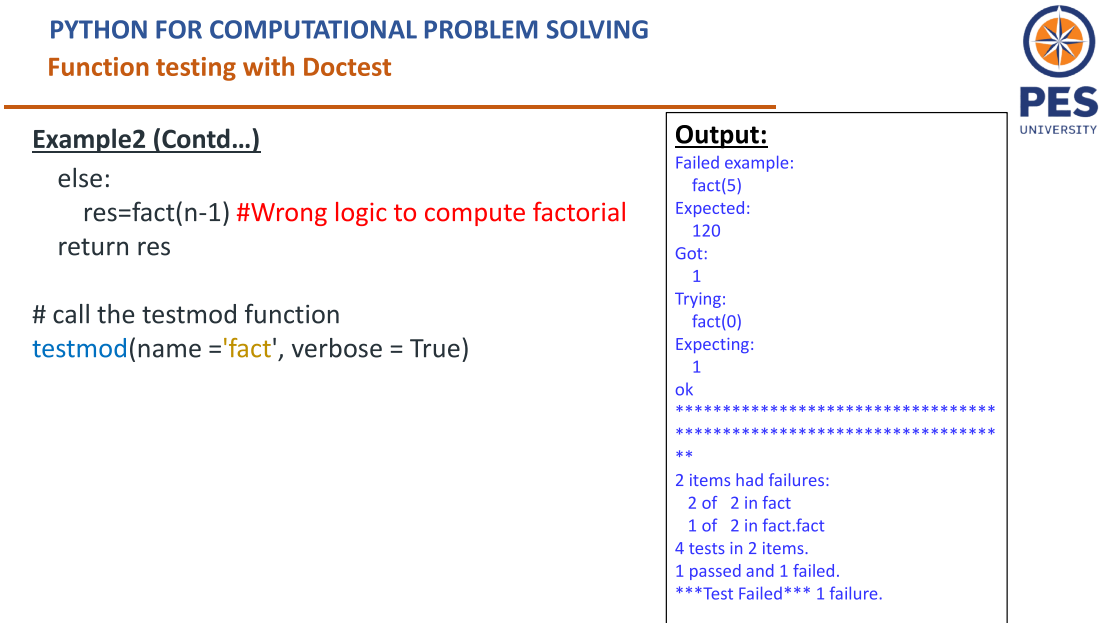


Figure G000015. UE24CS151A PCPS Unit 3 Sindhu Class 17 18 Pytest doctest pdb · source slide page 15.

EV0000145

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

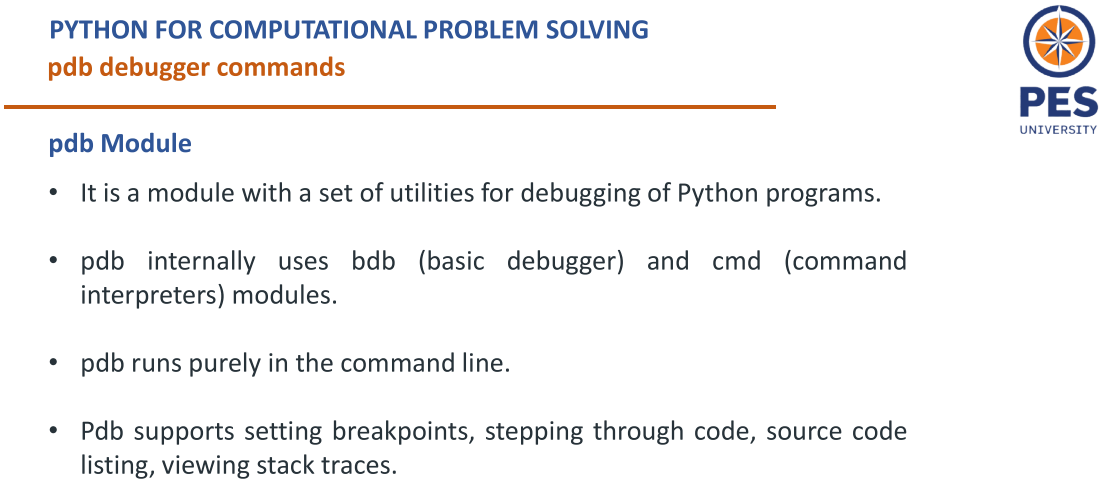


Figure G000016. UE24CS151A PCPS Unit 3 Sindhu Class 17 18 Pytest doctest pdb · source slide page 16.

EV0000148

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

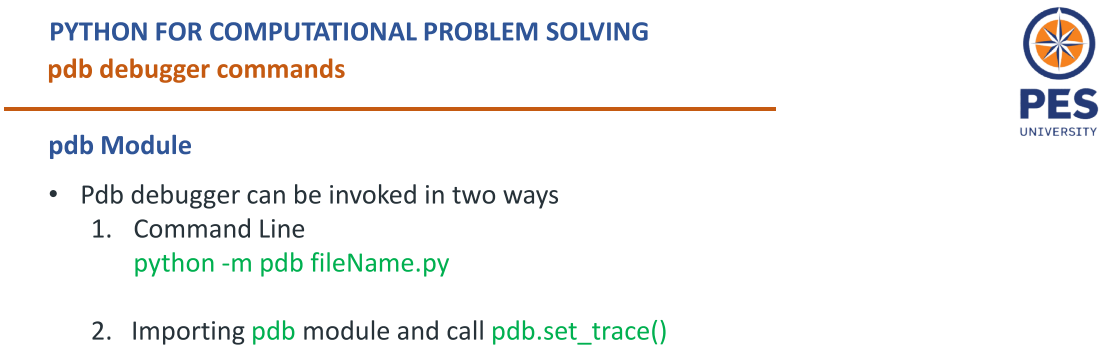


Figure G000017. UE24CS151A PCPS Unit 3 Sindhu Class 17 18 Pytest doctest pdb · source slide page 17.

EV0000151

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

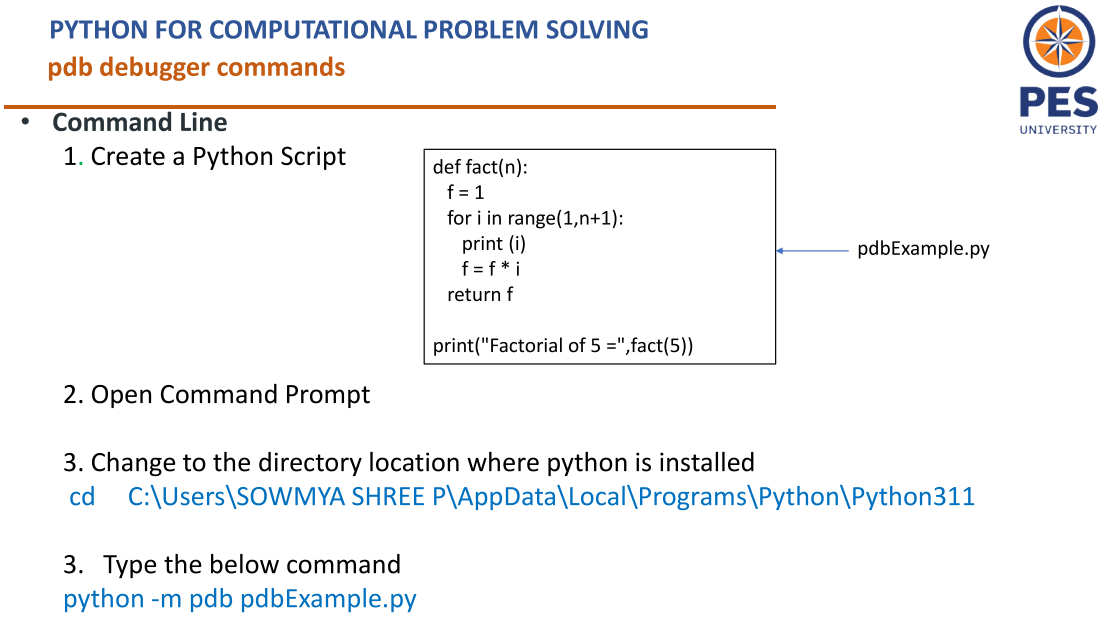


Figure G000018. UE24CS151A PCPS Unit 3 Sindhu Class 17 18 Pytest doctest pdb · source slide page 18.

EV0000154

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

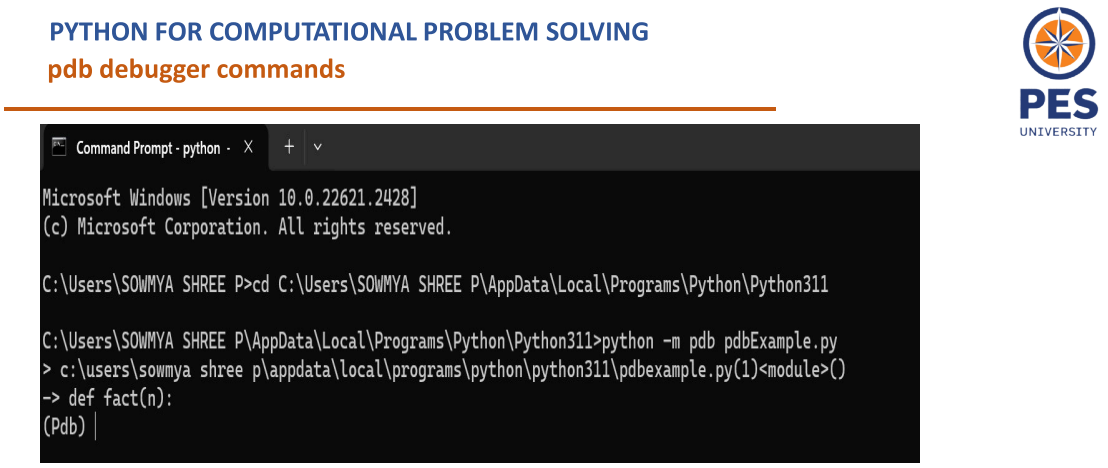


Figure G000019. UE24CS151A PCPS Unit 3 Sindhu Class 17 18 Pytest doctest pdb · source slide page 19.

EV0000157

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

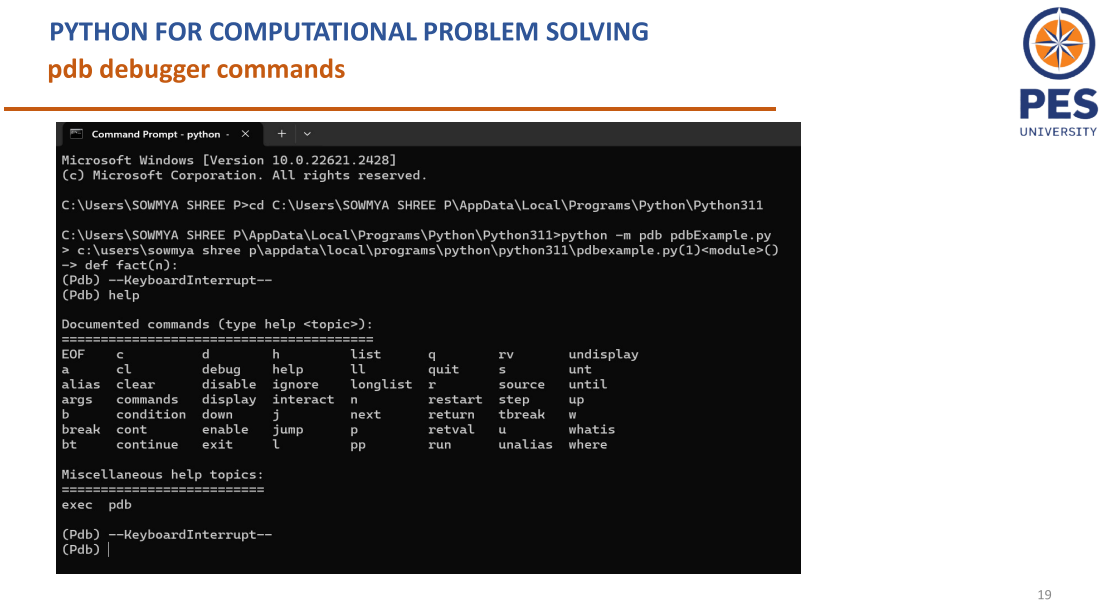


Figure G000020. UE24CS151A PCPS Unit 3 Sindhu Class 17 18 Pytest doctest pdb · source slide page 20.

EV0000160

> ***How to read it.*** Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

pdb debugger commands

PES UNIVERSITY

- list command - lists entire code with -> symbol to the left of a line at which program has halted.

```
(Pdb) list
  1  -> def fact(n):
  2         f = 1
  3         for i in range(1,n+1):
  4            print (i)
  5            f = f * i
  6         return f
  7
  8     print("Factorial of 5 =",fact(5))
[EOF]
(Pdb)
```

20

Figure G000021. UE24CS151A PCPS Unit 3 Sindhu Class 17 18 Pytest doctest pdb · source slide page 21.

EV0000163

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

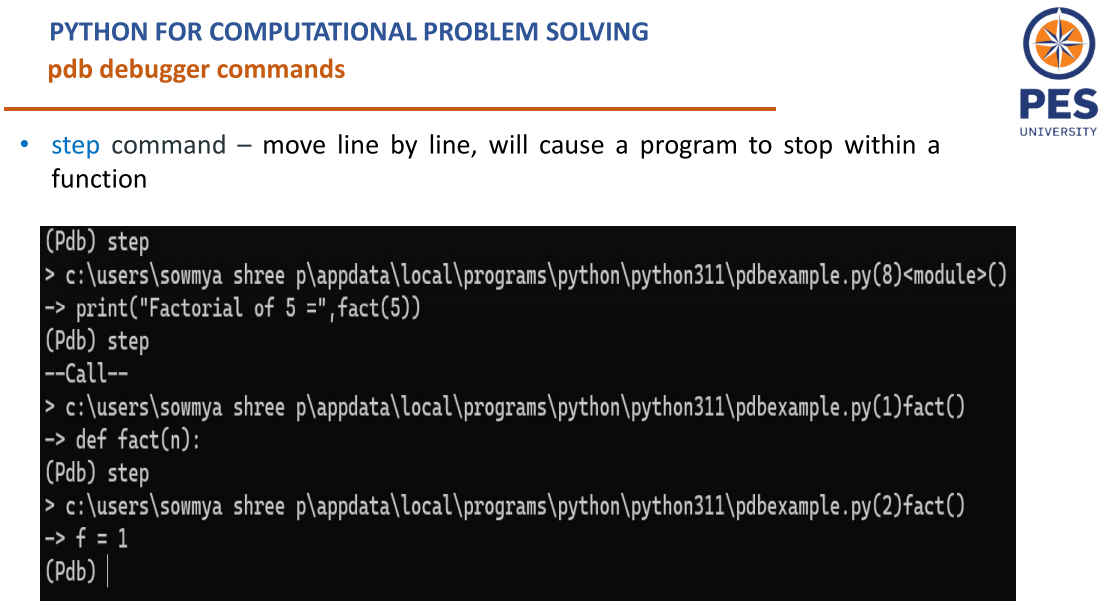


Figure G000022. UE24CS151A PCPS Unit 3 Sindhu Class 17 18 Pytest doctest pdb · source slide page 22.

EV0000166

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

pdb debugger commands

PES UNIVERSITY

- next command - move line by line, executes a called function and stops after it.

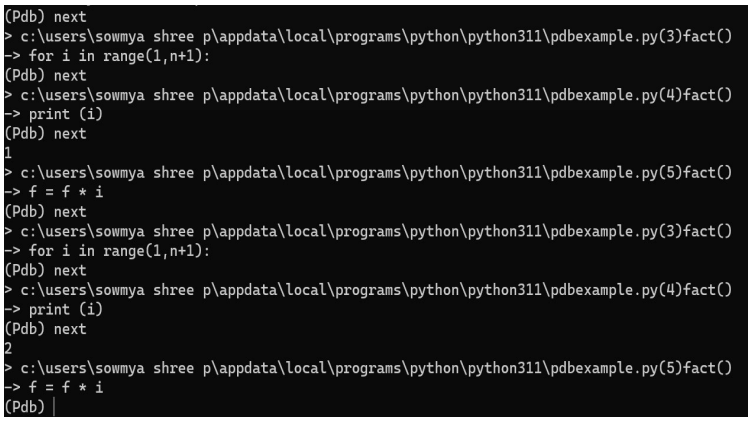


Figure G000023. UE24CS151A PCPS Unit 3 Sindhu Class 17 18 Pytest doctest pdb · source slide page 23.

EV0000169

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

pdb debugger commands

PES UNIVERSITY

- break command – set breakpoints within a program. Line number must be given.

```
(Pdb) break 4
Breakpoint 1 at c:\users\sowmya shree p\appdata\local\programs\python\python311\pdbexample.py:4
(Pdb) list
  1  -> def fact(n):
  2         f = 1
  3         for i in range(1,n+1):
  4 B          print (i)
  5            f = f * i
  6         return f
  7
  8     print("Factorial of 5 =",fact(5))
[EOF]
(Pdb)
```

- continue command – program execution will proceed till it encounters a breakpoint

```
(Pdb) continue
> c:\users\sowmya shree p\appdata\local\programs\python\python311\pdbexample.py(4)fact()
-> print (i)
(Pdb)
```

Figure G000024. UE24CS151A PCPS Unit 3 Sindhu Class 17 18 Pytest doctest pdb · source slide page 24.

EV0000172

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

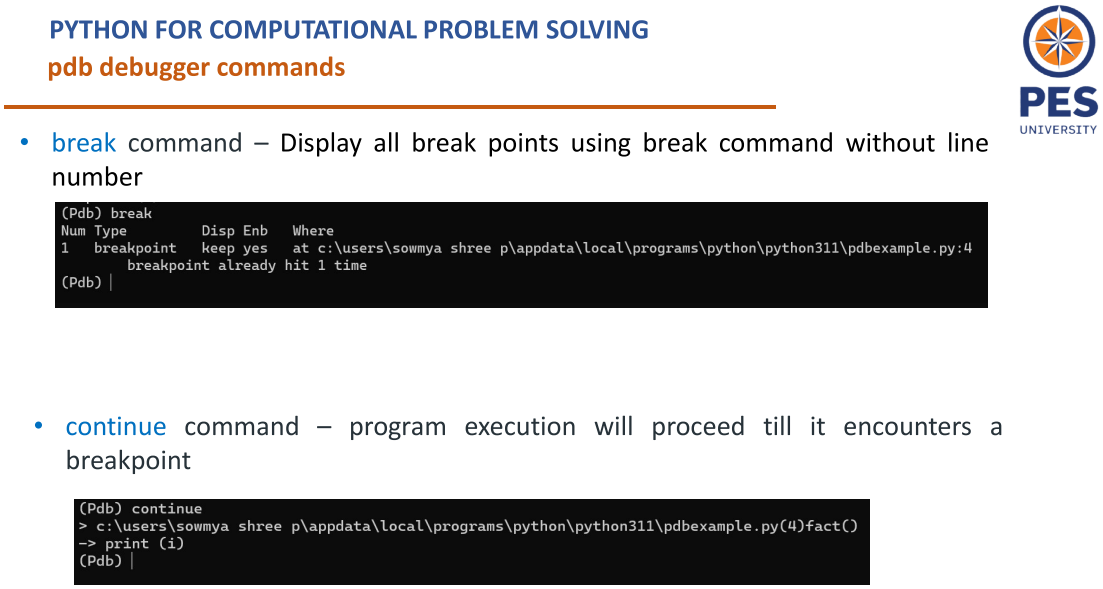


Figure G000025. UE24CS151A PCPS Unit 3 Sindhu Class 17 18 Pytest doctest pdb · source slide page 25.

EV0000175

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

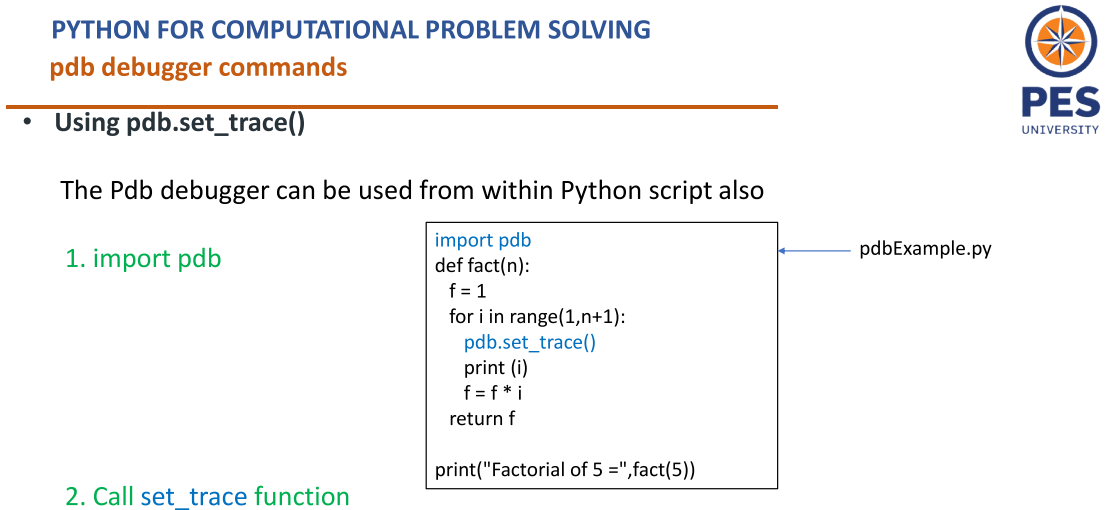


Figure G000026. UE24CS151A PCPS Unit 3 Sindhu Class 17 18 Pytest doctest pdb · source slide page 26.

EV0000178

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Testing – Pytest, Function testing with Doctest, pdb debugger commands

PES UNIVERSITY

Summary

- **pytest** – a tesing framework in Python, helps to write tests from simple unit tests to complex functional tests.
- **doctest** - a module that verifies whether the code work as intended. It allows generation of tests based on output from the standard Python interpreter shell.
- **pdb** - a module with a set of utilities for debugging of Python programs.

Figure G000027. UE24CS151A PCPS Unit 3 Sindhu Class 17 18 Pytest doctest pdb · source slide page 27.

EV0000181

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

THANK YOU

Department of Computer Science and Engineering

Prof. Sindhu R Pai – sindhurpai@pes.edu
Prof. Sowmya Shree P

Figure G000028. UE24CS151A PCPS Unit 3 Sindhu Class 17 18 Pytest doctest pdb · source slide page 28.

EV0000184

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Important points and why

1. Source slide page 1 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Testing and Debugging Prof. Sindhu R Pai PCPS Theory Anchor - 2024 Department of Computer Science and Engineering EV0000102 · EV0000103 · EV0000104
2. Source slide page 2 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Testing – Pytest Pytest • Pytest is a robust testing framework for Python. • It allows users to write test codes using Python programming language. •• ItIt helpshelps toto writewrite teststests fromfrom simplesimple unitunit teststests toto complexcomplex functionalfunctional teststests. EV0000105 · EV0000106 · EV0000107
3. Source slide page 3 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Testing - Pytest Advantages of Pytest • Free and open source • Simple syntax - very easy to start with •• RunRun multiplemultiple teststests inin parallelparallel, whichwhich reducesreduces thethe executionexecution timetime ofof thethe test suite • Automatically detect test file and test functions, if not mentioned explicitly • Allows to skip a subset of the tests during execution • Allows to run a subset of the entire test suite EV0000108 · EV0000109 · EV0000110

4. Source slide page 4 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Testing - Pytest Features of Pytest 1. Does not require API to use 2. Provides useful plugins 3. Can be written as a function or method 4. Gives useful failure information without the use of debuggers 5. Can be used to run doc tests and unit tests EV0000111 · EV0000112 · EV0000113
5. Source slide page 5 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Testing - Pytest Pytest – Environmental Setup 1. Open command prompt 2. Change directory to the location where Python is installed 3. Type command pip install pytest 4. Confirm the installation pytest -h EV0000114 · EV0000115 · EV0000116
6. Source slide page 6 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Testing - Pytest Pytest – Environmental Setup EV0000117 · EV0000118 · EV0000119
7. Source slide page 7 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Testing - Pytest Pytest – Example 1. Create a new directory (say “Automation”) and navigate into the directory in the command line. importp math 2. Create a file pytestExample.py def testsqrt(): num = 25 assert math.sqrt(num) == 5 def testsquare(): num = 73. Run the file using the command assert 7*7 == 40 pytest pytestExample.py def testequality(): assert 10 == 11 EV0000120 · EV0000121 · EV0000122
8. Source slide page 8 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Testing - Pytest Pytest – Example (Output) 7 EV0000123 · EV0000124 · EV0000125
9. Source slide page 9 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Function testing with Doctest Doctest • It is a module included in the Python programming language's standard library. • ItIt allowsallows thethe easyeasy generationgeneration ofof teststests basedbased onon outputoutput fromfrom thethe standard Python interpreter shell. • It finds patterns in the docstring. • Docstrings - provides description of a class or a function to provide a better understanding of the code. Also, used for testing purposes using doctest module. EV0000126 · EV0000127 · EV0000128
10. Source slide page 10 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Function testing with Doctest Need for Doctest • To check that a module’s docstrings are up-to-date (to ensure code still work as documented). •• ToTo performperform regressionregression testingtesting (verifying(verifying thethe changeschanges mademade toto thethe codecode will not impact the existing functionalities of the Software). EV0000129 · EV0000130 · EV0000131
11. Source slide page 11 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Function testing with Doctest Steps to write a function with doctest 1. Import the doctest module. 2. Write the function with docstring. 3. Inside the docstring, write the following two lines for testing the function. If ‘verbose’ is set to False(default), output will be shown in case of >>>function_name(args) failure only, not in the case of success. Expected Output 4. Write the function logic(Coding). 5. Call the doctest.testmod(name= function_name, verbose=True) EV0000132 · EV0000133 · EV0000134
12. Source slide page 12 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Function testing with Doctest Example1: Illustrating the testcase-pass # import testmod for testing a function from doctest import testmod # define a function to test def fact(n): ''' >>> fact(5) 120 Lines for testing the function. >>> fact(0) 1 ''' if n==0: res=1 EV0000135 · EV0000136 · EV0000137

13. Source slide page 13 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Function testing with Doctest Example1 (Contd...) Output: Trying: else: fact(5) res=n*fact(n-1) Expecting: return res 120 ok Trying:Trying: # call the testmod function fact(0) testmod(name ='fact', verbose = True) Expecting: 1 ok 1 items had no tests: fact 1 items passed all tests: 2 tests in fact.fact 2 tests in 2 items. 2 passed and 0 failed. Test passed. EV0000138 · EV0000139 · EV0000140

14. Source slide page 14 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Function testing with Doctest Example2: Illustrating the testcase-failed # import testmod for testing a function from doctest import testmod # define a function to test def fact(n): ''' >>> fact(5) 120 Lines for testing the function. >>> fact(0) 1 ''' if n==0: res=1 EV0000141 · EV0000142 · EV0000143

15. Source slide page 15 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Function testing with Doctest Output:Example2 (Contd...) Failed example: else: fact(5) res=fact(n-1) #Wrong logic to compute factorial Expected: 120 return res Got: 1 Trying:Trying: # call the testmod function fact(0) testmod(name ='fact', verbose = True) Expecting: 1 ok ********** ********** ** 2 items had failures: 2 of 2 in fact 1 of 2 in fact.fact 4 tests in 2 items. 1 passed and 1 failed. ***Test Failed*** 1 failure. EV0000144 · EV0000145 · EV0000146

16. Source slide page 16 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING pdb debugger commands pdb Module • It is a module with a set of utilities for debugging of Python programs. • pdb internally uses bdb (basic debugger) and cmd (command interpreters)interpreters) modules.modules. • pdb runs purely in the command line. • Pdb supports setting breakpoints, stepping through code, source code listing, viewing stack traces. EV0000147 · EV0000148 · EV0000149

17. Source slide page 17 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING pdb debugger commands pdb Module • Pdb debugger can be invoked in two ways 1. Command Line python -m pdb fileName.py 2. Importing pdb module and call pdb.set_trace() EV0000150 · EV0000151 · EV0000152

18. Source slide page 18 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING pdb debugger commands • Command Line 1. Create a Python Script def fact(n): f = 1 for i in range(1,n+1): print (i) pdbExample.py f = f * i return f print("Factorial of 5 =",fact(5)) 2. Open Command Prompt 3. Change to the directory location where python is installed cd C:\Users\SOWMYA SHREE P\AppData\Local\Programs\Python\Python311 3. Type the below command python -m pdb pdbExample.py EV0000153 · EV0000154 · EV0000155

19. Source slide page 19 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING pdb debugger commands Now, type help in front of the debugger prompt to know more about any command. EV0000156 · EV0000157 · EV0000158

20. Source slide page 20 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING pdb debugger commands 19 EV0000159 · EV0000160 · EV0000161

21. Source slide page 21 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING pdb debugger commands • list command - lists entire code with -> symbol to the left of a line at which program has halted. 20 EV0000162 · EV0000163 · EV0000164

22. Source slide page 22 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING pdb debugger commands • step command – move line by line, will cause a program to stop within a function EV0000165 · EV0000166 · EV0000167
23. Source slide page 23 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING pdb debugger commands • next command - move line by line, executes a called function and stops after it. EV0000168 · EV0000169 · EV0000170
24. Source slide page 24 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING pdb debugger commands • break command – set breakpoints within a program. Line number must be given. • continue command – program execution will proceed till it encounters a breakpoint EV0000171 · EV0000172 · EV0000173
25. Source slide page 25 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING pdb debugger commands • break command – Display all break points using break command without line number • continue command – program execution will proceed till it encounters a breakpoint EV0000174 · EV0000175 · EV0000176
26. Source slide page 26 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING pdb debugger commands • Using pdb.set_trace() The Pdb debugger can be used from within Python script also import pdb pdbExample.py 1. import pdb def fact(n): f = 1 for i in range(1,n+1): pdb.set_trace() print (i) f = f * i return f print("Factorial of 5 =",fact(5)) 2. Call set_trace function The behavior of the debugger will be exactly the same as we find it in a command line environment. EV0000177 · EV0000178 · EV0000179
27. Source slide page 27 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Testing – Pytest, Function testing with Doctest, pdb debugger commands Summary • pytest – a tesing framework in Python, helps to write tests from simple unit tests to complex functional tests. • doctest - a module that verifies whether the code work as intended. It allows generation of tests based on output from the standard Python interpreter shell. • pdb - a module with a set of utilities for debugging of Python programs. EV0000180 · EV0000181 · EV0000182
28. Source slide page 28 (extracted text): THANK YOU Departmentp of Computerp Science and Engineeringg g Prof. Sindhu R Pai – sindhurpai@pes.edu Prof. Sowmya Shree P EV0000183 · EV0000184 · EV0000185

SHORT-NOTE RECAP

Transcript block 1 (source interval 000000-000299 seconds): [00:00:00.164] Now I am going to demonstrate the value written from the normal function has to be tested in the test method of the PyTest product. So how to do this? So in order to do this we need to create a file. So how to create the file we already know we have to use the command g [00:00:17.808] Edit followed by the file name. With the file name in the test, framework has to be start with the keyword test followed by [00:00:26.788] File name I will give the file name as example two dot extension should be dot p y. So this we open the editor so where actually you can write your code. So if I want to import the PyTest framework in the program so I know I have to use [00:00:43.296] the statement import followed by pi test this particular statement will import the py test framework in your program now i want to write the normal function we already know to write the normal function we have to use the keyword df [00:00:59.936] followed by function name. So I will use the function name as F UNC and I will pass only the one parameter. If you want to pass two parameter also you can pass

but I will use only one parameter. Then so after this I want to return the value from this function. [00:01:17.408] We know to return the value from this function we have to use the statement written and I will give the statement as x plus one. So after writing the normal function now I will write the test method of the pie test. So how to write the test method of the pie test? We know we have to use the keyword [00:01:36.036] test followed by the method name. So after writing this now I want to check the value return from the normal function that is F U N C. Now to do this we know we have the statement in the PyTest framework that is asset followed by the condition. So I will [00:01:55.268] keep the condition f u and c because I have used the normal function as f unc and I will pass only one value because in the function definition I have given only one parameter that's why we have to pass only one value. I will pass the value as two [00:02:12.224] then I will check with the value is equal to t and I will give the statement within a double course test passed. So after giving this type of statements I will say the program using control and [00:02:26.276] and I will put control two. So how to run the program in the PyTest framework we know we have to give the command py dot test [00:02:36.484] Py dot test followed by the file name where the file name is test underscore example two dot two one so this particular statement execute the program see in the output it's collected one item the test has passed because [00:02:53.360] See the code. So in the core, so we have passed the value 2, that 2 will go to a value of x. X takes the value of 2, 2 plus 1 is 3, yes, 3 is equal to 3 is test got passed. So that's why in the output you got 1 pass. [00:03:08.772] 0.01 seconds. In this way we can check the value written from the normal function can be checked in the test method of the pie test framework. Now I am going to demonstrate how to run the multiple files in the pie test framework. [00:03:23.844] so in order to do this we need to create the two files I will create one file g ed forward by test and example one is the first file so and I will write some code in this so import [00:03:41.764] Then I will write the test method using B followed by test underscore method one. [00:03:48.996] Then I will write some quote here that is I will write x is equal to 2 and I will write y equals 3 how to check the values the condition we know we have the statement I said followed by the condition I will use x not equal to y. [00:04:07.268] and I will write the statement test got passed so after this I will say the program and I will exit from this now I will go into create one more file g edit [00:04:22.084] underscore example 2 dot p and I will import the py test framework in my program using the command input py test and I will write some function using the statement bf test underscore method [00:04:42.052] 2 and I will write one more statement here. x is equal to 2 and y is equal to 2 and I will check using the statements. [00:04:55.204] A S is a set and X is equal to Y and I will write some statements in the double quotes test got passed and I will say the program and I will exit from the program. So now I want to test these two files.

EV0000002 · EV0000003 · EV0000004 · EV0000005

Transcript block 2 (source interval 000300-000599 seconds): [00:05:14.436] test we knew we have to use the command py dot test [00:05:19.492] followed by the file names how many file names I have created I have created two files if you have created a three files we are supposed to give all the file names in the command line so I have created only two I will give the two file names what is the first file name the first file name is test [00:05:36.016] underscore example one dot p y give the space next give the second file name where the second file name is test underscore example two dot py so if you have created n you can give the n come file names here [00:05:51.460] and you can enter so see in the output is giving collected two items this indicates that we are executing the two test cases because in the first file we have the one test method and in the second [00:06:08.772] So

we have the one more method. So totally two test method we have executed. So that's why in output it's showing we have executed that is we have collected two items, which are the two items we have collected, one is from test. [00:06:22.160] example 1 another is from example 2. So that is why the output it showing we have executed two test cases. So two pass in how many seconds that is 0.01 seconds. Like this we can run n [00:06:39.204] [ASR low confidence; check audio] Pytest Program. [00:06:41.700] Suppose if I want to test thousand files, I can't give all the thousand files in a command line like pi dot test followed by file one, file two, file three can't give. So then how to do this? So we know that we have to give the command pi dot test. Suppose if I give just pi dot [00:07:00.100] what will happen so it will execute all the programs that is there in your folder so now I can't make [00:07:08.804] what actually it's going on what is the output of this. So in order to avoid this what we can do so we have to create my own directory by giving the command mkdir followed by [00:07:22.564] I will give the directory name. So this particular command will create the directory where you can dump all your program into this directory. [00:07:33.252] navigate into this directory. Now we have to give the command C D followed by testing. So this particular command will navigate into the directory which you have created. So after navigating into this directory now you can create a web file using the command [00:07:51.716] g edit followed by test and file name example one dot pin by now you can write your code pimport by dot test [00:08:06.916] Then I will write the function definition df followed by test method 1. Then I will write some statements like x is equal to 2 and I will write y is equal to 2. [00:08:24.964] After this I will check the connution using the statement asset followed by X equals Y and I will write the statement state got [00:08:37.604] So after this I will say the program and I will execute. Now after this I will create one more file. So G ed test underscore example2 dot py then I will write some piece of code [00:08:54.948] in this file also import pipe test and I will write the test method using the code bf test underscore method two [00:09:11.332] and I will write some statements x is equal to 2 and I will write y equals 3 and I will check using the statement I said x is not equal to y and I will write the message test got [00:09:32.004] and I will save it and I will come out of the editor now I want to run like this the n number of files I can't give like i dot test test example one [00:09:47.364] dot p y like this I can't give 1000 files if I created a thousand five so what actually you can do how to avoid this so in order to avoid this just you give the command pi dot text and press enter so it will execute all the

EV0000006 · EV0000007 · EV0000008 · EV0000009

Transcript block 3 (source interval 000600-000899 seconds): [00:10:02.980] that is there in that directory that is testing seeing the proof you got collected two items this indicates that you have executed the two test methods in the file test example one and test example two and both got passed in [00:10:19.108] 0.01 seconds so like this you can execute n number of test methods in n number of files by giving the command pi dot test

EV0000010 · EV0000011 · EV0000012 · EV0000013

Full transcript: 1615 normalized words, 42 timestamped segments, 629.717333 seconds duration. Low-confidence segments flagged: 1. Review locators: 00:06:39.204

EV0000002 · EV0000003 · EV0000004 · EV0000005

3 Pytest, doctest, pdb

This lesson is indexed from the source video titled Pytest, doctest, pdb. Its complete timestamped transcript is preserved below in segment order. The original video path and content hash are recorded in datasets/lecture-corpus.json. Transcript text is source wording; it has not been silently corrected or rewritten. A candidate slide deck is grouped here according to the current matching score; the exact path and page locators are listed in the source evidence. EV0000014 · EV0000015

SHORT-NOTE RECAP

Transcript block 1 (source interval 000000-000299 seconds): [00:00:00.196] Now I am going to demonstrate how to use this PDB debugger in your program. So in order to do this we know we have to use the statement import pdb. So how to do this I am going to demonstrate now. So first you create the file gedit file one [00:00:19.236] So it will give you the editor where you can enter your program. So in order to import the PDB debugger, we know we are supposed to give the statement import followed by PDB. This particular statement will import the PDB. [00:00:36.928] module in your program. So after this I am going to write my program that is factorial of a given number. So we know we are supposed to use DEF for a function now followed by function name fact of A. So now after this I am [00:00:54.276] assign the value fact equals 1 so now I'm going through all the value using for loop for i in range 1 comma eight plus 1 [00:01:07.652] So now if I want to go through each and every statement in your for loop what will happen I can see by using the function set trace that is available in the PDB in order to access this we have to give module name [00:01:25.072] That is PDB followed by dot followed by the function name that is set trace. If you give this statement, it will access the set trace method from the PDB module. So after this, I am going to print the value of I [00:01:43.172] so now I'm calculating the factorial of a number fact equals fact fact equals fact into i then i'm going to return the factorial written fact [00:02:02.372] So after returning the factorial of a number, now I'm going to call my function from the main. So by giving the statement if n equals main. So I'm going to call my function that is fact. [00:02:22.308] So and I'm going to print the value of a given number using the statement print and factorial of [00:02:35.588] In number, I am taking my number as 3, I am calling the function fact, I am going to pass the value as 3 because I want to calculate the factor of 3. So after this I have to say control S. [00:02:52.324] and have to exit my program so control Q now I am going to give the command python followed by file name.py if you are using the second version give python followed by file name.py [00:03:07.968] If you are using the third version, you are supposed to give python 3 followed by file name.py. Enter. So you will enter into the pdb debugging. So like this way I can use the debugger in my program. [00:03:24.036] Now we will see how to check the variables that is present on the stack. So using debugging we can able to check the program stack and the variables that are stored on that stack during the program execution. So in order to achieve this we have the command where and how to know [00:03:41.808] this command in order to the variables that is present on the stack I am going to demonstrate now. [00:03:47.748] So now we will see how to print the value of the variable. So in order to achieve this we are supposed to give the command p followed by the variable name in the command dropped. So how to do this I am going to demonstrate now. [00:04:03.044] to print the value of a variable so first we need to create one program using g edit g edit followed by variable is my file name with the extension dot py

[00:04:17.936] So now I will write my program. So how to import the PDB? We know we are supposed to give the statement import followed by PDB. So next I'm use the variable A with the value 10. [00:04:32.368] And how to trace we know we are supposed to give pdb dot set trace is a method that is available in the pdb. So this will trace each and every line what will happen in my program next I'm print my [00:04:48.128] variable using the statement print and the variable is a and I will say my program control s control q is I will quit my program now I am going to execute we know how to execute the program python followed by

EV0000014 · EV0000015 · EV0000016 · EV0000017

Transcript block 2 (source interval 000300-000599 seconds): [00:05:07.172] variable dot p y so this will enter into my pdb debugger now if I want to print the value of the variable what is my variable have used in my program have used the variable a if I want to [00:05:25.424] No the value of that variable A I have to give the command P followed by variable name where the variable name in my program is A and you press enter you will get the value of that variable that is A. So like this way using debugger I can print the value of the variable.

EV0000018 · EV0000019 · EV0000020 · EV0000021

Full transcript: 832 normalized words, 21 timestamped segments, 344.106667 seconds duration. Low-confidence segments flagged: 0.

EV0000014 · EV0000015 · EV0000016 · EV0000017

4 Python Classes and Objects

This lesson is indexed from the source video titled Python Classes and Objects. Its complete timestamped transcript is preserved below in segment order. The original video path and content hash are recorded in datasets/lecture-corpus.json. Transcript text is source wording; it has not been silently corrected or rewritten. No slide deck is assigned to this video; candidate slide sources remain separate in the unit master. EV0000022 · EV0000023

SHORT-NOTE RECAP

Transcript block 1 (source interval 000000-000299 seconds): [00:00:01.732] Hello, welcome back. Python for computational problem solving. In the previous class uh we discussed uh the concepts of object oriented programming. Today we will try to learn how do we create a class. [00:00:21.124] How do we uh work with the class? Let's get started. [00:00:29.188] So how do I create a class? [00:00:36.484] In order to create a class, I need to make use of a keyword called as class. I need to make use of a keyword called as a class. Followed by you will also have to specify what is the name of your particular class. [00:00:54.116] You can give your name itself as a name of the class, but [00:00:58.052] Uh it is always better to name it in a proper convention. The reason is whenever you are coming back in order to look at for uh look at your program and again, it should make sense in order to tell that what exactly you were trying to do. [00:01:14.624] Okay, so as I'm just giving you some examples here, I'm going to call it as what? Example one. [00:01:22.980] The name of my class is example one. Now what can go into the class? I will be telling a step by step at this point. Try to understand that I do not want to do anything. That's why I say it as a pass. Okay, pass indicates that you do not want to do anything. [00:01:41.220] Fine. Next, what I will be doing,

nothing else, I'll just say a print. I'll tell what is the type of my class. Okay. Type of my class. [00:01:58.756] The reason is we have said the class that you are going to be creating is a type. [00:02:04.420] Okay? Because type specifies what you have to do, but it never specifies how you have to do. Uh a small correction in the previous video I looked in uh the spelling of specifies was written wrong. Please uh correct it. [00:02:19.108] Okay. So now we're trying to find out what is a type of EX1. Okay. So it is a small letter here. So let me quickly rectify it and uh let me run this. Okay. So hope you know how to run the program. Python. [00:02:35.780] Class. What is I have said it as class itself. So we will use the same thing. Okay? And find out. So what is the type of EX1? EX1's type is type itself. Okay? Because [00:02:50.276] I need to create my own type. I'm trying to create my own type. Okay. So this is our first uh program. How do we create a class? [00:03:01.924] Okay so now we will extend this. We will extend this in order to make our life simple, let me comment it like this. [00:03:11.684] Okay, so that uh it is easy for you to look at uh uh and you are going to be getting only the output for that particular program. Okay, so now the next point that we have to understand is a class has a behavior. Class has a behavior. [00:03:35.364] Okay, a class has a a behavior. Okay. Now what does that mean? We'll look with an example. So I will say class EX2 is what I will say because we are into the second example. Right? And I will create a function. [00:03:53.636] Fine. So the name of the function is F1 in this case. I do not want to do anything other than just writing a print statement. Okay. This is fine. So I'll say in F1. Nothing else that I want to do. [00:04:13.732] Okay. So now as this function is a part of this particular class, how can I access this function outside the class? I can access it by making use of a function name. Okay. Dot the function name. Sorry. [00:04:32.932] Class name dot the function name. Okay. So this is what I will try to do it. Okay. So what did we say? We said that a class has a behavior. Fine. So what exactly is a behavior? Behavior in the sense it's a function. [00:04:50.404] And if I say what is the uh behavior of a student, he has to listen to the class. Okay? So that's the behavior of a particular student. What is the behavior of a teacher? To teach. That's the functionality. So similarly, in this case,

EV0000022 · EV0000023 · EV0000024 · EV0000025

Transcript block 2 (source interval 000300-000599 seconds): [00:05:06.272] A class has a behavior and the behavior that I defined as what? The function f1. So now how will I access this particular function which is a part of a class EX2? It has to be done using your ex2 dot f1. [00:05:21.904] Okay, so hope uh this is clear to you. This I call it as invocation of a function. A function is getting invoked. Okay, so hope uh this is clear to you. So let me remove this and put it here. [00:05:40.420] Okay. So we'll go ahead with the next one. The next one is a class also has [00:05:50.916] Attributes. [00:05:54.596] Class also has attributes. Okay. Fine. Attributes in the sense fields. Variables. It can also be a field or it can be a variable. So how do we model this? Clause, let me do it as EX3, because this is my EX3. [00:06:14.084] Fine. So now A is equal to class something. Okay because this is my variable. Class concept something. [00:06:28.196] Hope uh this is fine for you. I do not want to do anything else in this, so let it be as simple as possible. So we'll say print. Okay [00:06:39.588] Um I have to access A, but it is a part of which class? It's a part of EX3 class. So EX3 dot A will fetch me whatever the value that has been assigned to a variable in this case. [00:06:57.892] Quickly executes. So let me execute this. It says that class concept. Okay. Uh maybe I did not execute the uh output for the previous one. So let me comment till here itself. [00:07:17.444] We will see the output that I'm expecting is what in F1 because when on this example is executed and uh here I need to get the output called as class concept. So two outputs

we are expecting, let's check. Okay in X F1 and then class concept. Okay, hope it is clear to you. [00:07:39.012] And uh one thing that should always needs to be remembered is the indentation. [00:07:47.204] Fine. It's an header. Important thing is the indentation. This is called as an header. Okay? So the moment you have an header, the suit will follow what? The suit will follow the required indentation. By default, [00:08:02.656] I always use indentation, which is going to be the tab itself. So I've used an indentation which is a tab in this case. Okay. This statement says it is out of the class. Anything within the class with has uh with some indentation. [00:08:18.128] I say that this is a body of the class or the suit. Okay? S-U-I-T, we usually pronounce it as what? Suite. Okay? Suite of the class. And this is called as an header. Anything that ends with the colon, hope you remember that it is called as what? An header. [00:08:35.012] Okay, here also this is an header. Now this is a body of this. This itself becomes an header, and this becomes a body of this particular header, and so on. Okay, so hope it is clear to you. Let's uh move on to the next one. [00:08:55.428] Okay, so let me move it a little ahead so that uh it's easy for us to see. Okay. What we have done, we created a class, we created a behavior for the class, we also created an attribute for a particular class. Now what we will try to do is we will create an [00:09:13.316] variable for a particular class. Okay? We create a variable for a class which is of type of that particular class. What does that mean? See, when you when you I when I say you are a PAS student [00:09:31.684] Okay. You are a PES student. And suppose you belong to A section. [00:09:39.460] You belong to A section. Just an example that I'm trying to say. So now in this case, how many number of students can be there for this particular A section? Maybe sixty. Okay. I'll just assume it as it is sixteen students. [00:09:53.924] Okay. How I will name each of the students. At this point I may say role number one, role number two, role number three, four, and so on. Okay. So now

EV0000026 · EV0000027 · EV0000028 · EV0000029

Transcript block 3 (source interval 000600-000899 seconds): [00:10:05.092] I can say that section A is a class which has so many number of variables, and these variables are of type A section. Okay, and these variables are also called as objects. Variable of [00:10:25.476] A section type. This is also called as an object. It is also called as an instance. Okay? So now what are we going to be doing? We are going to be creating an instance. So I will say class EX4. [00:10:42.500] Okay? Do you want to do anything? No, I don't want to do anything at this point. So let it go like this. So now how will I uh create the object or an instance for a particular class? So I will say [00:10:56.900] Uh as we are talking about A section. Fine. So I will also take up an example called as A is equal to I need to make use of an EX4, which is the name of the class itself. Okay, and this is the [00:11:13.284] Syntax that you have to follow whenever you're trying to create an object. Okay? So object created. [00:11:24.932] creation. Hope this is uh clear to you. Okay so A is equal to what? This. Fine. So now what we will try to do is we'll try to print this. Print type of A. Let's find out what is a type of A [00:11:43.748] Okay. So let me go back here, execute this program. Okay. Type of A is what? It A belongs to your EX4 class and it is a part of your main class. [00:12:00.932] So hope uh you have uh understood this. Fine. The moment uh you create a class [00:12:11.364] Hope uh all the concepts are clear to you till now. So let me continue. Okay. So what we have done is we have created a class. Okay? A class, it doesn't have any implementation. I just created an object. Okay. So the moment you create an object. [00:12:28.964] Internally internally, there is a special function that gets created. [00:12:36.260] Okay, there is a special function of a class gets created. Okay, the moment you create or initialize an object. Okay? And this special function is called as underscore, underscore, init underscore, underscore. [00:12:55.652] Okay, so this is a special

function. You should always remember this, even though you do not need to create at this point internally, what has happened? Internally there is an underscore, underscore in it, underscore, underscore. [00:13:10.464] It's a special function, okay so now it is called as a special function. It gets created. So hope this is clear to you. So when I create an object, and I create an object like this, A is equal to EX4 of this function. Okay, so now. [00:13:30.596] in conceptually or internally what exactly this means. This means that there is a function, special function gets created which has an argument A. [00:13:44.580] Okay so even though you write it like this, conceptually it looks like this. Okay, conceptually it looks like this. Okay, so I hope uh this is clear to you. Okay, fine now. [00:14:01.924] We will complete this particular program. We will explicitly create underscore underscore in it underscore underscore. Okay. So hope this is clear to you. So we have created an object here. [00:14:13.200] Okay, once an object is created, internally there is a method that gets executed, which is called as what? A special function, and we will give a name to the special function, and the name of the special function is called as constructor. [00:14:32.644] Okay. It does a constructive job. Okay. So what does a constructive job it does? We will try to understand with an example. So hope this is clear to you. Let me remove the comment and put it here. Okay. [00:14:50.532] Right, so this is what I will try to do. Let me create a class. Okay, class ES4, and then I will say define underscore underscore in it underscore underscore.

EV0000030 · EV0000031 · EV0000032 · EV0000033

Transcript block 4 (source interval 000900-001199 seconds): [00:15:07.524] Okay, because I said a special function that gets uh created. Oh what do we print? Nothing. Okay, print constructor. Okay. We usually say it as constructor. Instead of typing such a big word, I just said it as construct. Cter. Okay? So now [00:15:27.172] What do I do? I'll just create a object. Fine? Of type what? Clause EXP. Nothing else that I really want to do here. [00:15:38.084] Okay, so we have already said conceptually how does it look like underscore underscore in it underscore underscore of A. Okay, so now let's execute to find out what happens. [00:15:53.092] And if you can see it throws me an error. Fine? It is throwing me an error. The reason is internally it looks like this, right? It looks like underscore, underscore of A. [00:16:06.308] Okay, so an instance is passed as the first argument to this particular constructor. Let me repeat, when you do this, the instance itself is passed as a first argument. [00:16:20.132] Okay, though this argument is implicit, it is happening implicitly, I don't need to do it, but it always expects me to specify the parameter explicitly. [00:16:32.196] Okay so because of which what do I need to do? I need to pass something. Let me repeat once again, the moment I create an object, internally the constructors gets executed. [00:16:44.800] Fine, even though this constructor has an imp uh implicit in nature, but what will happen is the object or an instance becomes [00:16:56.260] As an argument. Okay? It's an argument. So I am passing an argument. There is nobody here to hold my argument. Okay? Because of which I have to explicitly specify a particular parameter. Okay? I can specify a parameter. [00:17:14.084] Okay. So now let's execute this. Let's execute. [00:17:21.380] There is no problem with this. It's absolutely working fine. Okay so now you can specify any parameter. You can give any name to the parameter, but by practice we always use something called as self. [00:17:37.540] Okay. This is uh normally it we use it. Okay there is no role because we uh previously we executed with the parameter itself. So it worked out very fine. There was no harm in executing, but why self? Self is generally used. It's a [00:17:53.744] It's by practice we can say. Okay. So hope you uh are all fine with this particular concept. Okay. So Y cell normally called, nothing else. You use your own name, it is absolutely fine. Okay? So let me just extend this.

[00:18:11.812] Fine. And uh we will try to identify what we are trying to say. Okay. Let's print self here. Okay, and uh let me print A here. [00:18:24.900] Okay because I said internally it is passed as what? A, which means self is A. A is self. [00:18:34.148] Okay. So what is the output that I'm expecting? The output that I'm expecting is both of them should print and give me the same answer. Okay? So let's find out, will it give me or not? [00:18:48.260] Okay, and if you can see both of them are pointing to the same ones, which means A is same as self, self is same as A. Okay, so hope uh uh this is uh clear to you. I hope there are no doubts in this case. [00:19:06.916] Okay. So let me just go ahead. This uh uh probably this is the last example uh that I would be taking in order to explain the concept of uh creation of class. Okay so let's quickly do it. Class uh rectangle [00:19:30.628] Sorry if there are any uh spelling mistakes. It it's not intentional. Okay. So define why do I need to do init init method? It's a constructor. Okay. So what should go to my init method? Always [00:19:46.500] The first parameter should be what? To hold the instance of this particular class. Okay? As I'm talking about rectangle, I also speak about what? Length. I also speak about breadth. So let me create this.

EV0000034 · EV0000035 · EV0000036 · EV0000037

Transcript block 5 (source interval 001200-001499 seconds): [00:20:03.140] Okay, so what is the next thing that I have to do? Length. Length of which one? Correct? What length of which one is what I'm asking? Of that particular instance. How do I point that instance? Self. [00:20:19.396] Self dot length okay this is equal to L. Fine, hop this is clear. So I will say self dot breadth. Okay, is equal to b. [00:20:34.244] I uh hope uh there are no problems here. I will say print lent [00:20:44.932] Let's quickly just execute this to find out what is going to happen. Let me do this. There is nothing because what happens when I execute this? Only [00:20:59.524] Header is process. I never get into the body until we call this particular an object for this particular class gets created. Okay, so all I want to say is let it be commented. I'm going to write the program completely. [00:21:12.836] Okay, and then we will come back and uh make use of the same statement. Okay, so hope uh this is clear to you. I what do we do? We just print a length and uh a breadth. [00:21:30.628] Okay. Nothing much we want to do it here. Length into breadth is what you can say, but uh that we will try to do it later on. Okay. So let me say print. Okay. I'll say self.length. [00:21:49.092] Okay. Comma self dot breadth. [00:22:01.380] So this is what I will try to do. The reason why I commented also I will let you know at this point, please uh just bear with me. Okay? So we will create an object for this rectangle. So I say [00:22:12.784] R is the object for this rectangle. How do I need to create an object? Use the name of the class. Okay? C T A N G L E, a big name. Okay. Rectangle. Fine. So now internally, internally, what will happen? Internally, this is going to be like [00:22:33.188] underscore underscore in it underscore underscore of R. Okay. So now I have past self becomes R. But what do I need? I need to [00:22:46.688] Uh find what is the length and what is the breadth. I want to print both of them. So now I also need to pass the values. Okay so now how will I pass the values? I will pass the values like 10 comma twenty. [00:23:03.204] So internally this gets converted to what? 10 comma 20. So this is how it happens. So let's quickly execute this and try to find out. [00:23:17.156] Okay, so ten and twenty because we are not doing anything, we're just trying to print this. Fine. So let me come uncomment this and then try to execute. Push it here. And if you can see name length is not defined as what it was trying to say. [00:23:35.908] But when I commented this, I'm getting the output properly. The main reason is [00:23:41.316] Whenever you're making use of a variable, you should always use a fully qualified name. Length of which one? Length of this particular object. So it should be what? It

should be self.length. [00:23:57.604] So hope uh uh this is clear to you. Let me execute both of them. Fine. Length is 10, length is 10, comma 20. So [00:24:10.276] In this class, a pretty good uh way of uh we have understood so many things. We created an empty class, we came to know that class has a behavior, class has an attributes, okay. In this case, uh we came to know that uh there is an uh [00:24:26.080] Invalid function that gets invoked on creation of a particular object, which is called as a constructor. Okay. So now this is an example where a particular class has a function, but this is a built-in function. We have used constructor in order to initialize the variables. [00:24:45.892] Okay, so what we have done, we have used it in order to perform the initialization operation. Okay. So in the uh next class, what we will do is we will try to uh find out how I can add uh

EV0000038 · EV0000039 · EV0000040 · EV0000041

Transcript block 6 (source interval 001500-001799 seconds): [00:25:02.308] function to this particular class. Because we have done the function separately and we have done um the attributes separately, but we will try to find out how do we add [00:25:13.668] a function to this particular class. Okay? So hope you understood uh the concept of creation of a class. Happy learning, thank you.

EV0000042 · EV0000043 · EV0000044 · EV0000045

Full transcript: 3505 normalized words, 102 timestamped segments, 1529.386667 seconds duration. Low-confidence segments flagged: 0.

EV0000022 · EV0000023 · EV0000024 · EV0000025

5 Inheritance

This lesson is indexed from the source video titled Inheritance. Its complete timestamped transcript is preserved below in segment order. The original video path and content hash are recorded in datasets/lecture-corpus.json. Transcript text is source wording; it has not been silently corrected or rewritten. A candidate slide deck is grouped here according to the current matching score; the exact path and page locators are listed in the source evidence. EV0000046 · EV0000047

Graphs and diagrams

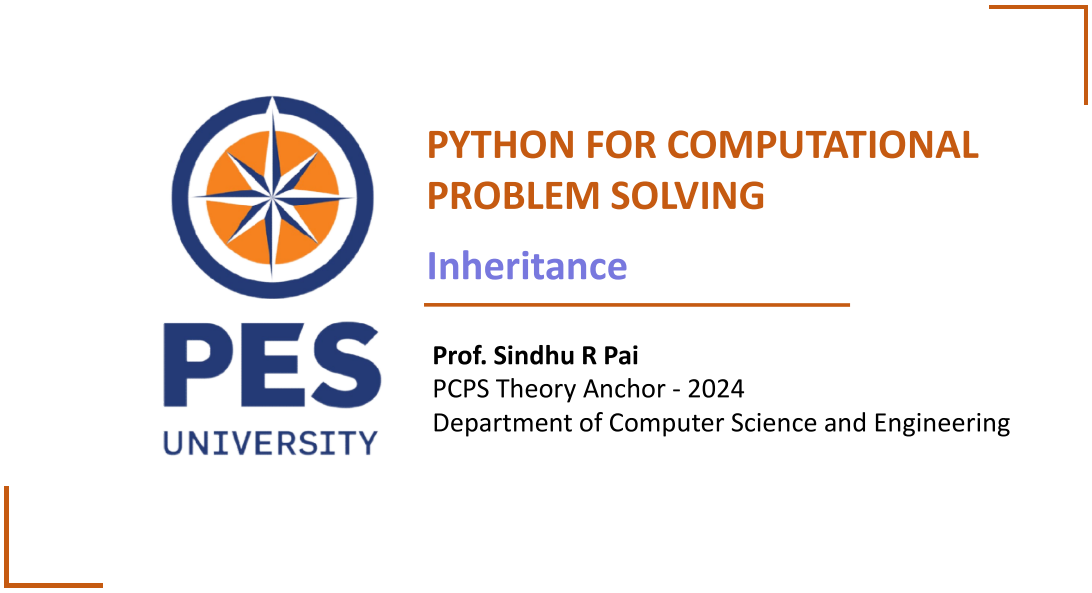


Figure G000078. UE24CS151A PCPS Unit 4 SindhuRPai Class 10 Inheritance · source slide page 1.

EV0000334

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Inheritance

PES UNIVERSITY

Introduction

- Acquiring or obtaining the features of one type in another type.
- Allows programmers to define a new class which inherits almost all the properties(data members and methods) of existing class.
- Two ways of relationships**: Is – a relationship** and **Has-a relationship**
- Is – a relationship is also known as **parent-child relationship**
- Has – a relationship is nothing but **containership or composition or collaboration**

2

Figure G000079. UE24CS151A PCPS Unit 4 SindhuRPai Class 10 Inheritance · source slide page 2.

EV0000337

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Inheritance

PES UNIVERSITY

Is – a relationship: Indicates that one class gets most or all of its features from a parent class.

When this kind of specialization occurs, there are three ways in which parent and child can interact.

1. Action on child imply an action on the parent
2. Action on the child override the action on the parent
3. Action on the child alter the action on the parent

3

Figure G000080. UE24CS151A PCPS Unit 4 SindhuRPai Class 10 Inheritance · source slide page 3.

EV0000340

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

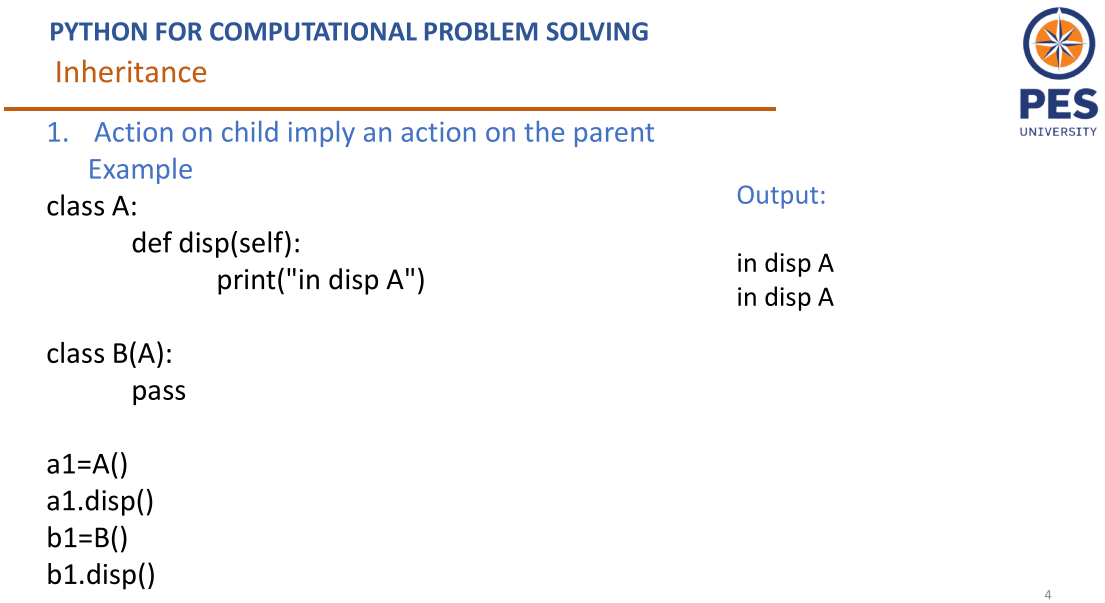


Figure G000081. UE24CS151A PCPS Unit 4 SindhuRPai Class 10 Inheritance · source slide page 4.

EV0000343

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

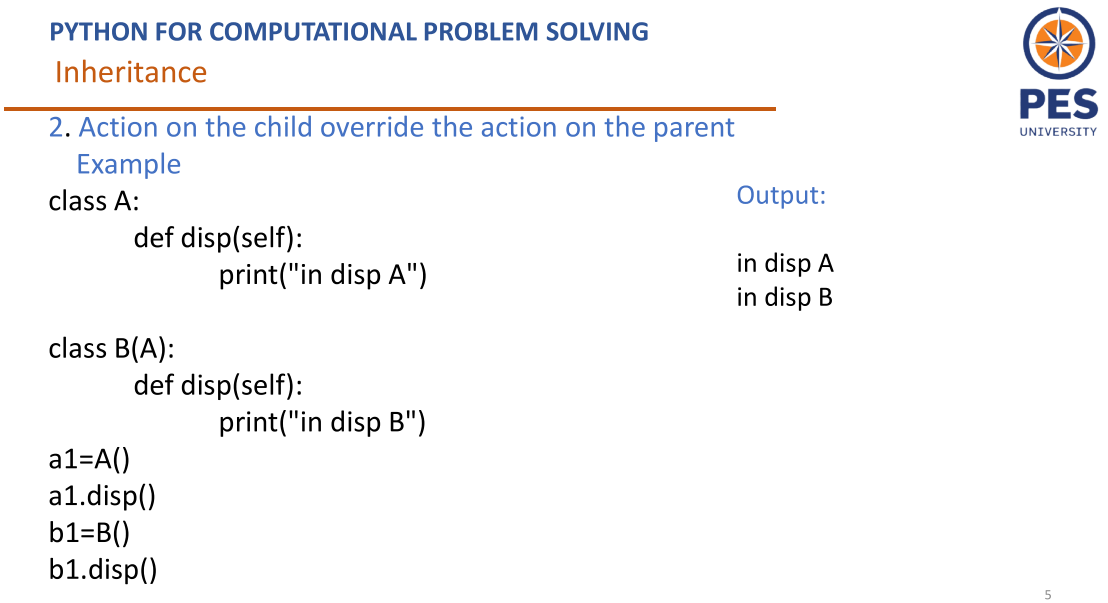


Figure G000082. UE24CS151A PCPS Unit 4 SindhuRPai Class 10 Inheritance · source slide page 5.

EV0000346

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

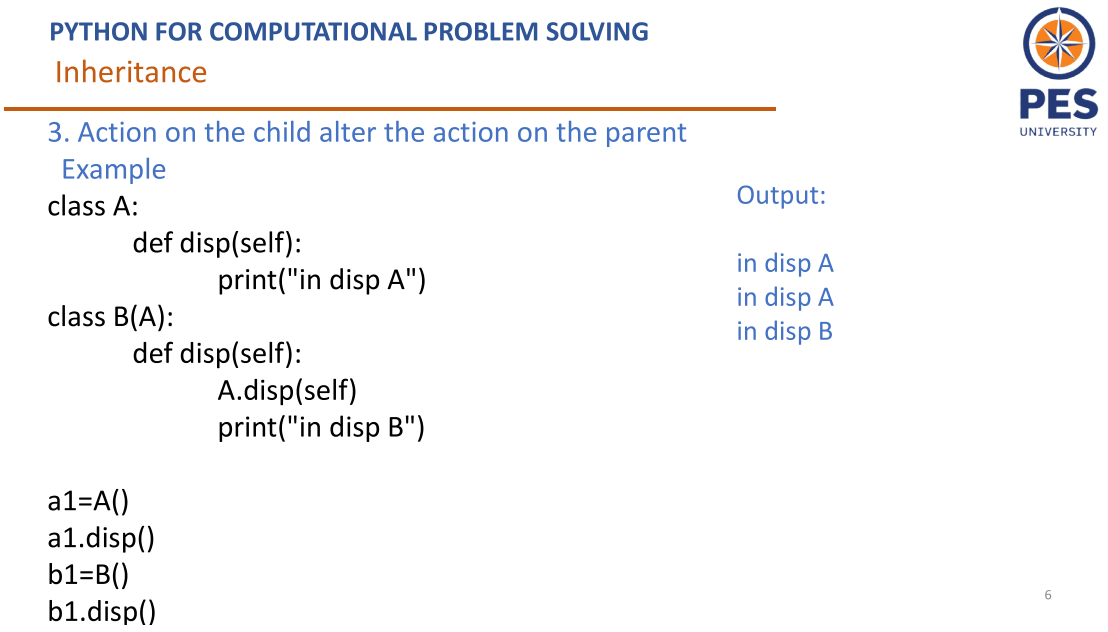


Figure G000083. UE24CS151A PCPS Unit 4 SindhuRPai Class 10 Inheritance · source slide page 6.

EV0000349

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

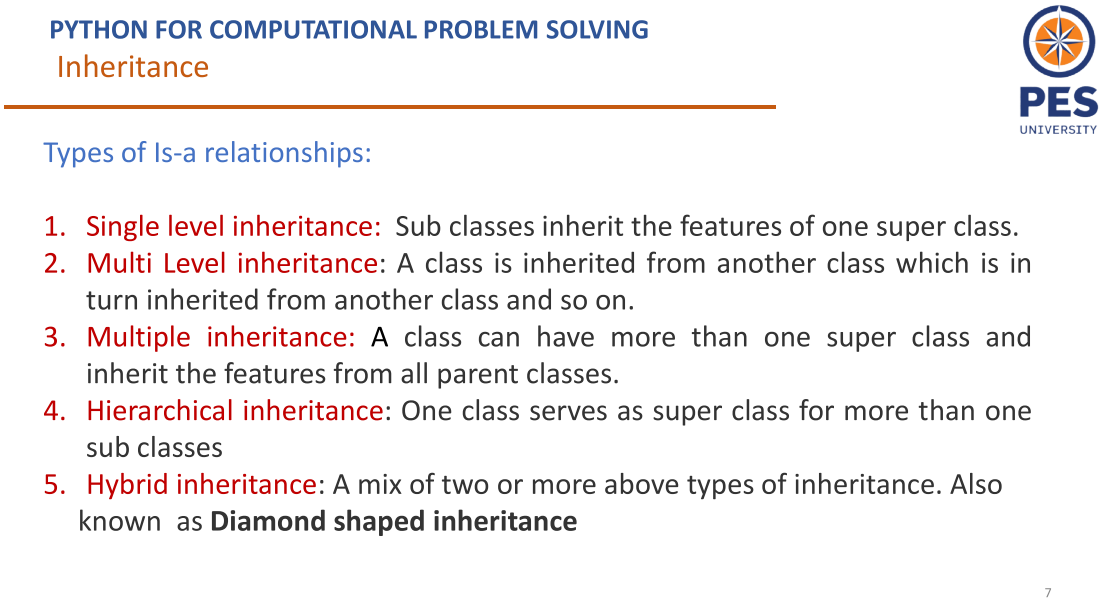


Figure G000084. UE24CS151A PCPS Unit 4 SindhuRPai Class 10 Inheritance · source slide page 7.

EV0000352

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Inheritance

PES UNIVERSITY

Benefits of inheritance:

- It allows to inherit the properties of a base class, to another class (derived) representing the real-world relationship.
- It provides the **reusability** of a code.
- Allows us to add more features to a class without modifying it.
- Transitive in nature, which means that if class B inherits from class A, then all the subclasses of B would automatically inherit from class A.
- Less development and maintenance expenses

8

Figure G000085. UE24CS151A PCPS Unit 4 SindhuRPai Class 10 Inheritance · source slide page 8.

EV0000355

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

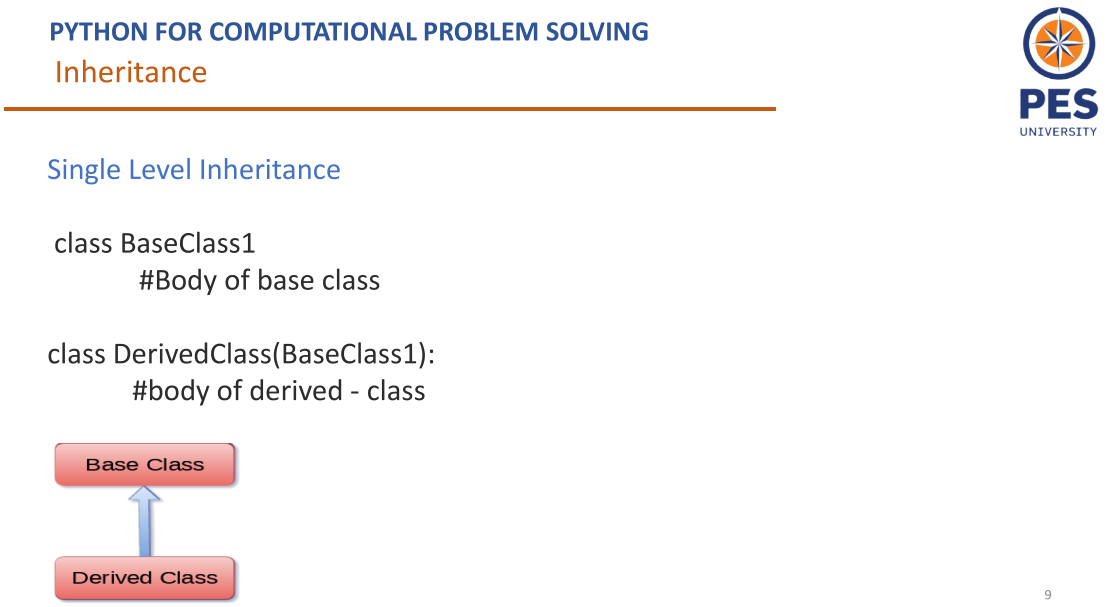


Figure G000086. UE24CS151A PCPS Unit 4 SindhuRPai Class 10 Inheritance · source slide page 9.

EV0000358

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

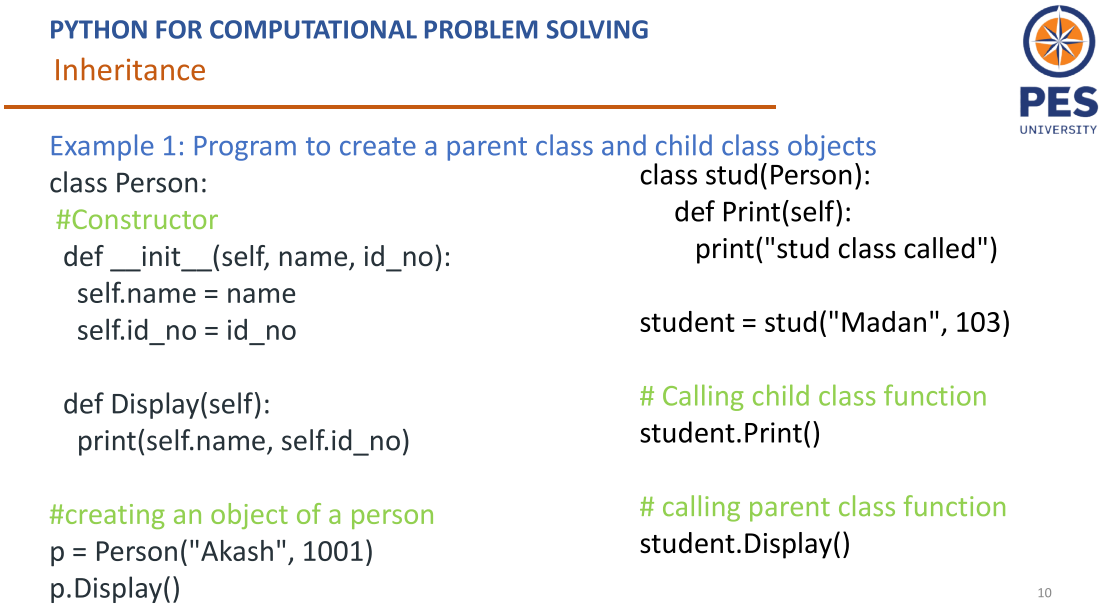


Figure G000087. UE24CS151A PCPS Unit 4 SindhuRPai Class 10 Inheritance · source slide page 10.

EV0000361

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

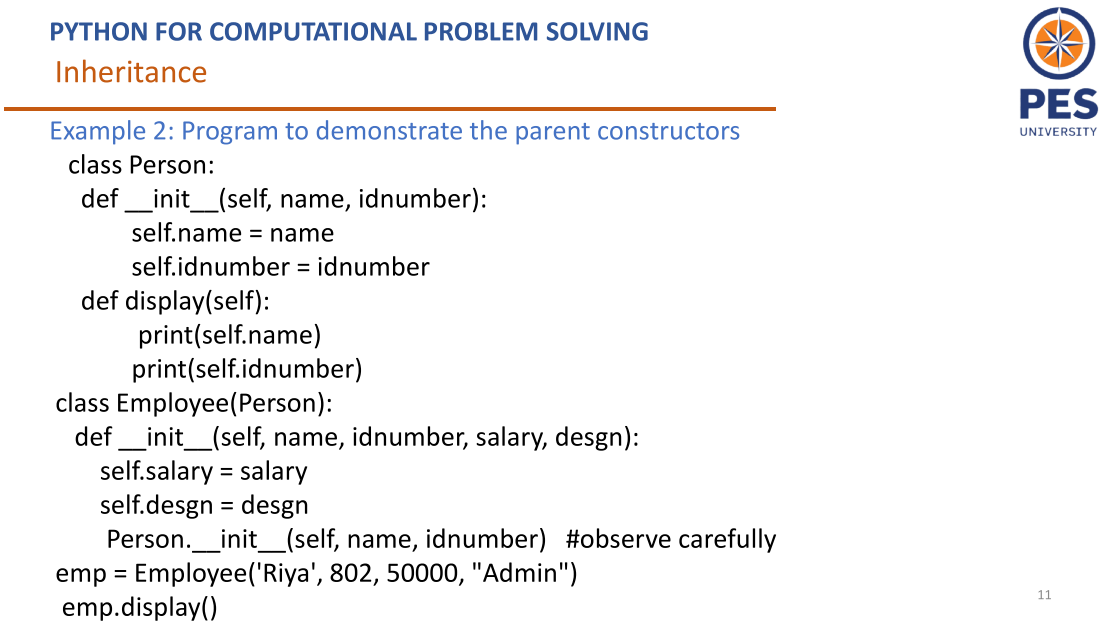


Figure G000088. UE24CS151A PCPS Unit 4 SindhuRPai Class 10 Inheritance · source slide page 11.

EV0000364

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

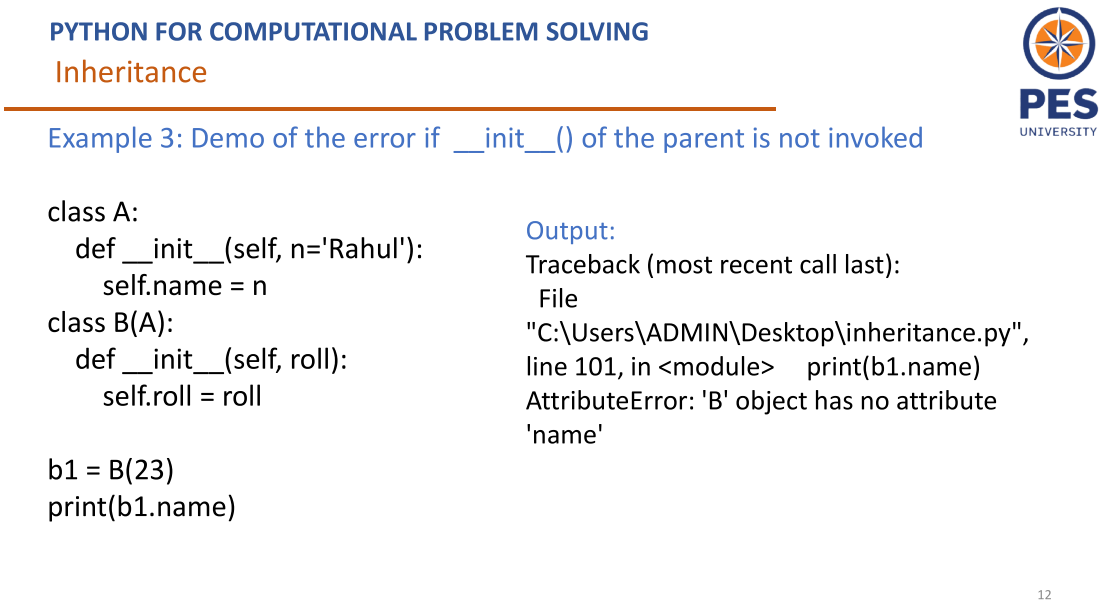


Figure G000089. UE24CS151A PCPS Unit 4 SindhuRPai Class 10 Inheritance · source slide page 12.

EV0000367

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Inheritance

PES UNIVERSITY

Super() Function

- It is a built-in function that provides a way to access methods and properties from a parent class within a subclass.

- There might be situations where the overridden method as well as the functionality of the parent method is required. That's where super() becomes helpful.

13

Figure G000090. UE24CS151A PCPS Unit 4 SindhuRPai Class 10 Inheritance · source slide page 13.

EV0000370

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

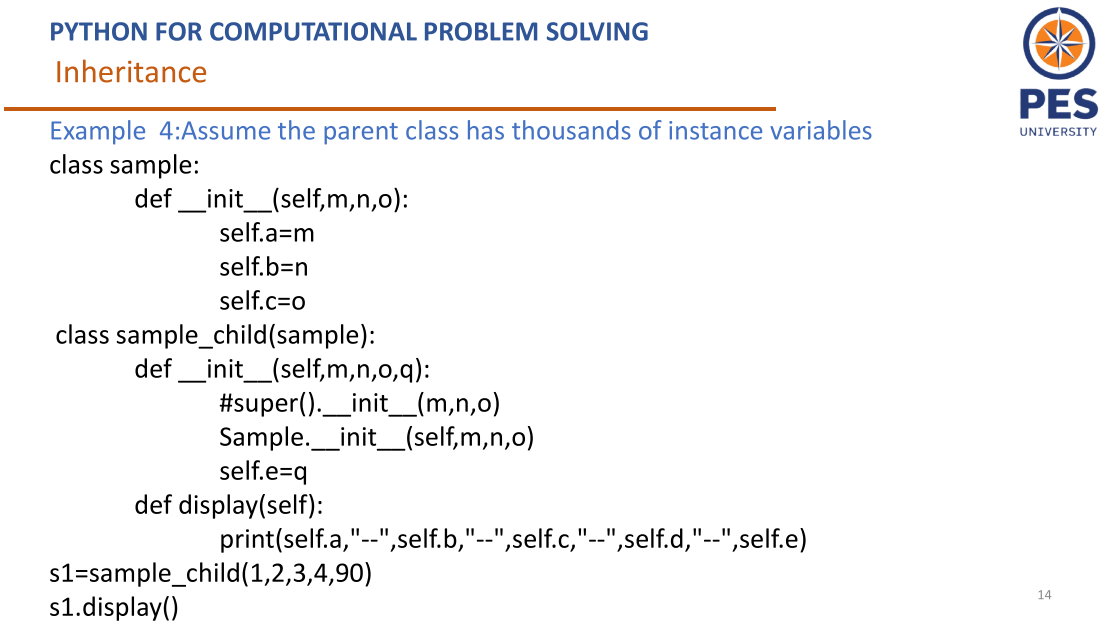


Figure G000091. UE24CS151A PCPS Unit 4 SindhuRPai Class 10 Inheritance · source slide page 14.

EV0000373

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

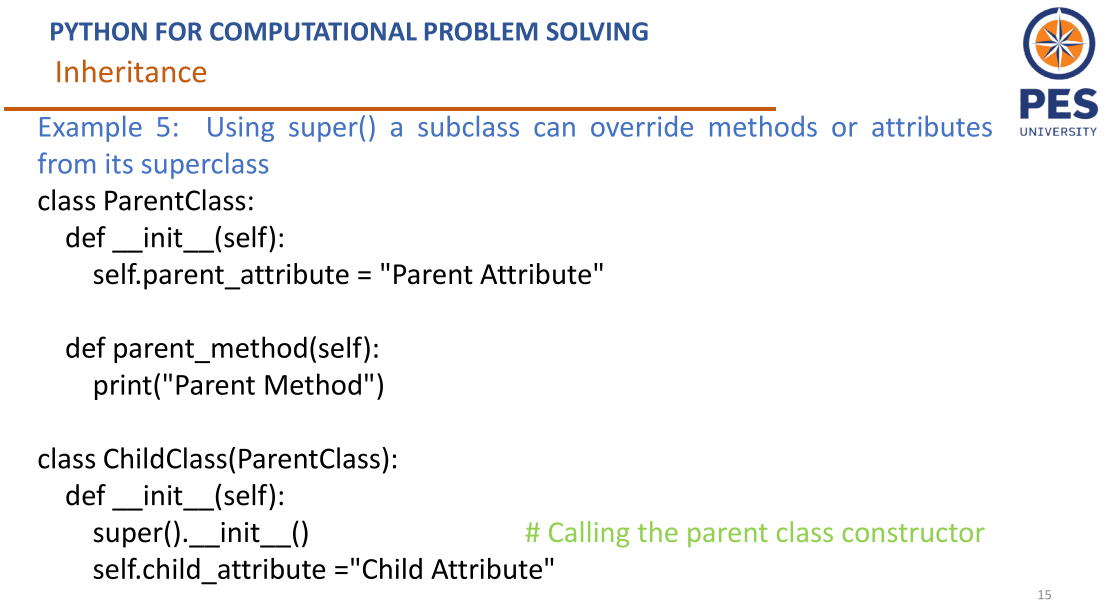


Figure G000092. UE24CS151A PCPS Unit 4 SindhuRPai Class 10 Inheritance · source slide page 15.

EV0000376

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

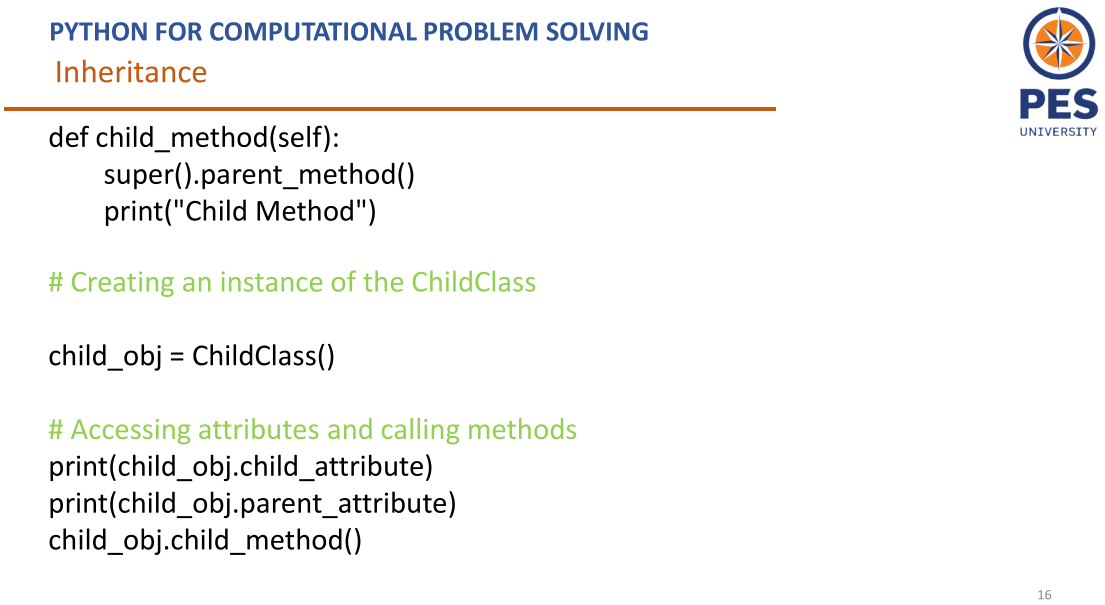


Figure G000093. UE24CS151A PCPS Unit 4 SindhuRPai Class 10 Inheritance · source slide page 16.

EV0000379

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

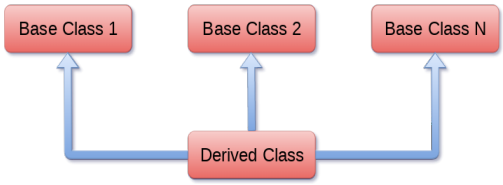
PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Inheritance

PES UNIVERSITY

Multiple inheritance

It provides the flexibility to inherit attributes and methods from more than one class



17

Figure G000094. UE24CS151A PCPS Unit 4 SindhuRPai Class 10 Inheritance · source slide page 17.

EV0000382

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

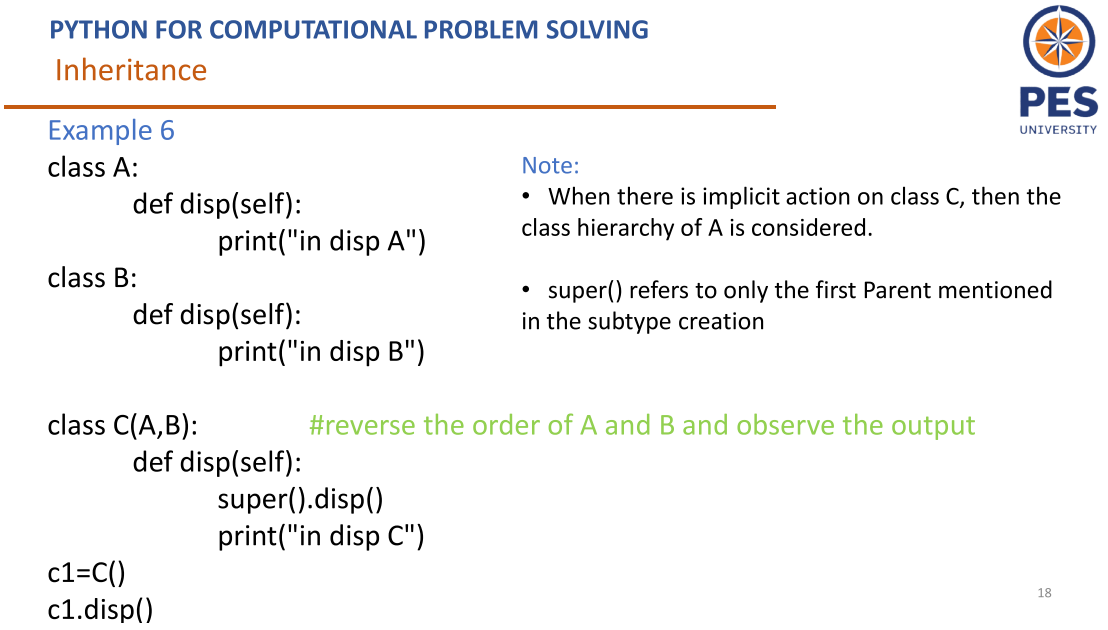


Figure G000095. UE24CS151A PCPS Unit 4 SindhuRPai Class 10 Inheritance · source slide page 18.

EV0000385

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Inheritance

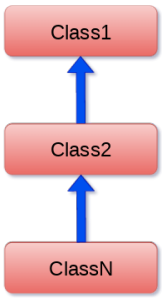
PES UNIVERSITY

Multi-Level inheritance

It refers to a type of inheritance where a subclass inherits from another subclass, forming a hierarchical chain of classes.

Syntax:

```
class class1:
        <class-suite>
class class2(class1):
        <class suite>
class class3(class2):
        <class suite>
```



19

Figure G000096. UE24CS151A PCPS Unit 4 SindhuRPai Class 10 Inheritance · source slide page 19.

EV0000388

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

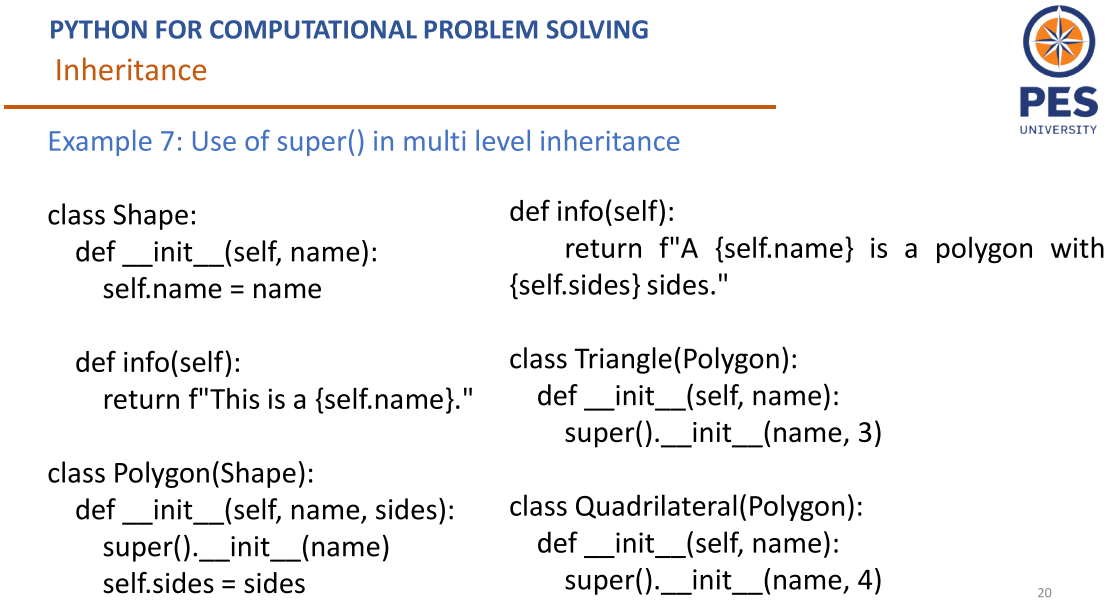


Figure G000097. UE24CS151A PCPS Unit 4 SindhuRPai Class 10 Inheritance · source slide page 20.

EV0000391

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

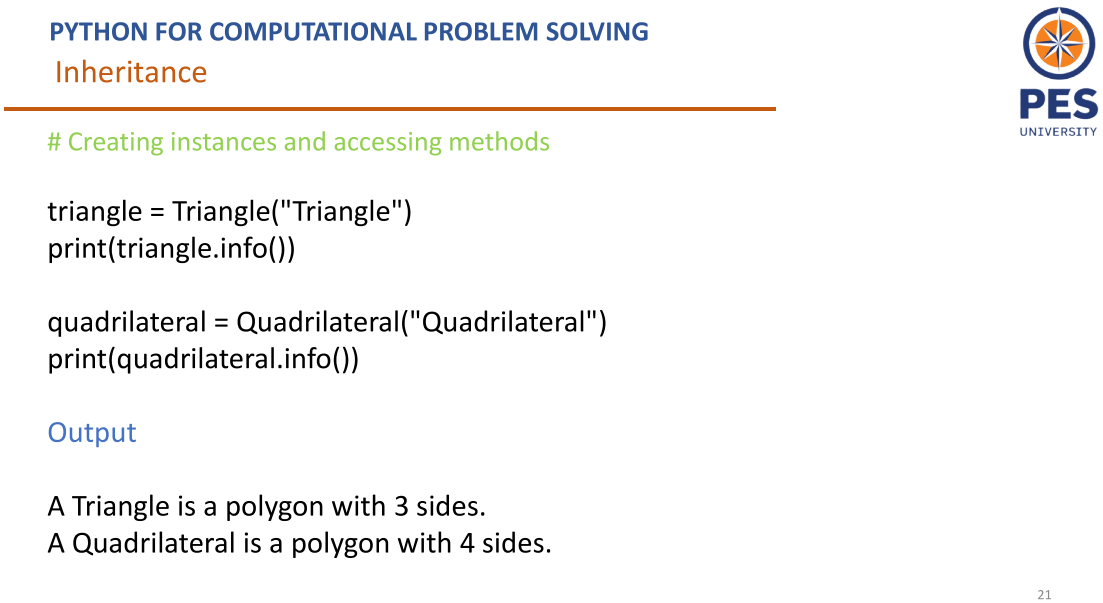


Figure G000098. UE24CS151A PCPS Unit 4 SindhuRPai Class 10 Inheritance · source slide page 21.

EV0000394

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Inheritance

PES UNIVERSITY

issubclass() and isinstance() methods

issubclass(sub, sup)

Used to check the relationships between the specified classes.
Returns True if the first class is the subclass of the second and False otherwise.

isinstance(obj,class)

Used to check the relationship between the objects and classes.
Returns True if the object is the instance of the specified class.

22

Figure G000099. UE24CS151A PCPS Unit 4 SindhuRPai Class 10 Inheritance · source slide page 22.

EV0000397

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

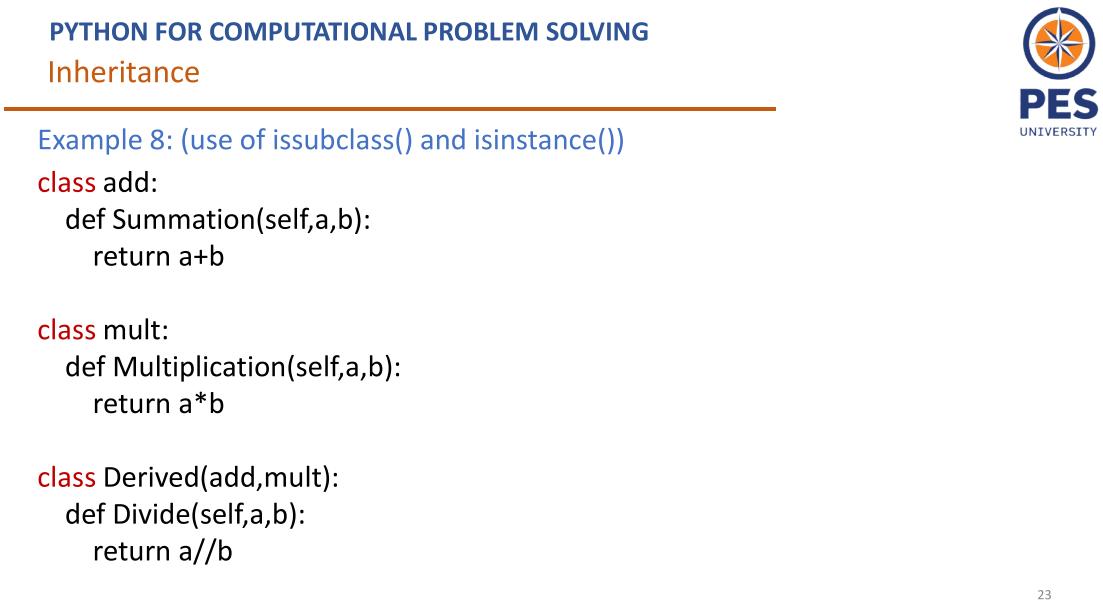


Figure G000100. UE24CS151A PCPS Unit 4 SindhuRPai Class 10 Inheritance · source slide page 23.

EV0000400

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

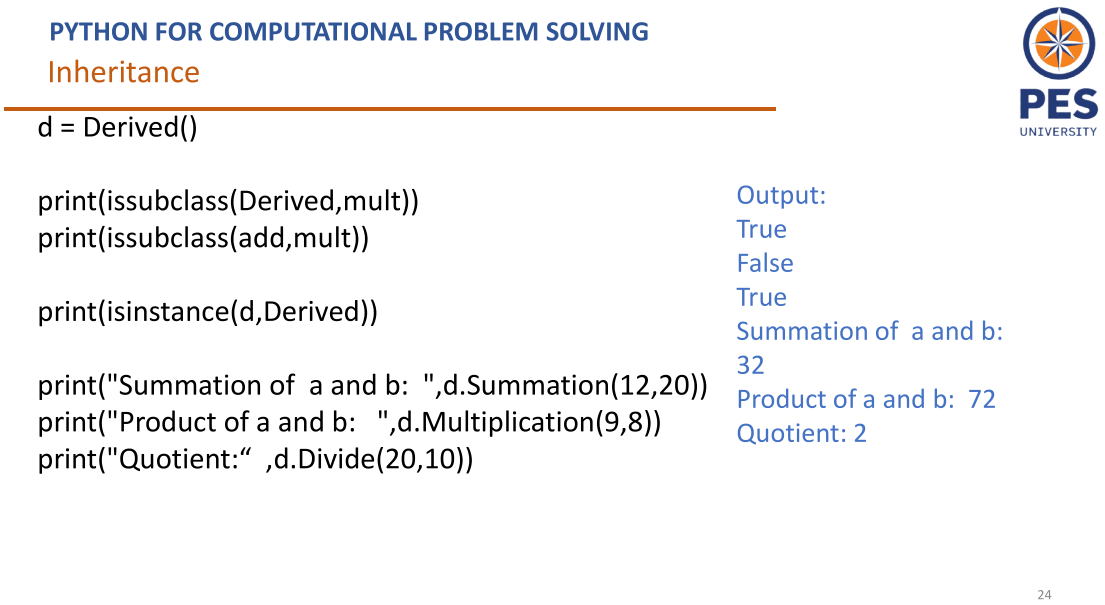


Figure G000101. UE24CS151A PCPS Unit 4 SindhuRPai Class 10 Inheritance · source slide page 24.

EV0000403

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

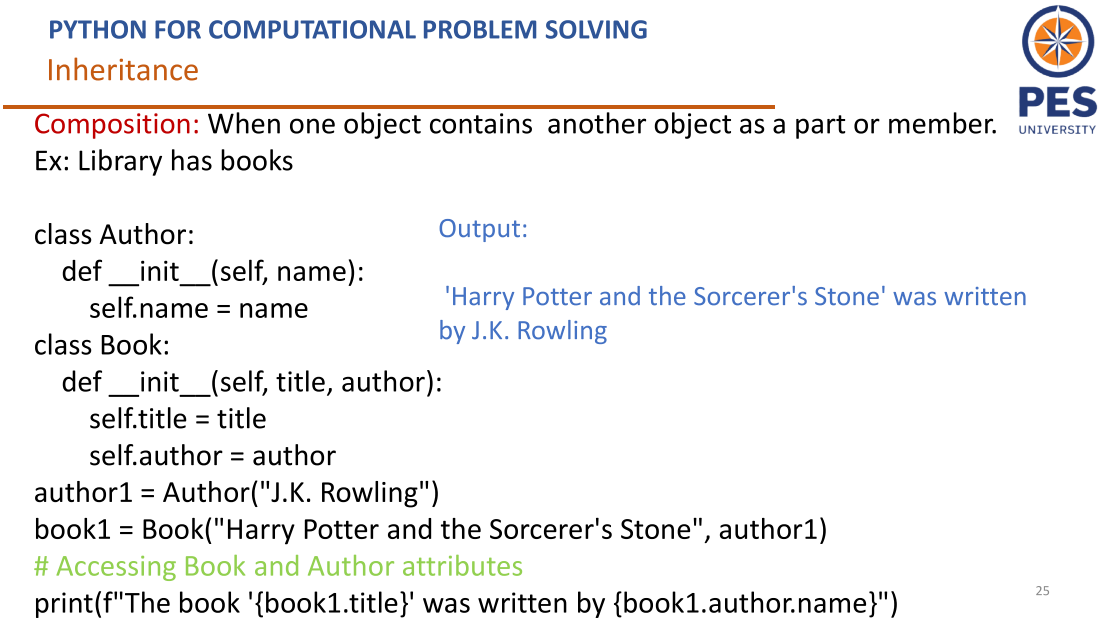


Figure G000102. UE24CS151A PCPS Unit 4 SindhuRPai Class 10 Inheritance · source slide page 25.

EV0000406

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

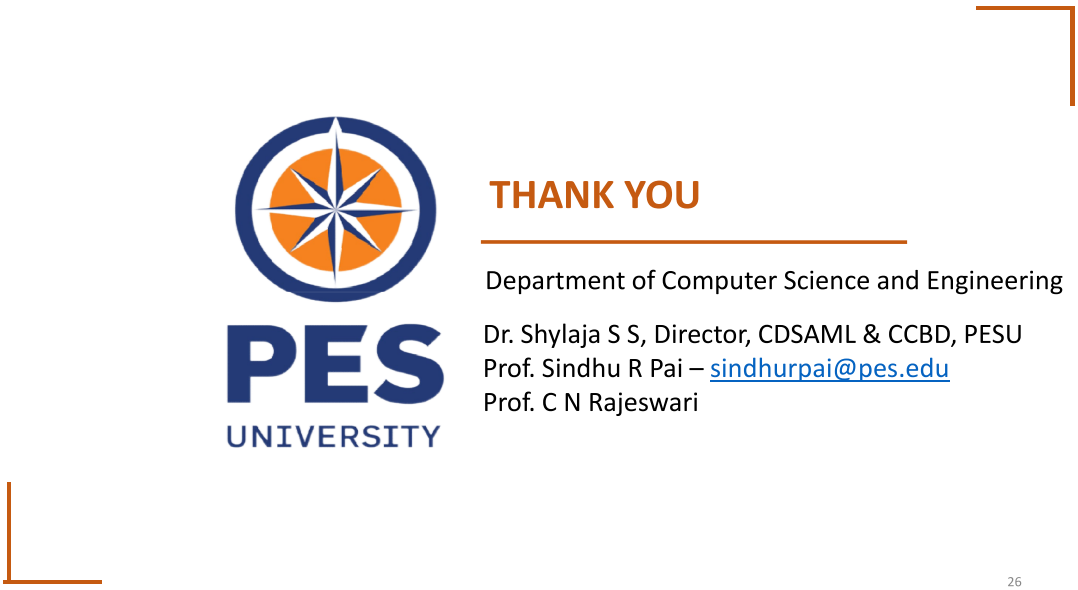


Figure G000103. UE24CS151A PCPS Unit 4 SindhuRPai Class 10 Inheritance · source slide page 26.

EV0000409

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Important points and why

1. Source slide page 1 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Inheritance Prof. Sindhu R Pai PCPS Theory Anchor - 2024 Department of Computer Science and Engineering EV0000333 · EV0000334 · EV0000335
2. Source slide page 2 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Inheritance Introduction • Acquiring or obtaining the features of one type in another type. • Allows programmersp g to define a new class which inherits almost all the properties(data members and methods) of existing class. • Two ways of relationships: Is – a relationship and Has-a relationship • Is – a relationship is also known as parent-child relationship • Has – a relationship is nothing but containership or composition or collaboration 2 EV0000336 · EV0000337 · EV0000338
3. Source slide page 3 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Inheritance Is – a relationship: Indicates that one class gets most or all of its features from a parent class. When this kind of specialization occurs, there are three ways in which parent and child can interact. 1. Action on child imply an action on the parent 2. Action on the child override the action on the parent 3. Action on the child alter the action on the parent 3 EV0000339 · EV0000340 · EV0000341

4. Source slide page 4 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Inheritance 1. Action on child imply an action on the parent Example Output:class A: def disp(self): in disp A print("in disp A") inin dispdisp AA class B(A): pass a1=A() a1.disp() b1=B() b1.disp() 4 EV0000342 · EV0000343 · EV0000344
5. Source slide page 5 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Inheritance 2. Action on the child override the action on the parent Example class A: Output: def disp(self): in disp A print("in disp A") inin dispdisp BB class B(A): def disp(self): print("in disp B") a1=A() a1.disp() b1=B() b1.disp() 5 EV0000345 · EV0000346 · EV0000347
6. Source slide page 6 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Inheritance 3. Action on the child alter the action on the parent Example Output:class A: def disp(self): in disp A print("in disp A") inin dispdisp AA classl B(A):( ) in disp B def disp(self): A.disp(self) print("in disp B") a1=A() a1.disp() b1=B() 6 b1.disp() EV0000348 · EV0000349 · EV0000350
7. Source slide page 7 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Inheritance Types of Is-a relationships: 1. Single level inheritance: Sub classes inherit the features of one super class. 2. Multi Level inheritance: A class is inherited from another class which is in turnturn inheritedinherited fromfrom anotheranother classclass andand soso onon. 3. Multiple inheritance: A class can have more than one super class and inherit the features from all parent classes. 4. Hierarchical inheritance: One class serves as super class for more than one sub classes 5. Hybrid inheritance: A mix of two or more above types of inheritance. Also known as Diamond shaped inheritance 7 EV0000351 · EV0000352 · EV0000353
8. Source slide page 8 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Inheritance Benefits of inheritance: • It allows to inherit the properties of a base class, to another class (derived) representing the real-world relationship. • It providesd theh reusabilityb l off a code.d • Allows us to add more features to a class without modifying it. • Transitive in nature, which means that if class B inherits from class A, then all the subclasses of B would automatically inherit from class A. • Less development and maintenance expenses 8 EV0000354 · EV0000355 · EV0000356
9. Source slide page 9 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Inheritance Single Level Inheritance class BaseClass1 #Body of base class class DerivedClass(BaseClass1): #body of derived - class 9 EV0000357 · EV0000358 · EV0000359
10. Source slide page 10 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Inheritance Example 1: Program to create a parent class and child class objects class stud(Person):class Person: def Print(self):#Constructor print("stud class called") def __init__(self, name, id_no): self.name = name student = stud("Madan", 103) self.id_no = id_no # Calling child class function def Display(self): student.Print() print(self.name, self.id_no) # calling parent class function#creating an object of a person student.Display()p = Person("Akash", 1001) 10p.Display() EV0000360 · EV0000361 · EV0000362

11. Source slide page 11 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Inheritance Example 2: Program to demonstrate the parent constructors class Person: def __init__(self, name, idnumber): self.name = name self.idnumber = idnumber defdef display(self):display(self): print(self.name) print(self.idnumber) class Employee(Person): def __init__(self, name, idnumber, salary, desgn): self.salary = salary self.desgn = desgn Person.__init__(self, name, idnumber) #observe carefully emp = Employee('Riya', 802, 50000, "Admin") emp.display() 11 EV0000363 · EV0000364 · EV0000365
12. Source slide page 12 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Inheritance Example 3: Demo of the error if __init__() of the parent is not invoked class A: Output: def __init__(self, n='Rahul'): Traceback (most recent call last): self.name = n FileFile class B(A): "C:\Users\ADMIN\Desktop\inheritance.py", def __init__(self, roll): line 101, in <module> print(b1.name) self.roll = roll AttributeError: 'B' object has no attribute 'name' b1 = B(23) print(b1.name) 12 EV0000366 · EV0000367 · EV0000368
13. Source slide page 13 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Inheritance Super() Function • It is a built-in function that provides a way to access methods and properties from a parent class within a subclass. • There might be situations where the overridden method as well as the functionality of the parent method is required. That's where super() becomes helpful. 13 EV0000369 · EV0000370 · EV0000371
14. Source slide page 14 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Inheritance Example 4:Assume the parent class has thousands of instance variables class sample: def __init__(self,m,n,o): self.a=m self.b=n selfself.cc=oo class sample_child(sample): def __init__(self,m,n,o,q): #super().__init__(m,n,o) Sample.__init__(self,m,n,o) self.e=q def display(self): print(self.a,"--",self.b,"--",self.c,"--",self.d,"--",self.e) s1=sample_child(1,2,3,4,90) s1.display() 14 EV0000372 · EV0000373 · EV0000374
15. Source slide page 15 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Inheritance Example 5: Using super() a subclass can override methods or attributes from its superclass class ParentClass: def __init__(self): self.parent_attribute = "Parent Attribute" def parent_method(self): print("Parent Method") class ChildClass(ParentClass): def __init__(self): super().__init__() # Calling the parent class constructor self.child_attribute ="Child Attribute" 15 EV0000375 · EV0000376 · EV0000377
16. Source slide page 16 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Inheritance def child_method(self): super().parent_method() print("Child Method") # Creating an instance of the ChildClass child_obj = ChildClass() # Accessing attributes and calling methods print(child_obj.child_attribute) print(child_obj.parent_attribute) child_obj.child_method() 16 EV0000378 · EV0000379 · EV0000380
17. Source slide page 17 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Inheritance Multiple inheritance It provides the flexibility to inherit attributes and methods from more than one class 17 EV0000381 · EV0000382 · EV0000383

18. Source slide page 18 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Inheritance Example 6 class A: Note: • When there is implicit action on class C, then the def disp(self): class hierarchy of A is considered. print("in disp A") class B: • super() refers to only the first Parent mentioned ddeff ddisp(self):(lf) in the subtype creation print("in disp B") class C(A,B): #reverse the order of A and B and observe the output def disp(self): super().disp() print("in disp C") c1=C() 18 c1.disp() EV0000384 · EV0000385 · EV0000386

19. Source slide page 19 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Inheritance Multi-Level inheritance It refers to a type of inheritance where a subclass inherits from another subclass, forming a hierarchical chain of classes. Syntax:y class class1: <class-suite> class class2(class1): <class suite> class class3(class2): <class suite> 19 EV0000387 · EV0000388 · EV0000389

20. Source slide page 20 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Inheritance Example 7: Use of super() in multi level inheritance class Shape: def info(self): def __init__(self, name): return f"A {self.name} is a polygon with self.name = name {self.sides} sides." def info(self): class Triangle(Polygon): return f"This is a {self.name}." def __init__(self, name): super().__init__(name, 3) class Polygon(Shape): def __init__(self, name, sides): class Quadrilateral(Polygon): super().__init__(name) def __init__(self, name): self.sides = sides super().__init__(name, 4) 20 EV0000390 · EV0000391 · EV0000392

21. Source slide page 21 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Inheritance # Creating instances and accessing methods triangle = Triangle("Triangle") print(triangle.info()) quadrilateral = Quadrilateral("Quadrilateral") print(quadrilateral.info()) Output A Triangle is a polygon with 3 sides. A Quadrilateral is a polygon with 4 sides. 21 EV0000393 · EV0000394 · EV0000395

22. Source slide page 22 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Inheritance issubclass() and isinstance() methods issubclass(sub, sup) UsedUsed toto checkcheck thethe relationshipsrelationships betweenbetween thethe specifiedspecified classesclasses. Returns True if the first class is the subclass of the second and False otherwise. isinstance(obj,class) Used to check the relationship between the objects and classes. Returns True if the object is the instance of the specified class. 22 EV0000396 · EV0000397 · EV0000398

23. Source slide page 23 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Inheritance Example 8: (use of issubclass() and isinstance()) class add: def Summation(self,a,b): return a+b class mult: def Multiplication(self,a,b): return a*b class Derived(add,mult): def Divide(self,a,b): return a//b 23 EV0000399 · EV0000400 · EV0000401

24. Source slide page 24 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Inheritance d = Derived() print(issubclass(Derived,mult)) Output: Trueprint(issubclass(add,mult)) False TrueTrueprint(isinstance(d,Derived))i t(i i t (d D i d)) Summation of a and b: 32 print("Summation of a and b: ",d.Summation(12,20)) Product of a and b: 72 print("Product of a and b: ",d.Multiplication(9,8)) Quotient: 2 print("Quotient:“ ,d.Divide(20,10)) 24 EV0000402 · EV0000403 · EV0000404

25. Source slide page 25 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Inheritance Composition: When one object contains another object as a part or member. Ex: Library has books Output:class Author: def __init__(self, name): 'HarryHarry PotterPotter andand thethe Sorcerer'sSorcerer s Stone'Stone waswas writtenwritten selfself.namename = namename by J.K. Rowlingclass Book: def __init__(self, title, author): self.title = title self.author = author author1 = Author("J.K. Rowling") book1 = Book("Harry Potter and the Sorcerer's Stone", author1) # Accessing Book and Author attributes print(f"The book '{book1.title}' was written by {book1.author.name}") 25 EV0000405 · EV0000406 · EV0000407
26. Source slide page 26 (extracted text): THANK YOU Departmentp of Computerp Science and Engineeringg g Dr. Shylaja S S, Director, CDSAML & CCBD, PESU Prof. Sindhu R Pai – sindhurpai@pes.edu Prof. C N Rajeswari 26 EV0000408 · EV0000409 · EV0000410

SHORT-NOTE RECAP

Transcript block 1 (source interval 000000-000299 seconds): [00:00:01.348] Hello all. Welcome back, Python for computational problem solving. In the last class, we looked into the concept of encapsulation. [00:00:14.372] we are going to be learning the concept of inheritance. [00:00:22.724] So we'll see an example in order to understand what exactly is inheritance, and how do we uh put the concept of inheritance using Python. So let's get started. The example that I would like to take is cats, tiger. [00:00:43.172] Lion. [00:00:46.980] Etc. Sorry. So now why this particular example? When I say I say a cat is a mammal, tiger is a cat, lion is also a cat. All these [00:01:06.628] Animals, they have a common characteristic. They walk on their toes. [00:01:15.332] There may be they climb the tree. Okay. There may be different ways in which they walk or they try to climb, but still all of them we address them as what? The cat itself. [00:01:28.612] Okay so now whenever there is a concept of ease a relationship there is an ease a relationship [00:01:40.676] If an easy relationship that exists, fine, then we call it as inheritance. A cat is a mammal, tiger is a cat, lion is a cat, and so on. [00:01:54.500] Yeah, fine. So in your uh um childhood or uh r right now also, people might say that you look like your uh father, you look like your grandfather, and so on. [00:02:07.812] That is also called as an inheritance. You inherit properties from your ancestors. Right? So now it may be ancestors, it may be your grandparent or your parent or your um uh the maternal or paternal uh people. So you basically inherit some properties. [00:02:27.652] So now how do we put all together in terms of Python is what we are going to be learning now. Fine. So now let's get started with the particular program. So now to start with, what I will do is I will create a class, and the name of the class is A. [00:02:45.892] And now do I really want to do anything in this case? No, nothing. So I'll say print this is in class A. This is what I will do. [00:03:02.724] Fine. Now I'll create a class called as B. What I really want to do here, nothing great other than just a print here. So I will say this is in class B. [00:03:19.460] But I said I want class B to inherit the properties of class A. Fine? I want to inherit the properties of class A. So in order to do that, this is the modification that we try to do. [00:03:35.460] Okay, so this is the modification we do. So which means B is a class which inherits from A. Okay. So now as usual, let's create an object for a class, but in this case I will create it for class B. [00:03:50.980] So which means class B inherits from A. So even though I try to uh print only one statement because it is inheriting, so I will also get the uh information that is present in your class A. Okay, so let's quickly try on with this. [00:04:08.964] And then let me execute. So this is in class A, this is in class B, and so on. I'll just do a simple modification to this program and I'll say A is equal to twenty. Okay. So let me

eliminate this print statement. I don't need a print statement. [00:04:27.972] Okay, so now A variable is created in class A, but I will try to access it using the object that is created by B. Okay so now what I will do is I'll see print O B J dot. [00:04:48.804] A. Okay. So OBJ. Okay. Now what have I done? We have created [00:04:57.156] An object for a class B, but I know that class B inherits from class A. So I'm using an object which belongs to class B in order to print A. Because of the inheritance, what should happen? I should also get the value called as

EV0000046 · EV0000047 · EV0000048 · EV0000049

Transcript block 2 (source interval 000300-000599 seconds): [00:05:12.464] Should I be here. Okay. See, I'll just write the value. [00:05:22.916] Value of A is. Okay. So let's see what output we get. Okay. So now when I execute [00:05:35.236] And C the value of A is twenty. So hope you have understood. I have not created a variable A, but because of the inheritance, I am able to access, I am able to access uh the value because I inherit I in the sense you can [00:05:51.844] Class B inherits the properties from class A. And that's how we have got this is in class B. Okay so. [00:06:00.996] This is called as the way in which we have written B is inheriting from A. Okay? So this we call it as what? Single inheritance or single level inheritance. [00:06:14.180] Okay, so there is one class getting inherited from the other class only. If you are inheriting from a single class itself, then it is called as what? Single inheritance. Hope this is clear to you. [00:06:27.600] Okay so let me uh go ahead uh with the actual problem which is uh easy for us to understand. Quickly I will code it so that you try to understand. So I'll say class operation. [00:06:43.396] So try to understand what is happening here. Fine. I'll say A is equal to 10, B is equal to 20. Now I will define a class called as add. Okay. I hope you know why the usage of self. [00:07:02.692] Okay, I am my intention is to perform addition which is sum. So I'll say self dot a plus self dot b. Okay, self dot a plus self dot b. Nothing. Right? So let me print this. [00:07:21.092] So I'll say print sum is okay. The value sum. So do this. Fine. Now I will create a class called as class something. Say my class. [00:07:38.532] Okay, and this inherits from operation. [00:07:46.660] inherits from operations. Okay. So now what do I do is nothing great other than performing a subtraction operation here. So let's do this. But [00:08:03.332] Just look what is going to happen. I'll say self dot B. What is the value of B? Yeah. So that self dot A. Pretty uh straightforward. Okay. I'll see print. [00:08:24.644] Subvalue is okay, sub itself. Okay. So this is what I will try to do. As usual. Let's create it to you so that it is easy for you to understand. So I'll say. [00:08:43.428] Create an object. Okay. So now let me create an object. Object OB is equal to for which class we are going to be doing? We are going to be doing it for my class. Okay. So my class. Please remember, my class inherits from [00:09:00.548] Class operations. Fine. A and B is defined in class A, but as I'm inheriting, I should also get the values from uh the class operations. Okay so now. [00:09:15.236] I'll say it as obsub. Okay. So let's execute this. The moment I do this, you have created an object for your my class. Okay. So now I'll go to my class where my sub function is defined. I will try to perform what? Subtraction operation. [00:09:34.884] Okay, so now value of A and B will be from which one? From the class operation, because I am performing inheritance. Okay, so let's quickly execute. [00:09:46.948] Fine. The sub subtracted value is ten because twenty minus ten. Okay now I'll also do something like this. Ob Please remember you are trying to access [00:09:59.300] The function add, which is in your operation class, but you have created an object for your my class. Because it is inheritance, I inherit all the properties from

EV0000050 · EV0000051 · EV0000052 · EV0000053

Transcript block 3 (source interval 000600-000899 seconds): [00:10:11.844] The operations class, because of which variables are available, I can also access the function. Okay, so let me execute this and find out what is the answer. Okay. So now subtract uh the sub value is 10, the sum value is 30. [00:10:31.332] Okay, so hope uh this concept is clear to you. So this is called as single level inheritance. Okay, let me write here. It is single inheritance. [00:10:49.732] So this is single inheritance. Let's quickly go ahead and do multi-level inheritance. [00:11:01.988] Multi-level inheritance. [00:11:10.308] Quickly uh do this. Why don't I uh copy paste the same thing? Okay, so that it is easy for you also to understand. Okay, let me copy paste. Okay, so now [00:11:28.548] I'll create one more class. Okay. Class. Oh what do we do? Multiplication, shall we do? Yes. So say multiply. [00:11:41.572] Multi. Okay. So this is what I will true I will try to do, but I'll see my class. [00:11:51.428] So now I'll say define multiply [00:11:58.756] Okay. Define a multiply. [00:12:09.988] or yes result is equal to self dot a into self dot b self dot b please remember [00:12:23.204] Quickly do this and then tell you few tips for you to remember. Okay? Uh final result or something. Final answer is what result. Okay so now what I will do is I will create an object [00:12:41.028] Okay, an object for which class? Multiple class, multi-class. Okay. So I'll say multi. This is fine. So now I've called an object for multi-class. [00:12:55.684] Fine. Multi inherits from my class, my class inherits from operation. So this is going something like this. So you have class A, this class A, you have class B, and you have class C. Okay, class C is inheriting from class B. [00:13:14.596] Okay. And class B is getting inherited from class A. So this is something is what it is happening fine. So now this we call it as multi-level. Okay. There are more than [00:13:29.056] One level in this case, so it is called as what? Multi-level inheritance. Hope this is clear to you. Okay. So now what I will do is I will say O B J dot um add. [00:13:43.808] Okay so there is an add, let me call this, I'll say obj dot subtract because there is also subtract and I'll say obj dot multiply. [00:14:00.068] Okay, OBJ dots multiple. Let's uh uh quickly execute this and check. Fine. Sub value is thirty. Uh wait a second, wait a second. Let me comment this, otherwise we will get more number of outputs in the final app. [00:14:21.284] Let me do this. I've commented, so these are the things that we had. Okay, hope this is clear to you. I'm not confusing. Execute. You can see sum is 30, subtracted value is 10, and the final answer, which is multiplication, is 200. [00:14:39.728] Okay, so this way of doing is called as what? Multi-level inheritance. Okay, so there is one more, okay, which is I have a class A, I have class B, there is something called as a class C, which inherits from class A as well as class B.

EV0000054 · EV0000055 · EV0000056 · EV0000057

Transcript block 4 (source interval 000900-001199 seconds): [00:15:00.484] Okay, so you have multiple classes. Okay, so now in this case class C is inheriting from class A as well as class B. So this is called as multiple inheritance. [00:15:20.868] So this is called as multiple inheritance. I kindly request you to uh try out a program of this particular sort. Only change that you are going to be doing is fine. [00:15:32.256] Instead of just my class, you will bring out this particular operations here itself. Please remember inheritance is happening only between class C, A, and B, and B and A are nowhere related to each other through the inheritance. Okay, in this particular example. Please always remember. [00:15:49.936] I urge you people to kindly try out these things. In case of any uh clarifications we can always discuss. Hope uh you have understood the concept. Thank you.

EV0000058 · EV0000059 · EV0000060 · EV0000061

Full transcript: 1967 normalized words, 64 timestamped segments, 963.114667 seconds duration. Low-confidence segments flagged: 0.

EV0000046 · EV0000047 · EV0000048 · EV0000049

6 Exceptions

This lesson is indexed from the source video titled Exceptions. Its complete timestamped transcript is preserved below in segment order. The original video path and content hash are recorded in datasets/lecture-corpus.json. Transcript text is source wording; it has not been silently corrected or rewritten. A candidate slide deck is grouped here according to the current matching score; the exact path and page locators are listed in the source evidence. EV0000062 · EV0000063

Graphs and diagrams

Figure G000123. UE24CS151A PCPS Unit 4 SindhuRPai Class 14 15 Exceptional Handling · source slide page 1.

EV0000469

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Exception Handling

PES UNIVERSITY

Introduction:

- Programming error is an error which may occur because of various reasons.

Program can produce some errors even if the program is perfect.

- This may be because of

1. Exceptional situations that may occur during the execution of a program.
2. The data coming from outside the program which is malformed .

2

Figure G000124. UE24CS151A PCPS Unit 4 SindhuRPai Class 14 15 Exceptional Handling · source slide page 2.

EV0000472

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

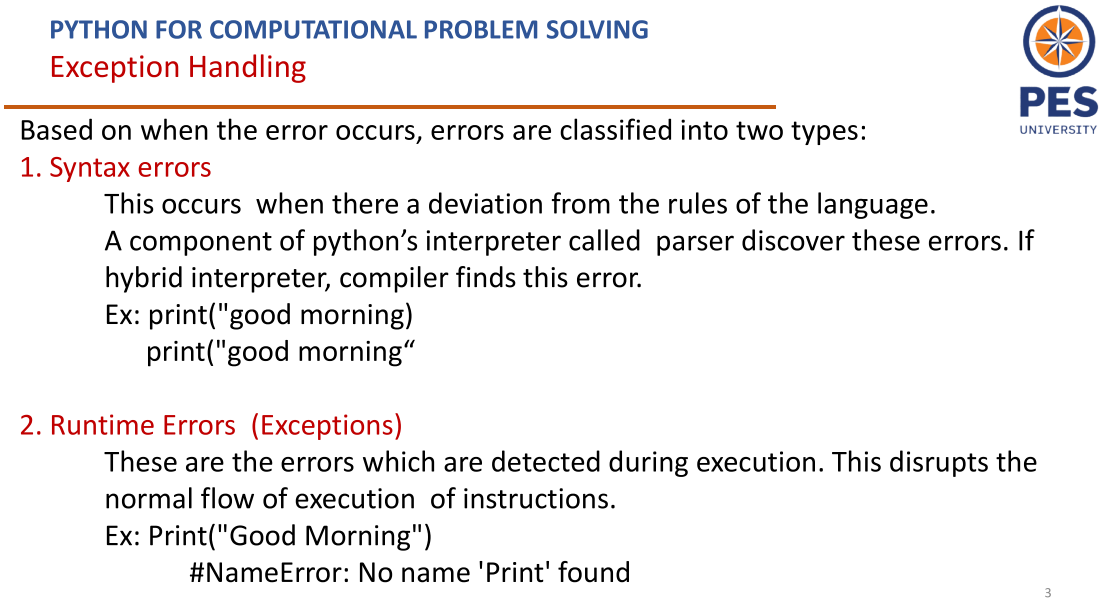


Figure G000125. UE24CS151A PCPS Unit 4 SindhuRPai Class 14 15 Exceptional Handling · source slide page 3.

EV0000475

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

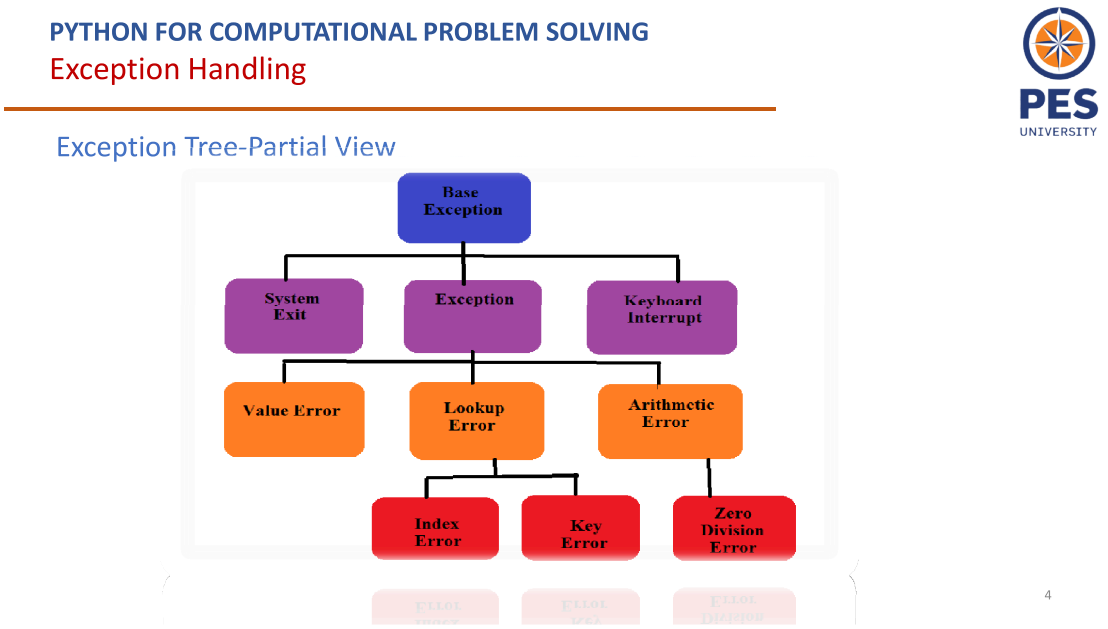


Figure G000126. UE24CS151A PCPS Unit 4 SindhuRPai Class 14 15 Exceptional Handling · source slide page 4.

EV0000478

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Exception Handling

PES UNIVERSITY

Two categories of Exceptions:

1. Built-In exception (System defined exceptions)
   Those exceptions which Python knows by default.

2. User defined exceptions
   User-defined type/class written for handling the exceptions

5

Figure G000127. UE24CS151A PCPS Unit 4 SindhuRPai Class 14 15 Exceptional Handling · source slide page 5.

EV0000481

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

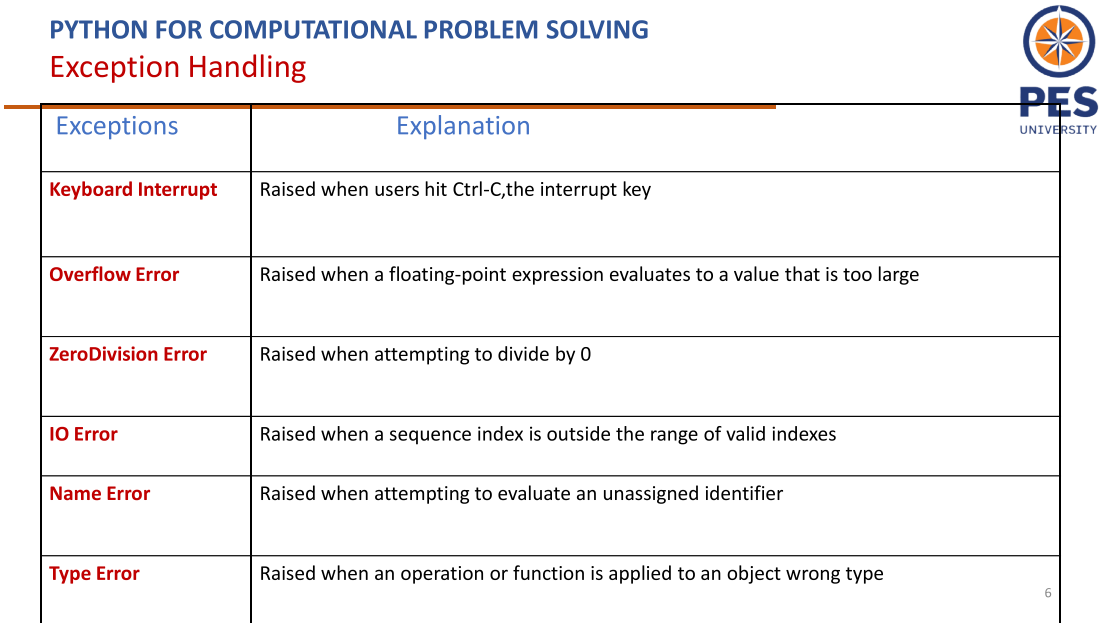


Figure G000128. UE24CS151A PCPS Unit 4 SindhuRPai Class 14 15 Exceptional Handling · source slide page 6.

EV0000484

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Exception Handling

PES UNIVERSITY

Exception Handling(How to deal with errors?)

- Exception handling is a way to deal with errors or exceptional situations that may occur during the execution of a program.
- It allows you to gracefully manage these situations instead of letting the program crash.
- Python provides try, except, else, and finally blocks for handling exceptions.

Constructs used in Exception handling -

try, except, else, finally, raise

7

Figure G000129. UE24CS151A PCPS Unit 4 SindhuRPai Class 14 15 Exceptional Handling · source slide page 7.

EV0000487

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Exception Handling

PES UNIVERSITY

try:

- Contains the code which might cause an exception
- It may have more than one except clause to specify handlers for different exceptions.
- At most one handler(except) will be executed.
- Handlers only handle exceptions that occur in the corresponding try-block, not in the other handlers of the same try block.

8

Figure G000130. UE24CS151A PCPS Unit 4 SindhuRPai Class 14 15 Exceptional Handling · source slide page 8.

EV0000490

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Exception Handling

PES UNIVERSITY

except :

- The exception thrown in try block, is caught in except block and the statements in this block are executed.
- Can have multiple exceptions as a parameterized tuple.
- If many except clauses are there, then the **specific classes are specified first** and **then the parent classes are specified.**
- Last except clause provide a common and default way of handling all exceptions.

9

Figure G000131. UE24CS151A PCPS Unit 4 SindhuRPai Class 14 15 Exceptional Handling · source slide page 9.

EV0000493

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Exception Handling

PES UNIVERSITY

else :

- This block will be executed if no errors were raised.
- That is , this block gets executed when try is successful

Finally:

- The statements in this clause will executed regardless of whether an exception occurred or not in the try block.
- Its primary purpose is to perform **clean up actions or tasks and close the resources** used in the program.

raise

- Is used to forcefully throw an exception

10

Figure G000132. UE24CS151A PCPS Unit 4 SindhuRPai Class 14 15 Exceptional Handling · source slide page 10.

EV0000496

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

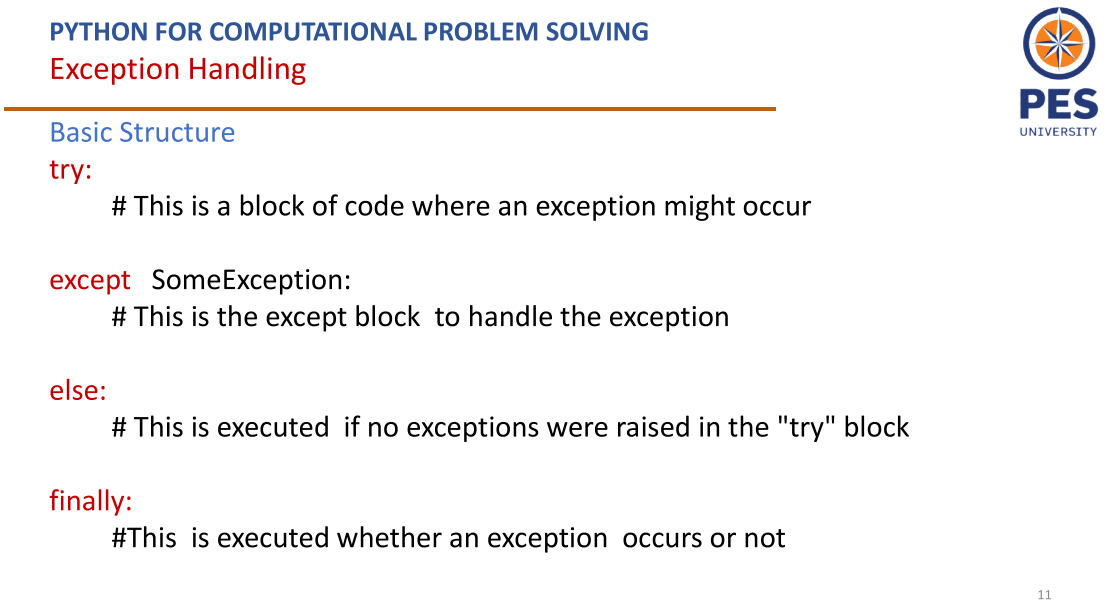


Figure G000133. UE24CS151A PCPS Unit 4 SindhuRPai Class 14 15 Exceptional Handling · source slide page 11.

EV0000499

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Exception Handling

PES UNIVERSITY

Example 1: (with try and except only)

```
try:
        print(x)

except:   #default block

        print("An exception occurred as x is not defined ")
```

Output

An exception occurred as x is not defined

12

Figure G000134. UE24CS151A PCPS Unit 4 SindhuRPai Class 14 15 Exceptional Handling · source slide page 12.

EV0000502

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

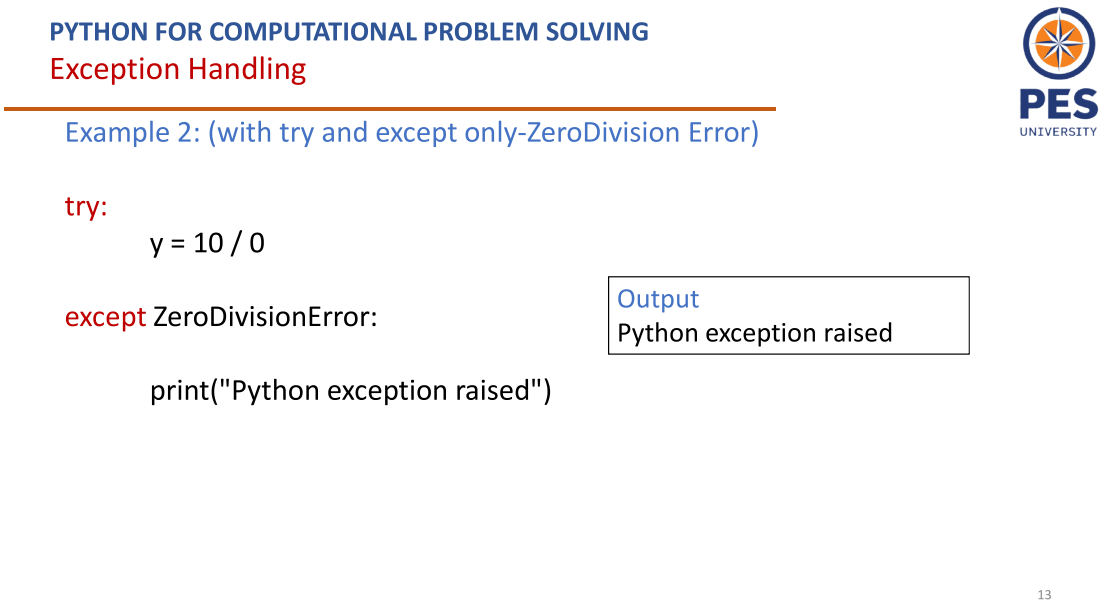


Figure G000135. UE24CS151A PCPS Unit 4 SindhuRPai Class 14 15 Exceptional Handling · source slide page 13.

EV0000505

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Exception Handling

PES UNIVERSITY

Example 3: (with try,except ,else and finally)

```
try:
    result = 10 / 0          # This will raise a ZeroDivisionError

except  ZeroDivisionError as e:
    print("Error:", e)       # Handling the ZeroDivisionError

else:
    print("No exceptions occurred.")

finally:
    print("This will always execute, regardless of exceptions.")
```

Output:

```
Error: division by zero
This will always execute, regardless of exceptions.
```

14

Figure G000136. UE24CS151A PCPS Unit 4 SindhuRPai Class 14 15 Exceptional Handling · source slide page 14.

EV0000508

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

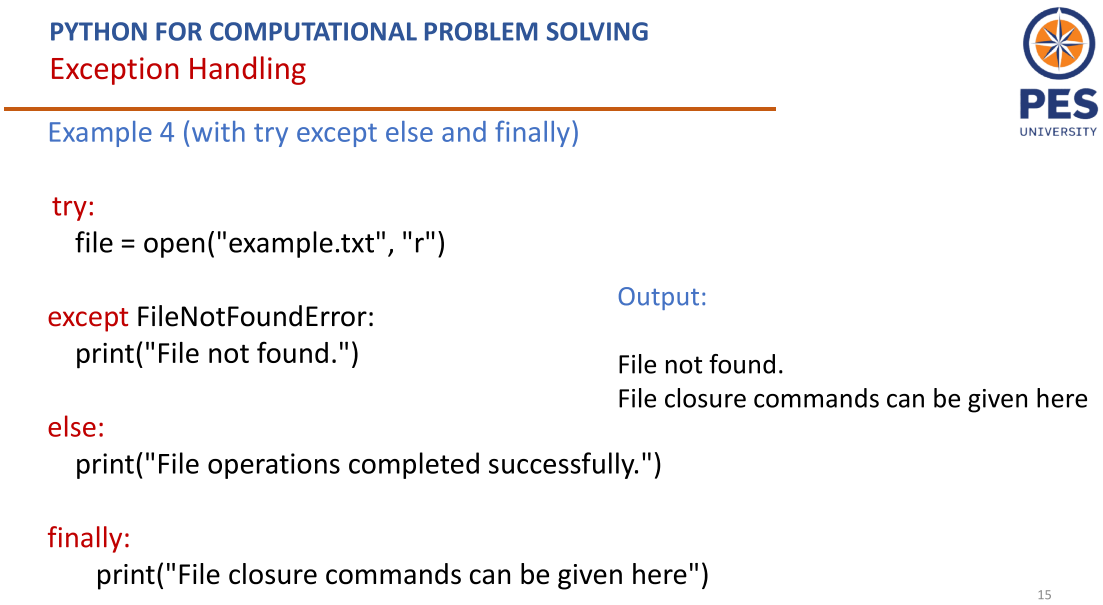


Figure G000137. UE24CS151A PCPS Unit 4 SindhuRPai Class 14 15 Exceptional Handling · source slide page 15.

EV0000511

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Exception Handling

PES UNIVERSITY

Raising Exceptions:

Exceptions can be raised manually using the raise statement, which raises the custom exceptions. Ca also be used to raise builtin exceptions.

Example 5

```
try:
   age = int(input("Enter your age: "))

   if age < 0:
      raise ValueError("Age cannot be negative.")

except ValueError as e:   #as keyword is used to create an object
   print("Error:", e)
```

Output:

```
Enter your age: -7
Error: Age cannot be negative.
```

16

Figure G000138. UE24CS151A PCPS Unit 4 SindhuRPai Class 14 15 Exceptional Handling · source slide page 16.

EV0000514

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Exception Handling

PES UNIVERSITY

Accessing Exception Information:

When an exception occurs, you can access information about the exception using "as" to assign it to a variable.

```
try:
    res= 20 / 0
except Exception as e:
    print("Exception type:", type(e).__name__)
    print("Exception details:", e)
```

Output:

```
Exception type: ZeroDivisionError
Exception details: division by zero
```

17

Figure G000139. UE24CS151A PCPS Unit 4 SindhuRPai Class 14 15 Exceptional Handling · source slide page 17.

EV0000517

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Exception Handling

PES UNIVERSITY

Matching of except blocks:

•The raised or thrown exception object is matched with the except blocks in the order in which they occur in the try-except statement.

•The code following **the first match is executed.**

•It is **always the first match and not the best match**.

• If no match occurs and if there is a default except block, this will be executed.

18

Figure G000140. UE24CS151A PCPS Unit 4 SindhuRPai Class 14 15 Exceptional Handling · source slide page 18.

EV0000520

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Exception Handling

PES UNIVERSITY

Handling Multiple Exceptions:
Example 6

```
try:
    num1 = int(input("Enter first number: "))
    num2 = int(input("Enter second number: "))

    result = num1 / num2      # This might raise a ZeroDivisionError

    my_list = [1, 2, 3]

    index = int(input("Enter an index for the list: "))
    value = my_list[index]      # This might raise an IndexError
```

19

Figure G000141. UE24CS151A PCPS Unit 4 SindhuRPai Class 14 15 Exceptional Handling · source slide page 19.

EV0000523

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

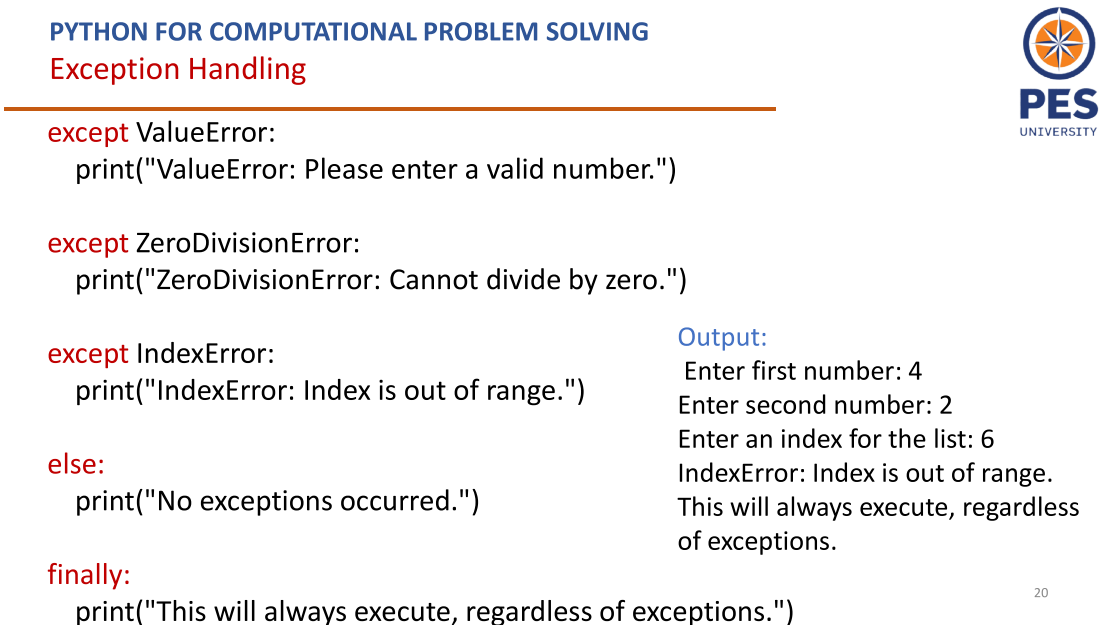


Figure G000142. UE24CS151A PCPS Unit 4 SindhuRPai Class 14 15 Exceptional Handling · source slide page 20.

EV0000526

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Exception Handling

PES UNIVERSITY

2.User defined exceptions

•Users can define custom exceptions by creating a new class.

•This **exception class has to be derived, either directly or indirectly, from the built-in Exception class**.

•Most of the built-in exceptions are also derived from this class.

•The new exception, like other exceptions, can be raised using the raise statement with an optional error message.

21

Figure G000143. UE24CS151A PCPS Unit 4 SindhuRPai Class 14 15 Exceptional Handling · source slide page 21.

EV0000529

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

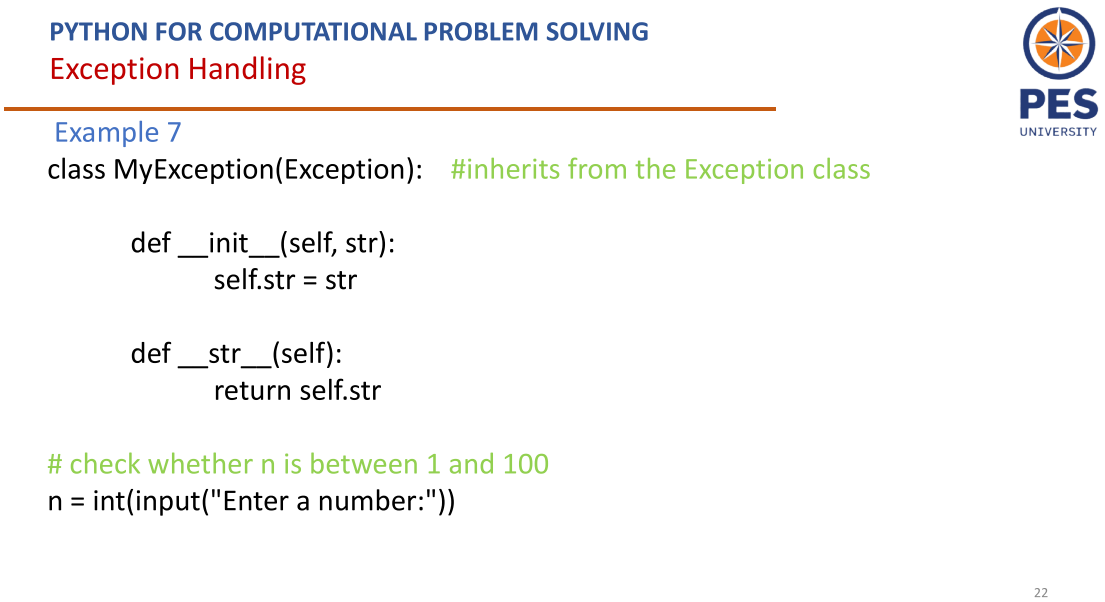


Figure G000144. UE24CS151A PCPS Unit 4 SindhuRPai Class 14 15 Exceptional Handling · source slide page 22.

EV0000532

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

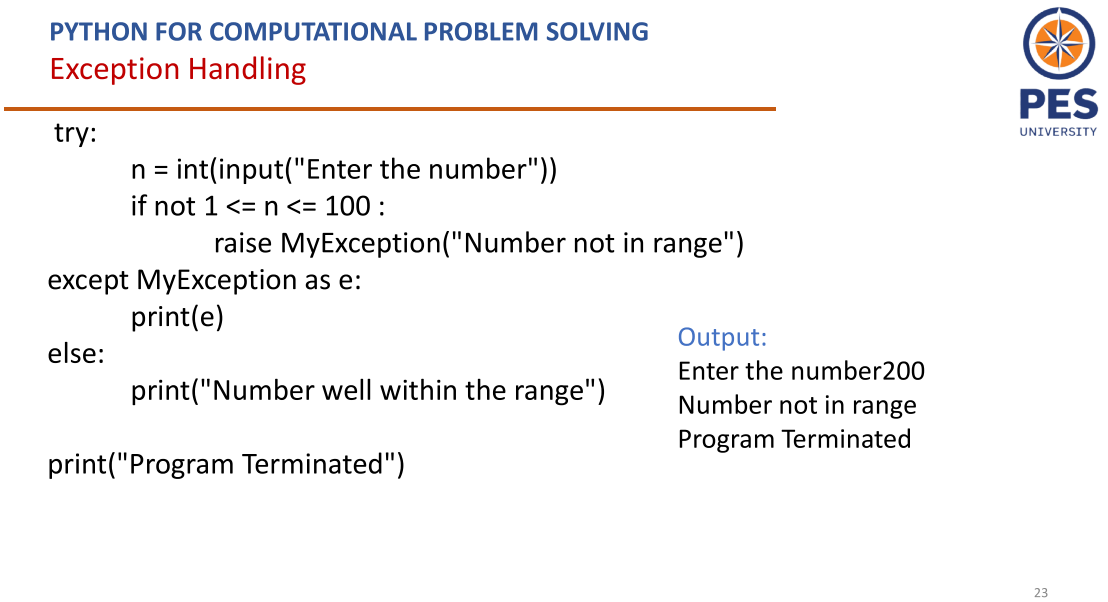


Figure G000145. UE24CS151A PCPS Unit 4 SindhuRPai Class 14 15 Exceptional Handling · source slide page 23.

EV0000535

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Exception Handling

PES UNIVERSITY

Example 8

```
class FiveDivisionError(Exception):
        pass
try:
        n1=int(input("Enter the first number"))
        n2=int(input("Enter the second number"))
        if n2==5:
                raise FiveDivisionError("Dividing by 5 not possible")
        d=n1/n2
        print("The answer is",d)
except (FiveDivisionError,ZeroDivisionError) as e:
        print(e)
print("Program ends")
```

Output:

```
Enter the first number 7
Enter the second number 5
Dividing by 5 not possible
Program ends
```

24

Figure G000146. UE24CS151A PCPS Unit 4 SindhuRPai Class 14 15 Exceptional Handling · source slide page 24.

EV0000538

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

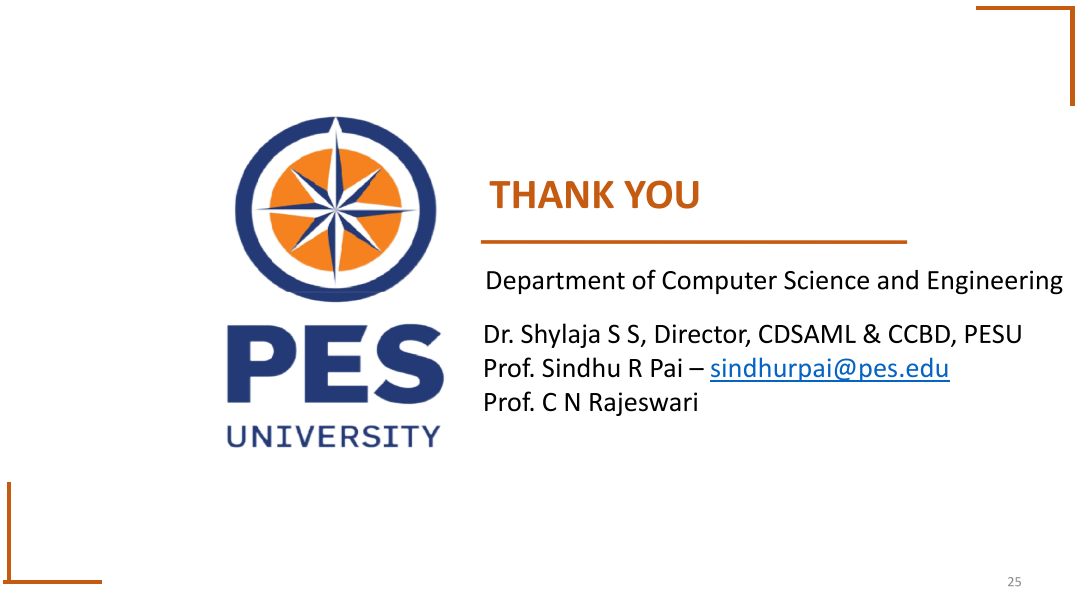


Figure G000147. UE24CS151A PCPS Unit 4 SindhuRPai Class 14 15 Exceptional Handling · source slide page 25.

EV0000541

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Important points and why

1. Source slide page 1 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Exception handling in Python Prof. Sindhu R Pai PCPS Theory Anchor - 2024 Department of Computer Science and Engineering EV0000468 · EV0000469 · EV0000470
2. Source slide page 2 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Exception Handling Introduction: • Programming error is an error which may occur because of various reasons. PProgram can produced some errors even ifif ththe program iis perfect.f t • This may be because of 1. Exceptional situations that may occur during the execution of a program. 2. The data coming from outside the program which is malformed . 2 EV0000471 · EV0000472 · EV0000473

3. Source slide page 3 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Exception Handling Based on when the error occurs, errors are classified into two types: 1. Syntax errors This occurs when there a deviation from the rules of the language. A component of python's interpreter called parser discover these errors. If hybrid interpreter, compiler finds this error. Ex: print("good(" d morning)) print("good morning“ 2. Runtime Errors (Exceptions) These are the errors which are detected during execution. This disrupts the normal flow of execution of instructions. Ex: Print("Good Morning") #NameError: No name 'Print' found 3 EV0000474 · EV0000475 · EV0000476
4. Source slide page 4 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Exception Handling Exception Tree-Partial View 4 EV0000477 · EV0000478 · EV0000479
5. Source slide page 5 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Exception Handling Two categories of Exceptions: 1. Built-In exception (System defined exceptions) Thoseh exceptions whichh h Pythonh kknows bby ddefault.f l 2. User defined exceptions User-defined type/class written for handling the exceptions 5 EV0000480 · EV0000481 · EV0000482
6. Source slide page 6 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Exception Handling Exceptions Explanation Keyboard Interrupt Raised when users hit Ctrl-C,the interrupt key Overflow Error Raised when a floating-point expression evaluates to a value that is too large ZeroDivision Error Raised when attempting to divide by 0 IO Error Raised when a sequence index is outside the range of valid indexes Name Error Raised when attempting to evaluate an unassigned identifier Type Error Raised when an operation or function is applied to an object wrong type 6 EV0000483 · EV0000484 · EV0000485
7. Source slide page 7 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Exception Handling Exception Handling(How to deal with errors?) • Exception handling is a way to deal with errors or exceptional situations that may occur during the execution of a program. • ItIt allowsll you tto gracefullyf ll manage ththese situationsit ti iinsteadt d off llettingtti ththe program crash. • Python provides try, except, else, and finally blocks for handling exceptions. Constructs used in Exception handling - try, except, else, finally, raise 7 EV0000486 · EV0000487 · EV0000488
8. Source slide page 8 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Exception Handling try: • Contains the code which might cause an exception • It may have more than one except clause to specify handlers for different exceptions. • At most one handler(except) will be executed. • Handlers only handle exceptions that occur in the corresponding try-block, not in the other handlers of the same try block. 8 EV0000489 · EV0000490 · EV0000491
9. Source slide page 9 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Exception Handling except : • The exception thrown in try block, is caught in except block and the statements in this block are executed. • Can have multiple exceptions as a parameterized tuple. • If many except clauses are there, then the specific classes are specified first and then the parent classes are specified. • Last except clause provide a common and default way of handling all exceptions. 9 EV0000492 · EV0000493 · EV0000494

10. Source slide page 10 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Exception Handling else : • This block will be executed if no errors were raised. • That is , this block gets executed when try is successful Finally: • Theh statements in thish clausel willll executedd regardlessdl off whetherh h an exception occurred or not in the try block. • Its primary purpose is to perform clean up actions or tasks and close the resources used in the program. raise • Is used to forcefully throw an exception 10 EV0000495 · EV0000496 · EV0000497
11. Source slide page 11 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Exception Handling Basic Structure try: # This is a block of code where an exception might occur except SomeException: # Thish is theh except blblockk to hhandledl theh exception else: # This is executed if no exceptions were raised in the "try" block finally: #This is executed whether an exception occurs or not 11 EV0000498 · EV0000499 · EV0000500
12. Source slide page 12 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Exception Handling Example 1: (with try and except only) try: print(x) except: #default block print("An exception occurred as x is not defined ") Output An exception occurred as x is not defined 12 EV0000501 · EV0000502 · EV0000503
13. Source slide page 13 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Exception Handling Example 2: (with try and except only-ZeroDivision Error) try: y = 10 / 0 OutputOutput except ZeroDivisionError: Python exception raised print("Python exception raised") 13 EV0000504 · EV0000505 · EV0000506
14. Source slide page 14 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Exception Handling Example 3: (with try,except ,else and finally) try: result = 10 / 0 # This will raise a ZeroDivisionError except ZeroDivisionError as e: print("Error:",(" " e)) # Handlingdl theh ZeroDivisionError Output: else: Error: division by zero print("No exceptions occurred.") This will always execute, regardless of exceptions. finally: print("This will always execute, regardless of exceptions.") 14 EV0000507 · EV0000508 · EV0000509
15. Source slide page 15 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Exception Handling Example 4 (with try except else and finally) try: file = open("example.txt", "r") Output:Output: except FileNotFoundError:l d print("File not found.") File not found. File closure commands can be given here else: print("File operations completed successfully.") finally: print("File closure commands can be given here") 15 EV0000510 · EV0000511 · EV0000512
16. Source slide page 16 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Exception Handling Raising Exceptions: Exceptions can be raised manually using the raise statement, which raises the custom exceptions. Ca also be used to raise builtin exceptions. Example 5 try: Output:Output: age = int(input("Enter((" your age: "))")) Enter your age: -7 Error: Age cannot be negative. if age < 0: raise ValueError("Age cannot be negative.") except ValueError as e: #as keyword is used to create an object print("Error:", e) 16 EV0000513 · EV0000514 · EV0000515
17. Source slide page 17 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Exception Handling Accessing Exception Information: When an exception occurs, you can access information about the exception using “as” to assign it to a variable. Output:Output: try: Exception type: ZeroDivisionError Exception details: division by zero res= 20 / 0 except Exception as e: print("Exception type:", type(e).__name__) print("Exception details:", e) 17 EV0000516 · EV0000517 · EV0000518

18. Source slide page 18 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Exception Handling Matching of except blocks: •The raised or thrown exception object is matched with the except blocks in the order in which they occur in the try-except statement. •Theh coded ffollowingll theh ffirst matchh is executed.d •It is always the first match and not the best match. • If no match occurs and if there is a default except block, this will be executed. 18 EV0000519 · EV0000520 · EV0000521

19. Source slide page 19 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Exception Handling Handling Multiple Exceptions: Example 6 try: num1 = int(input("Enter first number: ")) num2 = int(input("Enter second number: ")) result = num1 / num2 # This might raise a ZeroDivisionError my_list = [1, 2, 3] index = int(input("Enter an index for the list: ")) value = my_list[index] # This might raise an IndexError 19 EV0000522 · EV0000523 · EV0000524

20. Source slide page 20 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Exception Handling except ValueError: print("ValueError: Please enter a valid number.") except ZeroDivisionError: print("ZeroDivisionError: Cannot divide by zero.") Output:except IndexError: Enter first number: 4 print("IndexError: Index is out of range.") Enter second number: 2 Enter an index for the list: 6 else: IndexError: Index is out of range. print("No exceptions occurred.") This will always execute, regardless of exceptions. finally: 20 print("This will always execute, regardless of exceptions.") EV0000525 · EV0000526 · EV0000527

21. Source slide page 21 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Exception Handling 2.User defined exceptions •Users can define custom exceptions by creating a new class. •This exception class has to be derived, either directly or indirectly, from theh bbuilt-inl Exception class.l •Most of the built-in exceptions are also derived from this class. •The new exception, like other exceptions, can be raised using the raise statement with an optional error message. 21 EV0000528 · EV0000529 · EV0000530

22. Source slide page 22 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Exception Handling Example 7 class MyException(Exception): #inherits from the Exception class def __init__(self, str): self.str = str def __str__(self): return self.str # check whether n is between 1 and 100 n = int(input("Enter a number:")) 22 EV0000531 · EV0000532 · EV0000533

23. Source slide page 23 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Exception Handling try: n = int(input("Enter the number")) if not 1 <= n <= 100 : raise MyException("Number not in range") except MyException as e: print(e)( ) Output:else: Enter the number200 print("Number well within the range") Number not in range Program Terminated print("Program Terminated") 23 EV0000534 · EV0000535 · EV0000536

24. Source slide page 24 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Exception Handling Example 8 class FiveDivisionError(Exception): Output: pass Enter the first number 7 try: Enter the second number 5 n1=int(input("Enter the first number")) Dividing by 5 not possible n2=int(input("Enter((" theh secondd number"))b ")) Program ends if n2==5: raise FiveDivisionError("Dividing by 5 not possible") d=n1/n2 print("The answer is",d) except (FiveDivisionError,ZeroDivisionError) as e: print(e) print("Program ends") 24 EV0000537 · EV0000538 · EV0000539

25. Source slide page 25 (extracted text): THANK YOU Departmentp of Computerp Science and Engineeringg g Dr. Shylaja S S, Director, CDSAML & CCBD, PESU Prof. Sindhu R Pai – sindhurpai@pes.edu Prof. C N Rajeswari 25 EV0000540 · EV0000541 · EV0000542

SHORT-NOTE RECAP

Transcript block 1 (source interval 000000-000299 seconds): [00:00:02.724] Yeah. [00:00:12.260] So previous static variables, right? [00:00:26.596] We will see one more small example. [00:00:36.900] Unless you pay attention. [00:00:50.500] So this is a typical example where you can make out difference between a static variable and a normal variable. So here you can see your count this count which has been defined inside your class fruits. [00:01:10.596] Can see here I have created a object called [00:01:17.284] Taking the fruit name and the count of the fruits. Okay. So when I look at apples as an object, you can see this count belongs to is this also a class attribute? Or a static attribute? Is this count? I have two count. [00:01:36.996] So it's evident that since I told self.count, self is nothing but the object that I've created. So this will be apples.count. So this count just belongs to that one particular object that we have created because it is self.count. But when we look here. [00:01:53.536] I am saying fruits.com. We know that fruits is nothing but my class name. So this becomes a class variable, which is a static variable. So what I'm doing here that every time I create an object fruit, I have count for its own. [00:02:10.144] kind of root and I have template of sum of total number of roots. Okay so since we I'm basically the object which is changing my static variable is class itself. So when class itself changed the value of the static variable will it reflect to all the objects or not? [00:02:30.916] It will reflect basically when I try to access this roots.com, obviously I'll get the [00:02:39.108] simulative so apples i'll get apple count as three and food count as three where i make another object piece you can see that it is name of the food count as four now food star count should be three plus four seven is it clear any doubts there [00:02:58.596] Okay. So you can see here when I make apples.count, this I'll get it as three. When I say peach dot count, I'll get it as four. When I say fruits.count, we will get it as seven. [00:03:15.588] Four plus three seven because here you can see I made a cumulative stuff. [00:03:20.804] If at all I was only thinking of a local variable that is non-static variables, I cannot do cumulative sum like this because it does not carry the same value across all the objects. So here I can do it because this is a static variable. Okay [00:03:38.980] So other ways of actually accessing your static variable is this. The apples is an object. I'm trying to access the class name of my object apples. So apples dot class is nothing but fruits, right? So it is as fruit as calling. [00:03:55.872] This is another syntax to access your class variable to the static variable, and you can see a type of A. What do I get say if I take a variable A equal to 10 and I check for type of A? What will be type of A? It would be [00:04:13.892] Right? So here I'm saying type of page. So what will be the value? Class is a user defined type, right? So when I say type of space, what will be the value? It will be [00:04:29.060] It's fruits, right? In our case, it's fruits. So it is again nothing but fruits.com. So this is another way of finding the I mean accessing the classic. [00:04:41.476] Is it clear you need to be familiar with all possible syntaxes because when you will be given questions to find out or error, if you don't know these syntaxes, then you'll end up not writing it properly. [00:04:55.776] So it was it is quite logical to understand here also. Apples classes foods of similarly type software.com any doubts

EV0000062 · EV0000063 · EV0000064 · EV0000065

Transcript block 2 (source interval 000300-000599 seconds): [00:05:19.140] One second. Comment that operator overloading. [00:05:47.268] Okay, this is this is from another program, you could just ignore this. [00:06:04.132] Okay, can you relate here? Apples.count three, apple peach.com four, fruits.count seven, and then apples.class.count seven, and begin type of peaks.count seven. Any doubts or any clarification? Take down a question quickly. [00:06:22.756] create a class create a class marks create a class calculate marks [00:06:37.380] In which in which a student's in which a student's individual subject marks [00:06:51.428] For ISA 1, ISA 2 and ESA. [00:07:00.132] Each individual subjects, ISA one, ISA2 and ESA Marks is initialized. [00:07:13.828] [ASR low confidence; check audio] Except, except. [00:07:18.820] [ASR low confidence; check audio] ISA 1, ISA 2, and ISA marks for. Accept ISA 1, ISA 2, and ISA marks for. [00:07:32.740] Chemistry Python Max [00:07:41.892] Civil and electronics and electronics find the total marks score find the total marks score and and [00:08:01.380] Also find and also find percentage and percentage [00:08:21.220] Did all of you get an idea why will you be doing this? You will create a class called as calculate marks, right? And in your init function you will be taking ISA one, ISA2, and ESA Mark. So how many objects you will be creating? [00:08:41.348] How many objects? [00:08:46.052] How many objects will you get to get? [00:08:52.100] How many subjects have I taught? Five subjects. Right? So don't you think you should be doing like this? [00:09:06.916] I'll not write the entire code that you people write just to tell how you should be using a class variable only that I'll tell. [00:09:17.380] This is your class. I will just say should be taking [00:09:29.380] [ASR low confidence; check audio] Okay, ISA 1, ISA 2, and ESA marks, right? You make respective assignments, right? [00:09:44.676] Or five subjects. So I'll be creating something like this. [00:09:54.948] Yeah, she looks like this.

EV0000066 · EV0000067 · EV0000068 · EV0000069

Transcript block 3 (source interval 000600-000899 seconds): [00:10:03.300] Okay. So like this you will be creating for five subjects. Every time say I'll get ISA one, Isa 2, and ESA for math, similarly to Python, etc. Now you also need to calculate the total marks and percentage. So can I write like this? [00:10:29.412] And then in my can I write what should I write to access total marks here? [00:10:46.564] [ASR low confidence; check audio] As. Equal to. [00:10:52.356] All these right? self.isa1 plus self.isa2, etc. [00:11:08.132] [ASR low confidence; check audio] Yes, under the internet country. [00:11:14.980] No, but still you need to take self. Why? Because this uh this is with respect to math object, right? [00:11:24.548] Yeah. [00:11:28.196] No, you need to be whatever I write here though that if at all this I discussed when I did the first program if at all I you might take like this dot [00:11:42.980] A1 is equal to IC1, then it is self dot A1. It is not ISA1. So basically what this just happens to be parameters. These are your variables. So I have to say self dot. [00:12:06.084] Any clarifications to complete this? Okay, execute ones to get a good idea about your static variables. [00:12:20.612] We'll go to static methods now. Now the way we have static variables, we can also have static methods. [00:12:33.636] methods are common across the objects okay it is a common method which is present across all the objects [00:12:46.500] static function or static method okay so how do i make out that it is common across any object you can say i'm not passed so when do i pass cell whenever i want the demarcation as to this [00:13:04.004] Basically executing for which particular object. That's why my I pass a self here, right? So when this has to execute, is it with respect to S1 or is it with respect to S2? [00:13:15.280] I I need to make out that accordingly it will execute so we know that this is with respect to objects, so it will be for respect to objects only. But when you're looking at these static methods, these are the common methods

across any objects. So it does not [00:13:34.276] Like how we are discussed every method inside a class will have one parameter by default that is cell that will take name of the object, right? But your [00:13:46.656] it does not depend on the object so it will not take any parameter like self if you wanted to make any other calculation you can pass the parameters but self is not necessary by because it's a common method any doubt okay so uh [00:14:05.220] With respect to both static method and static variables, I have done this. So you can say I have just taken name and age. Okay and basically count. [00:14:16.516] Now here say I've created a uh object called S Pen, which is a student I'm passing name H. So name and H here. Now when I call Spen dot display, it basically goes here. So this also I got an [00:14:33.648] Why because I had to tell you all dictionary. If at all I say self-taught dictionary, what happens is that all its variables become keys and whatever value I've assigned will become value. [00:14:47.024] So should this dictionary look like it should be name because name is my variable name name colon whatever I have passed ram here, right? So name colon ra and then age colon 90 and count colon one. Why because with respect to each object every time I'm

EV0000070 · EV0000071 · EV0000072 · EV0000073

Transcript block 4 (source interval 000900-001199 seconds): [00:15:04.548] equal to one so count will be equal to one now student dot count is a class variable or the static variable so it will count number of students okay as you know given application like accept details of a student [00:15:20.288] One after the other and also get me the total count of the students who have registered. If this is the case, then this give me total number of students. Okay. [00:15:32.260] So now you can see I have called the static method display count. Okay, I have called the static method display count. When I go here, it basically accesses whatever is necessary for the student document. [00:15:49.828] Okay, and you can see here this one that is at the rate of static method, it's only optional, but this is optional when you are actually calling your [00:16:01.476] method like this but if you want to call your method with respect to object then you need to be giving this syntax that is the decorative syntax any doubts any doubt is it clear [00:17:01.604] Okay. [00:17:06.212] And this is with respect to [00:17:14.084] Any clarifications? So in case if I commend this the last statement can not expect forward via object. [00:17:31.620] Okay, so whenever I want to basically call it via object, need to be giving you a decorative syntax. [00:17:57.156] Please do not write your uh assignments in the class. Any other subject assignments, if at all you are writing, please do not write in the class. If at all if somebody is writing, please do not write. [00:18:13.412] Any clarifications, any doubts? [00:18:19.172] My everyone are looking very dull. You had a lot of work. [00:18:25.988] Okay, at least are you able to understand. If you don't write it's okay, anyway I'll be sending this document. Just see that you understand. [00:18:44.932] Okay. This is a simple program where I've used the static method and I'm finding if a person has a second name or not. All of you know this logic, right? Basically, when I get a string, I can split based on spaces. So I'll get the list, right? So I'll get list of these. [00:19:03.072] Uh string complete string, right? Scott will be one string and robin will be second string. So whenever the length of those strings are greater than one, obviously it's a second thing. Just remember this. [00:19:18.916] Remember this logic. I just gave as an extra example, that's all. [00:19:27.332] And the last topic before we end, take down composition. Object composition. [00:19:39.556] You have seen inheritance, right? So what is inheritance? [00:19:52.452] Yeah. [00:19:56.676] So basically all the characteristics of base classes can be utilized in the derived class and that is called as inheritance and you know the syntax also right say when I say class B

EV0000074 · EV0000075 · EV0000076 · EV0000077

Transcript block 5 (source interval 001200-001499 seconds): [00:20:08.932] The parameter A that means to say that B is a derived class and A is a space class. Okay. So similar to that, we have something called as object composition. Here it is something like a class as another class within itself. Okay. [00:20:26.724] I have two independent classes. [00:20:29.604] A and then say I have class B. If class B creates an object of class A, that means that that object can actually access all the characteristics of my class A. So that is called as object composition. So in this case. [00:20:53.156] You can see here I have a class called as component. Okay. And I have a method called as M1. All simple methods. I've just written the phrases. Nothing uh logical in this. Now I've created a class call as composite. Okay. [00:21:10.288] What is that it makes it as an object composition specifically. I'm saying self dot object one is equal to component. That means to say that I am creating an object of one class inside another class. [00:21:27.492] Okay and this is called composition. It composes the characteristics of some other class. Why? Because I have created an object of that class inside. [00:21:41.380] Okay, so basically now on I can actually access all the features of my component function or component class. Any doubts? [00:21:59.844] So here I have created obj2, which is of type composite, which calls its own init function in which I have created an object, another object, which is of component. Now if I say obj 2.m2, see here in M2, I have called M1. [00:22:20.452] Okay, this is possible because I had created an object obtain of type component. Any doubts? Any doubts, people? [00:22:37.188] Okay, so you can just observe here. [00:22:41.412] It says component class object created that happier self.object one. So composite class objects created. And then I basically make HBJ2. So it creates this. Okay, it goes to init function of this. [00:22:59.012] Okay, it says composite class object is also created, and then I've called m2, that is obj 2.m2. So m2 method is called, and then inside my m2, I have called m1 method, so m1 method is called actually m1 is a method that belongs to my other class. [00:23:18.564] Any doubts? Any doubt? You want to write to understand? Or if it's okay the next topic. I'll just say this, you can analyze once. [00:23:48.420] Difference between inheritance and composition is that in inheritance you will not create any object of the other class, but you will need that inherited as a parameter. So you can access all the pictures here. You have not inherited, but you have created object of another class.

EV0000078 · EV0000079 · EV0000080 · EV0000081

Transcript block 6 (source interval 001500-001799 seconds): [00:25:18.308] S students are present now. [00:25:24.996] So five are absent. It's Vincent, Aditya, Krishna, Anshal, Abai, R, Abhishek, yes. Abhishek is Kupri. You were present. Morning also. Morning you were absent. [00:25:46.308] Please remove your mask when you present morning. [00:26:57.348] So previous one week these these are the things that we have covered. You can just recollect. [00:27:06.596] Objects are basically those variables which will allow you to access the characteristic features of the class, or you can also say object is an instance of a class, and then [00:27:21.348] functions inside a class by default it will have variable parameter which is called a self unless and until it is a static method okay you studied about constructors you studied about parametrized and non-parameterized constructors [00:27:38.468] And then you studied about these default functions which are automatically accessed when you pass this can be access when you pass a string to another function, right? And then you studied about access modifiers, public and private, right? By default, what are the type of methods and variables inside a class? [00:27:59.012] It is private All of you are so colour. It will be what public or private? By default it will be [00:28:12.388] Public. Otherwise if it needs to be private, then you need to prefix the underscore, underscore. Then you studied about

inheritance. You studied about multiple inheritance. And then you studied out to call in it function, huh? [00:28:27.396] Base class, uh from base class, I mean from derived class, you also call base classes in its function, and you also studied about super method, which basically allows you to access methods and in it method from your base class. [00:28:42.944] You studied about static variables, static methods, operator overloading, and then you studied about composition and you also studied about [00:28:58.468] Function overloading, right? Or method overloading. [00:29:12.612] Okay. So you can just recollect. [00:29:19.268] Remember all these concepts. Any clarification among all these topics? Any doubts? Is it clear? Is it clear to all of you? Hope you have been doing your own work problem that I'm giving it take only 10 to 15 minutes. [00:29:40.420] What?

EV0000082 · EV0000083 · EV0000084 · EV0000085

Transcript block 7 (source interval 001800-002099 seconds): [00:30:08.868] Take down the next topic exception handling. Exception handling. [00:30:24.932] As the name says when we say exception, always it'll be normal except for certain reasons it might not work. So that's an exception, right? Something like in your timetable. [00:30:37.040] Whichever subjects are there, those subjects will be taught on those respective days unless and until the faculty is absent. So that subject will not happen on a particular day, only on an exception that faculty might not. [00:30:52.036] So in normal sense everything will go fine, but on certain conditions it might not work, and that is called as an exception. Okay. So you we have been studying about errors, right? What are more kind of errors we have? We have syntax errors, we have semantic errors. [00:31:08.592] We don't have any problem with syntax errors, because your interpreter is capable of handling it. Right? When it comes to your semantic error, we will have to deal with it. [00:31:20.064] Right, so one of the way we dealt with it is by testing. We saw by test and we saw it basically tells us where exactly we have gone wrong. So that is one way of handling your uh samar. One more way is your exceptions. So what does your exception does is that [00:31:39.652] It basically tells us that how we need to handle those semantic errors. Okay, there are two ways of handling your exception. So one is by uh by your interpreter itself. Something like when I try to divide a number by zero. [00:31:57.636] And uh say for example I have a list, I have five elements in the list, I'm trying to access the sixth element in the list. Okay all such what your interpreter could basically expect, those set of exceptions are already there in a class called as exception. [00:32:14.852] So Python has a class, built in class called as exceptions. And in that exception, all these inbuilt exceptions are being defined. Something like you know divide by zero, index out of form. All these exceptions have been given. Okay. [00:32:30.352] But say for example, I'm basically taking your map and maximum marks is two hundred. And I try to enter something about two hundred that cannot be anticipated by your interpreter, right? Why? Because that is application oriented. My application maximum marks is two hundred. [00:32:47.264] it need not be same always so such kind of application dependent semantic errors should be handled by the programmer itself okay by writing a user defined exception is that clear so [00:33:04.420] Basically, when I when we talk about exception handling, we will think about the errors. So syntax error we don't think of and we think about only semantic errors. [00:33:19.236] Right. So this this is very clear to us. This is shown by my interpreter itself. If I take AB and I say C equal to AB to give me a error, right? So now we'll look at exceptions. I told about written exceptions and user defined exceptions. [00:33:37.284] So these are few of the written exceptions, which are already there in your uh interpreter, but you can override on this. It's not necessary that I should be using the

same message. [00:33:50.320] I can also override on this like how we did overriding of underscore underscore add like that you can also overwrite these exceptions, but in bed I have certain uh uh phrase about these exceptions. Say for example, uh keyboard interrupt, I am trying to kill a process which is under exception. [00:34:10.020] So we have a exception for that. Zero division error. I'm trying to divide by zero. So we we can make outright any division is possible, but only exception is that when I have a denominator zero. [00:34:23.780] Then we have IO error. Then I told about index error. So these are the few of the errors which are in bit. Now name error. Okay so I am trying to access some variable which is not there in my old program. I don't have a variable called as byte. [00:34:40.708] But still I try to print my that means that it's a name error because I don't have a variable of that. [00:34:47.140] And we have a type error. I'm trying to add uh a string and a number, right? I cannot do that because there's a type error. I cannot add a string and a picture. So that's a type error. And we have something called as value error.

EV0000086 · EV0000087 · EV0000088 · EV0000089

Transcript block 8 (source interval 002100-002399 seconds): [00:35:04.804] So uh similarly your value error. So we'll see if you have the example executed here. I have x equal to 10, it's an interactive mode, and I try to print y. [00:35:18.468] Okay, so you can see here I have a name error because Y is not defined. Here I'm taking a number and this is also a number but it's in form of a string, right? So I'm trying to add a string and an integer. So you can see here I have a type error here. Because type mismatch. [00:35:36.900] Here I have index. There's no where number call is 40 in my list, but I'm trying to find an index where 40 resides. So that is not there, so it's a value error. I'm trying trying to access some value which is not there, so it's a value error. [00:35:52.624] And if I'm trying to say in this case itself, I try to access say L of four. I have zero one two index. If I try to access L of 4 index, fourth index is not there, so it will be index out of bounds exception. Okay. [00:36:08.836] These are the common things that you need to remember. [00:36:13.540] Okay you would have come across this when you were programming but you would have not observed so these are the common errors so yeah I have given you a few examples when I just use six divided by zero in my interception this is the error which comes [00:36:29.412] Then I have a list try to access this. I've got axe of the form. And then A plus 10, it's a name error because I don't have a variable called A and I'm trying to increment it by 10. So it's a [00:36:42.468] Name error, tenure, two and six. I cannot multiply two strings, so it's the type error. Now you're [00:36:50.820] Here I cannot convert this floor which is in form of a string to an integer. I cannot type cast this, right? So it's a value error. Similarly, here it's a very huge number, so it's a error or exception. [00:37:06.144] These are all the exceptions. Any dogs? I'm just going fast because it's simple and nobody's asking me anything. So [00:37:15.908] So we'll see how we will handle exceptions. Simple, what we will do is that I have a program. So those set of instructions which I feel it might undergo some kind of semantic error, that I'll put it in a block, and that block is called as a tri block. Okay. [00:37:33.408] So your all those need not be the entire program, only those statements which you have you anticipate to have an exception, you put it under this right block. Okay, and now you will handle it. You will we can use we will use a keyword called as except. [00:37:50.320] Which basically this block of code which is under accept will get executed only if an exception is raised in the right block. I try to say it's A divided by B, I accept both values of A and B. [00:38:05.252] And if it is say A is 100 and B is 20, so there's no exception, right? So this except block will not be executed. But when I give value of A as 10 and value of B as zero, then there will be an exception at that point of time. [00:38:19.908] Set up instruction which is under it, okay? Executed. Any

doubts? [00:38:27.364] [ASR low confidence; check audio] Okay. [00:38:32.580] A simple example, file print x, an exception occurred as it is, not the fight. Okay. We will just see what happens if I just executed it as it is. [00:39:09.380] Execute this [00:39:21.796] Okay. This is what we get.

EV0000090 · EV0000091 · EV0000092 · EV0000093

Transcript block 9 (source interval 002400-002699 seconds): [00:40:17.796] Okay, can you make out the difference? When I did not use except, this is uh your error is shown by your interpreter. Obviously, if you are writing a application for your client, he would not like to see this. Why? Because he doesn't understand what this is. [00:40:32.912] Unless Until he is a programmer, he will not understand what this usually clients say it will be non-programmer, right? So they are very comfortable with such kind of statements. Okay, so the difference when I handle exception and when I don't handle exception. This is not handle, this is [00:41:17.188] Okay. Okay people listen. [00:41:23.428] No, this is here I had handled it like this, a tri block and a accept block. I can also handle it by giving our [00:41:33.668] Inbuilt exception name. Say for divide by zero, I have inbuilt exception. So I can say except zero division error. So this is the name of the exception, which is built in in your interpreter. Okay. This is how we can add it by using [00:41:51.056] [ASR low confidence; check audio] Inbuilt exception, this is how we end. Okay. [00:41:59.492] Here again we have handled the name error. So I have just handled the name error here. So what is happening actually? I'm dividing it by zero. So I have used the wrong exception name error. [00:42:13.316] Okay, so basically it is giving me the error because it's not a name error, right? Actually y equal to one divided by zero was a zero division error. But while writing my except block, I wrote I am [00:42:26.352] Use a uh wrong inbuilt exception. So anyway, even if I do that, it will not uh basically affect it will show the right exception. Okay [00:42:39.204] So how many exceptions can I have? Say my trial block has some 10 instructions. Okay. In that I can accept that could be a divide by zero, that could be an index of, it could be something else as well. [00:42:52.336] So like that, I have so many uh exception which might occur in my pride block. So how many except blocks can I have? I can have any number of except blocks. [00:43:03.044] Okay, in this case you can see I have two except blocks, one is a type error, one more is a zero division error. So like this I can have any number of except blocks for a tripod. For a single tribal, you can have any number of except blocks. [00:43:20.196] So this statement is something which will execute irrespective of whether there was an error or there was no error. Okay. In this case, when an exception was raised, so divide by zero error was raised. So it says division by zero not allowed. And then you can see out of [00:43:39.396] Try accept block. So these are common set of statements which are out outside my triexcept block. So what do we mean by saying that even after you have an exception, you can have set of other statements also. It doesn't mean that there's an exception, it will not execute the rest of the code, it will still execute the rest of the code. [00:43:59.972] Can you doubt? [00:44:27.588] Okay. [00:44:30.980] okay people see it so which when i have a multiple accept block [00:44:37.924] How does it basically consider which except block to be executed when there is some kind of exception? So it will first look for the first match. Okay. Say uh something like there are set up except block, so it will check the exception which has got raised. [00:44:54.864] Does it belong to this category? It doesn't, then it goes to next category. And then it goes to next category. Like you have two similar kinds of exceptions which is occurred. In that case, it will take the first exception block which is there. It will not look for the best exception block.

EV0000094 · EV0000095 · EV0000096 · EV0000097

Transcript block 10 (source interval 002700-002999 seconds): [00:45:11.680] It will look for the first matching except not. [00:45:20.292] Okay, so A is 5, B is 6, so print X by B. So here you can see that I have a default message. Here A is 5 and B is 6. So A divided by B integer one. So it's not a type error, it's not a [00:45:37.600] divide by zero error so it will basically go to your default okay [00:45:59.588] So we have set up examples here. [00:46:07.012] With except you can have a else block. Okay. One pride block can have any number of except block. Similarly, with set of except blocks you can also have a else block. Okay. [00:46:20.740] Basically, this is when no error occurs. Earlier, what used to happen? If the error occurs, it goes to next step block. But if error does not occur, you want set of operations to be done, then you can put it under else block. Is that clear? [00:46:35.396] When do we use the else block? If at all you have two things to decide if a error, if an exception occurs, what to do. And if exception does not occur, what to do, then what to do if an exception does not occur, you can put it under else block. [00:46:55.908] This is basically raising our own exceptions. Till now you have seen all built-in exceptions. Say, for example, the example that I gave, Marks is maximum marks is 200. I'm trying to enter marks with greater than 200. [00:47:11.312] That is not input exception, so I have raise my own exception. So syntax is raise and then I'll give exception name. So in this case, I'll give exception name as uh larger than max max and then I can put a message. [00:47:26.980] Okay, so this is how we write a user defined exception. Please take syntax of this. Take syntax of your user defined exception. Abhishek, what are you doing? [00:47:44.612] I don't see. [00:48:02.980] I hope you have taken the normal inbuilt exception handling, right? Try and then except. You can try it any number of except. [00:48:14.212] [ASR low confidence; check audio] And set off except with else, yeah. Can you just remove your mask? [00:48:22.916] [ASR low confidence; check audio] At what time? 11.50. [00:48:28.196] Thank you.

EV0000098 · EV0000099 · EV0000100 · EV0000101

Full transcript: 5195 normalized words, 184 timestamped segments, 2977.792 seconds duration. Low-confidence segments flagged: 9. Review locators: 00:07:13.828, 00:07:18.820, 00:09:29.380, 00:10:46.564, 00:11:08.132, 00:38:27.364, 00:41:51.056, 00:48:14.212, 00:48:22.916

EV0000062 · EV0000063 · EV0000064 · EV0000065

7 Source slide deck: Virtual Environments

This entry preserves the complete slide deck titled Virtual Environments as searchable extracted page text and page-faithful rendered images. Its association with a particular video is not confirmed; it remains a separate topic under Python for computational Problem Solving, Unit_04, to avoid an unsupported match. EV0000186 · EV0000187

Graphs and diagrams

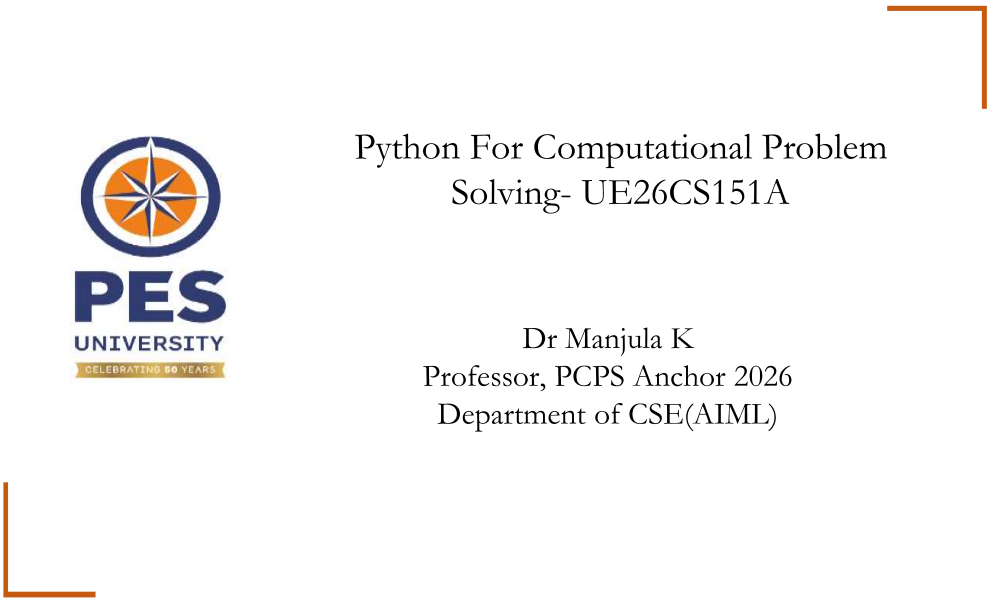


Figure G000029. Virtual Environments · source slide page 1.

EV0000187

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

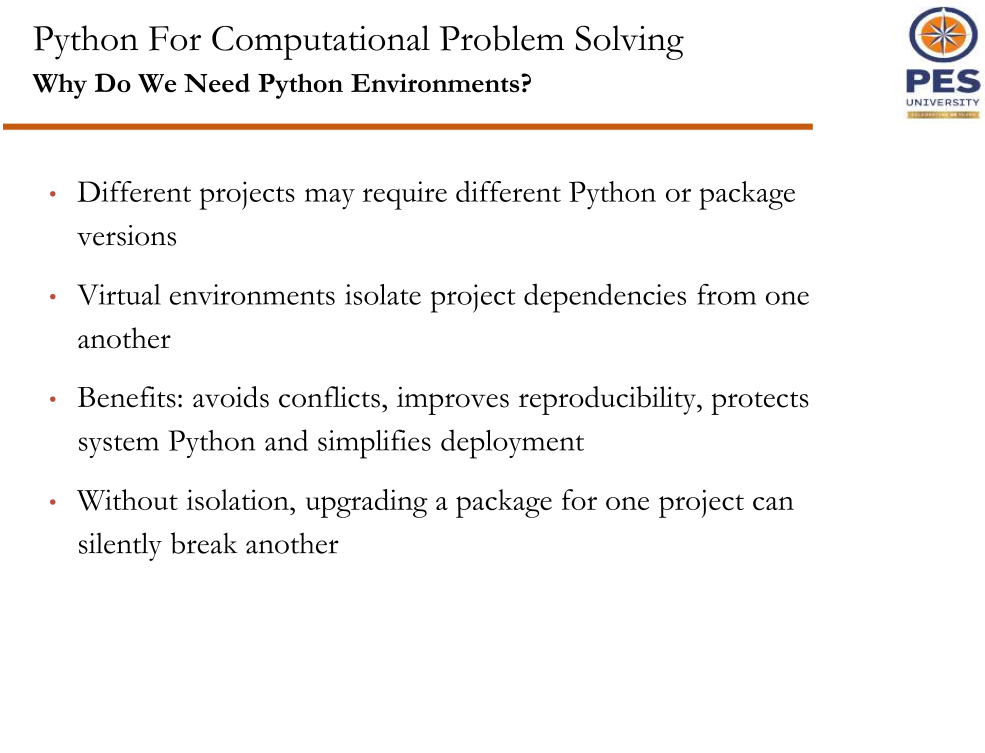


Figure G000030. Virtual Environments · source slide page 2.

EV0000190

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

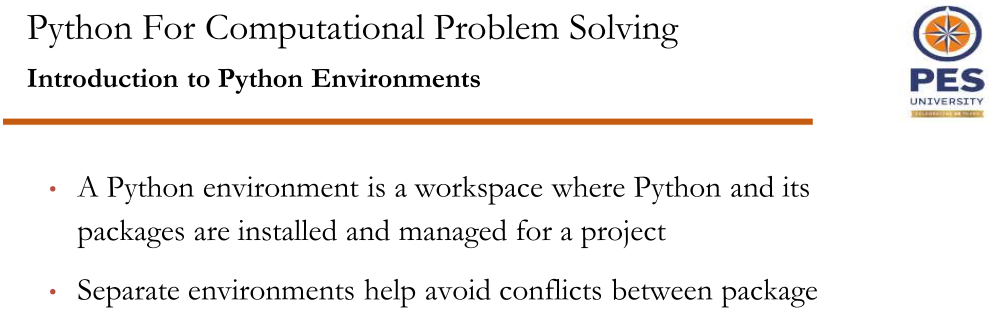


Figure G000031. Virtual Environments · source slide page 3.

EV0000193

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Python For Computational Problem Solving

What is a Virtual Environment?

- An isolated directory containing a Python interpreter and a separate package installation area
- Python provides the built-in venv module for creating virtual environments
- No extra installation needed — venv ships with Python 3.3 and later
- Each environment behaves like its own self-contained Python installation

Figure G000032. Virtual Environments · source slide page 4.

EV0000196

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Figure G000033. Virtual Environments · source slide page 5.

EV0000199

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Figure G000034. Virtual Environments · source slide page 6.

EV0000202

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Python For Computational Problem Solving

Deactivating a Virtual Environment

- Use deactivate when finished working with the environment

```
deactivate
```

- Deactivation does not delete the environment — it simply exits it
- The same environment can be reactivated anytime later

Figure G000035. Virtual Environments · source slide page 7.

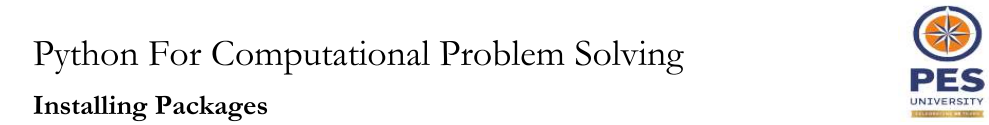
EV0000205

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Figure G000036. Virtual Environments · source slide page 8.

EV0000208

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.



- Install one package or several packages together

```
python -m pip install numpy matplotlib pandas
```

- Packages can be imported into Python right after installation
- Installing a specific version:

```
python -m pip install numpy==1.26.4
```

- Version constraints using >= and <:

```
python -m pip install "numpy>=1.25"  |
"numpy<2.0"
```

Figure G000037. Virtual Environments · source slide page 9.

EV0000211

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

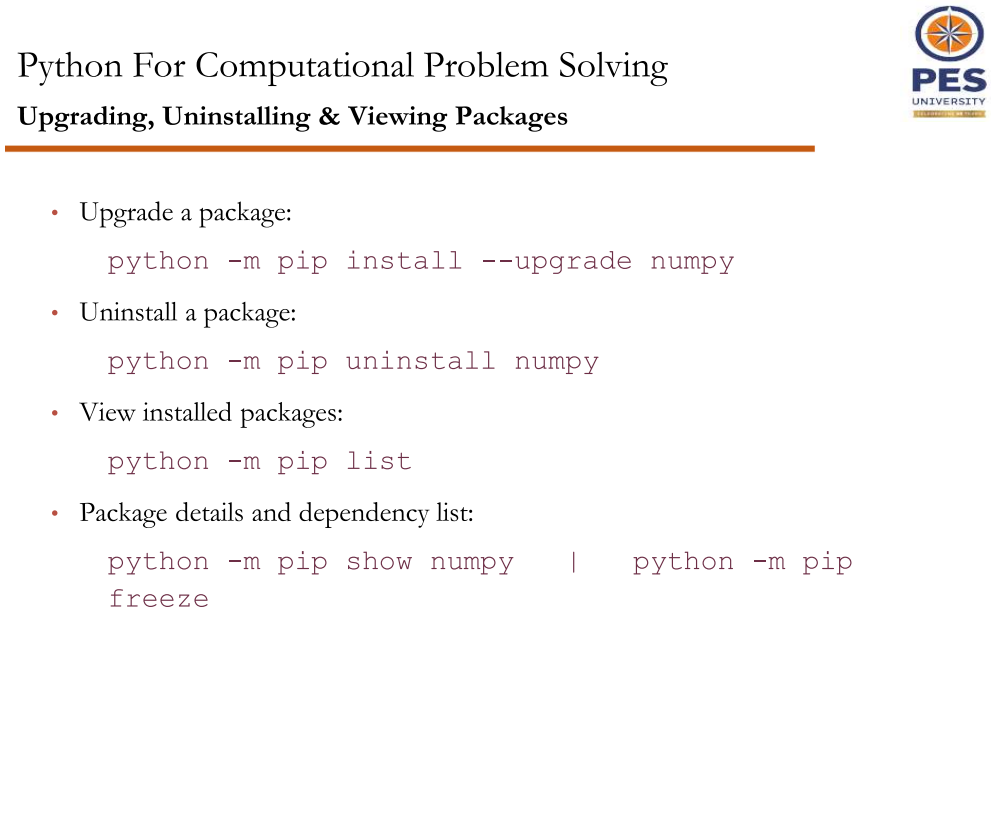


Figure G000038. Virtual Environments · source slide page 10.

EV0000214

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

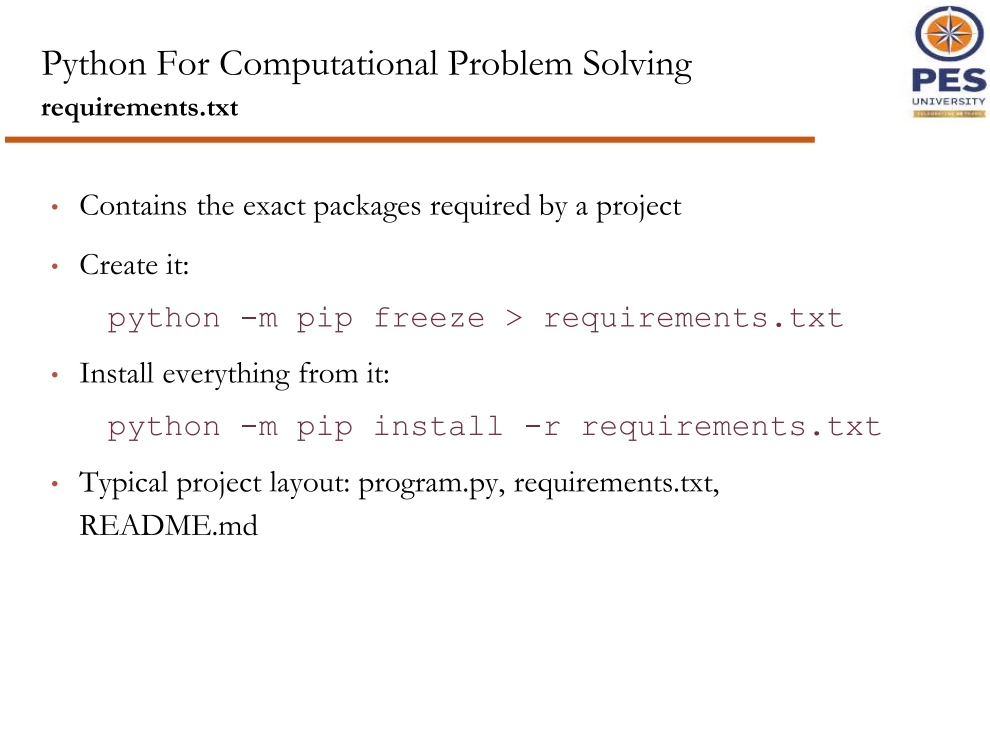


Figure G000039. Virtual Environments · source slide page 11.

EV0000217

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Python For Computational Problem Solving

Sharing a Python Project

- Share the project folder along with requirements.txt — not the environment itself
- Another user creates a fresh virtual environment on their own machine
- They install every dependency from requirements.txt in one command
- Keeps projects lightweight, portable and reproducible across machines

Figure G000040. Virtual Environments · source slide page 12.

EV0000220

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

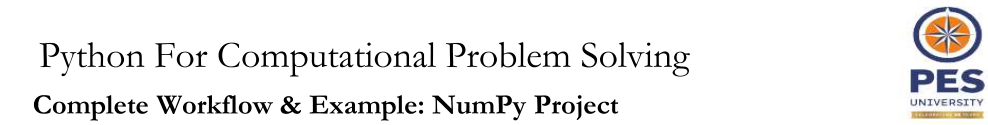


Figure G000041. Virtual Environments · source slide page 13.

EV0000223

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Python For Computational Problem Solving

Why Use python -m pip?

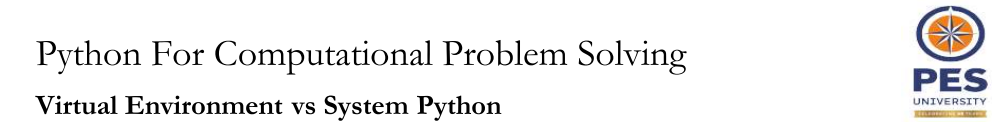
- Runs pip using the specifically selected Python interpreter
- Reduces confusion when multiple Python installations or environments exist
- Guarantees packages install into the intended environment, not a different one
- Recommended form:

```
python -m pip install package_name
```

Figure G000042. Virtual Environments · source slide page 14.

EV0000226

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.



- System Python: packages are shared across many projects, so conflicts are more likely
- Virtual environment: packages are isolated per project, minimizing conflicts
- Changes to system Python can affect the whole machine; venv changes stay contained
- Best practice: always develop inside a project-specific virtual environment

Figure G000043. Virtual Environments · source slide page 15.

EV0000229

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Figure G000044. Virtual Environments · source slide page 16.

EV0000232

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

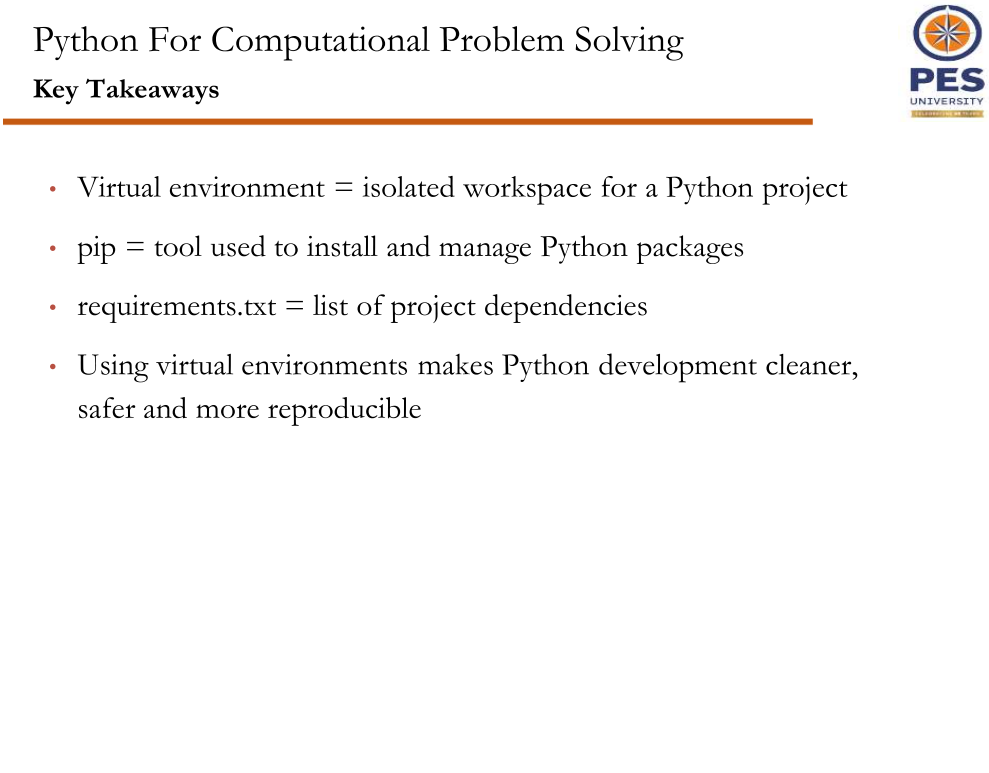


Figure G000045. Virtual Environments · source slide page 17.

EV0000235

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

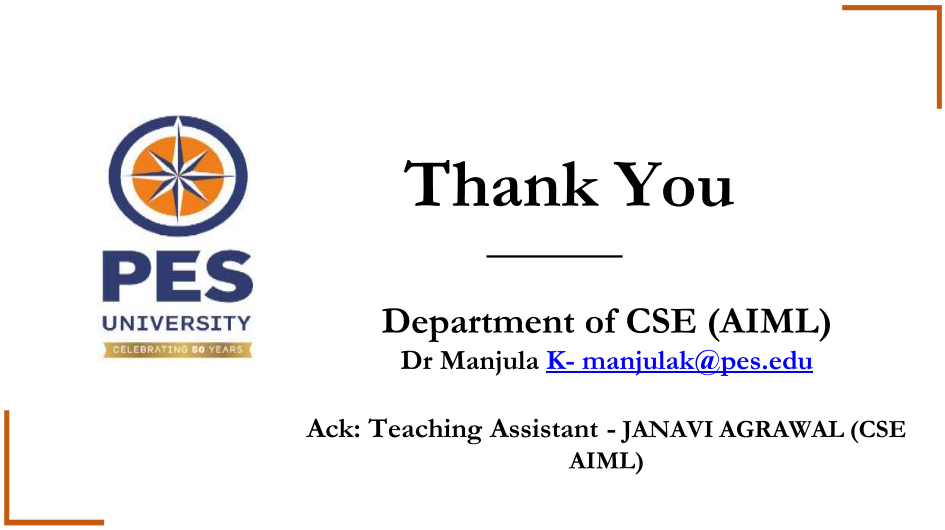


Figure G000046. Virtual Environments · source slide page 18.

EV0000238

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Important points and why

1. Source slide page 1 (extracted text): Python For Computational Problem Solving- UE26CS151A Dr Manjula K Professor, PCPS Anchor 2026 Department of CSE(AIML) EV0000186 · EV0000187 · EV0000188
2. Source slide page 2 (extracted text): Python For Computational Problem Solving Why Do We Need Python Environments? • Different projects may require different Python or package versions • Virtual environments isolate project dependencies from one another • Benefits: avoids conflicts, improves reproducibility, protects system Python and simplifies deployment • Without isolation, upgrading a package for one project can silently break another EV0000189 · EV0000190 · EV0000191
3. Source slide page 3 (extracted text): Python For Computational Problem Solving Introduction to Python Environments • A Python environment is a workspace where Python and its packages are installed and managed for a project • Separate environments help avoid conflicts between package versions and project dependencies • Key topics covered: virtual environments, venv, pip and requirements.txt • Used throughout real-world development to keep projects reproducible and portable EV0000192 · EV0000193 · EV0000194

4. Source slide page 4 (extracted text): Python For Computational Problem Solving What is a Virtual Environment? • An isolated directory containing a Python interpreter and a separate package installation area • Python provides the built-in venv module for creating virtual environments • No extra installation needed — venv ships with Python 3.3 and later • Each environment behaves like its own self-contained Python installation EV0000195 · EV0000196 · EV0000197
5. Source slide page 5 (extracted text): Python For Computational Problem Solving Creating a Virtual Environment • Navigate to the project directory first • Create an environment using venv: python -m venv myenv • This creates a folder named myenv containing the interpreter and package directories • Environment names such as myenv, venv or .venv are common conventions EV0000198 · EV0000199 · EV0000200
6. Source slide page 6 (extracted text): Python For Computational Problem Solving Activating a Virtual Environment • Windows Command Prompt: myenv\Scripts\activate • Windows PowerShell: .\myenv\Scripts\Activate.ps1 • Linux / macOS: source myenv/bin/activate • An active environment is shown by (myenv) in the terminal prompt EV0000201 · EV0000202 · EV0000203
7. Source slide page 7 (extracted text): Python For Computational Problem Solving Deactivating a Virtual Environment • Use deactivate when finished working with the environment deactivate • Deactivation does not delete the environment — it simply exits it • The same environment can be reactivated anytime later EV0000204 · EV0000205 · EV0000206
8. Source slide page 8 (extracted text): Python For Computational Problem Solving What is pip? • pip is Python's package installer • Used to install, upgrade, remove and inspect Python packages python -m pip install numpy • Comes bundled with Python by default • Works both inside a virtual environment and with the system Python EV0000207 · EV0000208 · EV0000209
9. Source slide page 9 (extracted text): Python For Computational Problem Solving Installing Packages • Install one package or several packages together python -m pip install numpy matplotlib pandas • Packages can be imported into Python right after installation • Installing a specific version: python -m pip install numpy==1.26.4 • Version constraints using >= and <: python -m pip install "numpy>=1.25" | "numpy<2.0" EV0000210 · EV0000211 · EV0000212
10. Source slide page 10 (extracted text): Python For Computational Problem Solving Upgrading, Uninstalling & Viewing Packages • Upgrade a package: python -m pip install --upgrade numpy • Uninstall a package: python -m pip uninstall numpy • View installed packages: python -m pip list • Package details and dependency list: python -m pip show numpy | python -m pip freeze EV0000213 · EV0000214 · EV0000215
11. Source slide page 11 (extracted text): Python For Computational Problem Solving requirements.txt • Contains the exact packages required by a project • Create it: python -m pip freeze > requirements.txt • Install everything from it: python -m pip install -r requirements.txt • Typical project layout: program.py, requirements.txt, README.md EV0000216 · EV0000217 · EV0000218
12. Source slide page 12 (extracted text): Python For Computational Problem Solving Sharing a Python Project • Share the project folder along with requirements.txt — not the environment itself • Another user creates a fresh virtual environment on their own machine • They install every dependency from requirements.txt in one command • Keeps projects lightweight, portable and reproducible across machines EV0000219 · EV0000220 · EV0000221

13. Source slide page 13 (extracted text): Python For Computational Problem Solving Complete Workflow & Example: NumPy Project • Create project → create venv → activate → install packages → develop/test → generate requirements.txt → share → recreate environment • Example: setting up a NumPy project mkdir numpy_project cd numpy_project python -m venv venv python -m pip install numpy EV0000222 · EV0000223 · EV0000224
14. Source slide page 14 (extracted text): Python For Computational Problem Solving Why Use python -m pip? • Runs pip using the specifically selected Python interpreter • Reduces confusion when multiple Python installations or environments exist • Guarantees packages install into the intended environment, not a different one • Recommended form: python -m pip install package_name EV0000225 · EV0000226 · EV0000227
15. Source slide page 15 (extracted text): Python For Computational Problem Solving Virtual Environment vs System Python • System Python: packages are shared across many projects, so conflicts are more likely • Virtual environment: packages are isolated per project, minimizing conflicts • Changes to system Python can affect the whole machine; venv changes stay contained • Best practice: always develop inside a project-specific virtual environment EV0000228 · EV0000229 · EV0000230
16. Source slide page 16 (extracted text): Python For Computational Problem Solving Quick Reference – Important Commands • Create: python -m venv myenv • Activate (Windows) / (Linux/macOS): myenv\Scripts\activate | source myenv/bin/activate • Deactivate: deactivate • Install / List: python -m pip install numpy | python -m pip list • Freeze / Install requirements: python -m pip freeze > requirements.txt | python -m pip install -r requirements.txt EV0000231 · EV0000232 · EV0000233
17. Source slide page 17 (extracted text): Python For Computational Problem Solving Key Takeaways • Virtual environment = isolated workspace for a Python project • pip = tool used to install and manage Python packages • requirements.txt = list of project dependencies • Using virtual environments makes Python development cleaner, safer and more reproducible EV0000234 · EV0000235 · EV0000236
18. Source slide page 18 (extracted text): Thank You Department of CSE (AIML) Dr Manjula K-manjulak@pes.edu Ack: Teaching Assistant - JANAVI AGRAWAL (CSE AIML) EV0000237 · EV0000238 · EV0000239

SHORT-NOTE RECAP

Source deck preserved: Virtual Environments. Its extracted page text is listed as searchable points and all rendered source pages are attached as figures. Match to a specific video is score-based; this index does not claim a manual content review.

EV0000186

8 Source slide deck: UE24CS151A PCPS Unit 4 SindhuRPai Class 8 9 OOP

This entry preserves the complete slide deck titled UE24CS151A PCPS Unit 4 SindhuRPai Class 8 9 OOP as searchable extracted page text and page-faithful rendered images. Its association with a particular video is not confirmed; it remains a separate topic under Python for computational Problem Solving, Unit_04, to avoid an unsupported match. EV0000240 · EV0000241

Graphs and diagrams

Figure G000047. UE24CS151A PCPS Unit 4 SindhuRPai Class 8 9 OOP · source slide page 1. EV0000241

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Object oriented Programming

PES UNIVERSITY

Programming Paradigm: Style or way or strategy using which we write the solution

Procedure Oriented Programming (POP)

- Focus is on procedures.
- Procedure is a set of instructions used to accomplish a specific task.
- It can be routines, sub-routines, functions etc.

Examples : C, Pascal, Fortran, Cobol, Algol etc.

2

Figure G000048. UE24CS151A PCPS Unit 4 SindhuRPai Class 8 9 OOP · source slide page 2.

EV0000244

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Object oriented Programming

PES UNIVERSITY

Object Oriented Programming (OOP)

Focus is on the data and the operations that manipulate the data.

Helps in organizing and designing code by representing real-world entities as objects.

OOP is mainly useful to develop big and complex projects carried out by large teams.

Ex: Java, C#, C++, Python etc.

3

Figure G000049. UE24CS151A PCPS Unit 4 SindhuRPai Class 8 9 OOP · source slide page 3.

EV0000247

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Object oriented Programming

PES UNIVERSITY

Features of OOP

Data Abstraction:

- The way you view an object(Ex: employee, library, customers etc)
- Represents the essential features without background details.
- Depending on the application, abstraction has to be implemented.

Data encapsulation:

- Binding of data and procedures as a single unit.
- Encapsulation is a way to implement data abstraction.

4

Figure G000050. UE24CS151A PCPS Unit 4 SindhuRPai Class 8 9 OOP · source slide page 4.

EV0000250

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Object oriented Programming

PES UNIVERSITY

Data Hiding:

- Its about who can access the data .
- Implemented using access specifiers.

Inheritance:

- Capability of one class (child) to derive the capabilities of another class(parent)
- Code reusability is the major benefit of inheritance.

Polymorphism:

- Capability of an object of a class to behave differently in response to the data.

5

Figure G000051. UE24CS151A PCPS Unit 4 SindhuRPai Class 8 9 OOP · source slide page 5.

EV0000253

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Object oriented Programming

PES UNIVERSITY

Key concepts in OOP:

class:

- It is a methodology to create an entity.
- Blueprint or template for creating objects.
- Defines attributes (variables) and methods (functions) that all objects of that class will have.
- The class by itself is a **type and implementation**.
- A class specifies the set of instance variables and methods that are “bundled together”

6

Figure G000052. UE24CS151A PCPS Unit 4 SindhuRPai Class 8 9 OOP · source slide page 6.

EV0000256

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

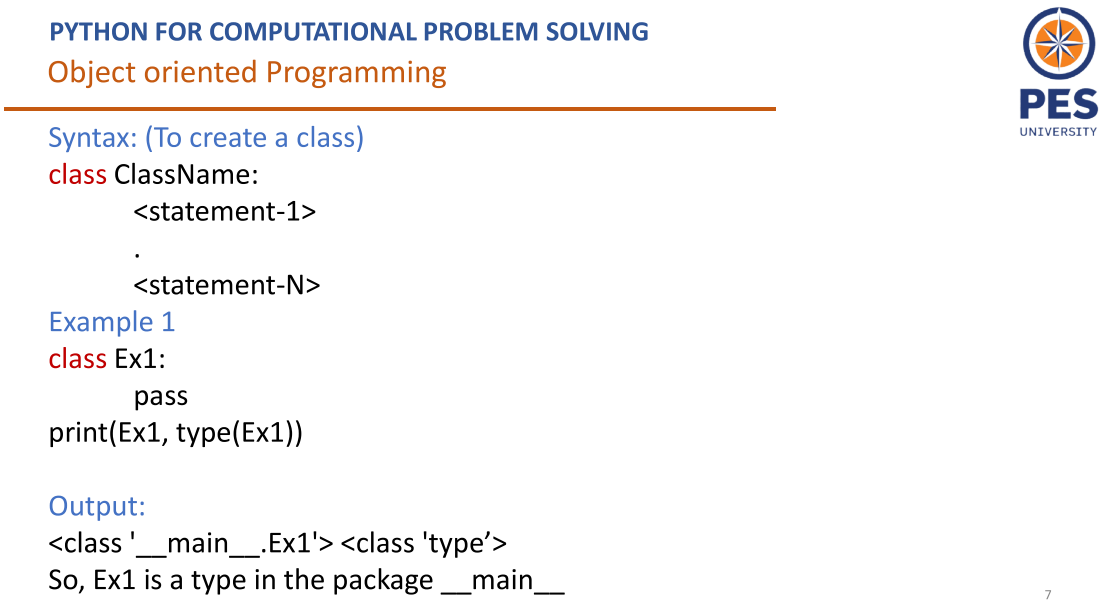


Figure G000053. UE24CS151A PCPS Unit 4 SindhuRPai Class 8 9 OOP · source slide page 7.

EV0000259

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

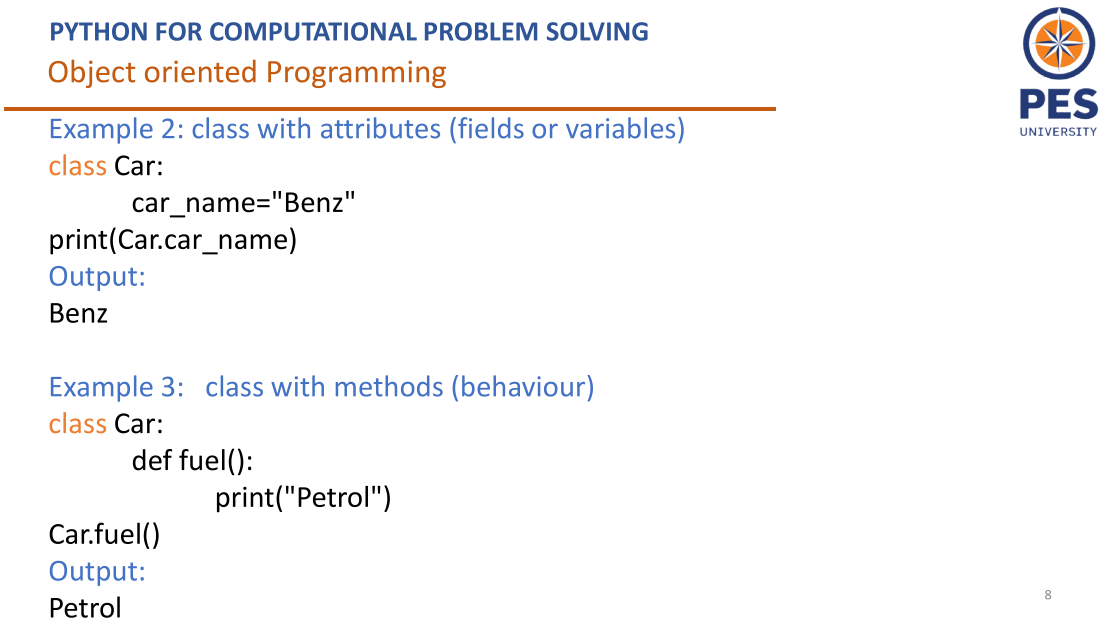


Figure G000054. UE24CS151A PCPS Unit 4 SindhuRPai Class 8 9 OOP · source slide page 8.

EV0000262

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Object oriented Programming

PES UNIVERSITY

Example 4: Class having data and a method.

```
class Car:
        car_name="Benz"
        def display():
                print("This is a Petrol Car")
print(Car.car_name)
Car.display()
```

Output:

```
Benz
This is a Petrol Car
```

9

Figure G000055. UE24CS151A PCPS Unit 4 SindhuRPai Class 8 9 OOP · source slide page 9.

EV0000265

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Object oriented Programming

PES UNIVERSITY

Objects (Instances):

- It is a a physical instance of a class.
- Represents the real-world entities
- Have their own attributes (class variables/instance variables) and methods(class functions), as defined by the class.

10

Figure G000056. UE24CS151A PCPS Unit 4 SindhuRPai Class 8 9 OOP · source slide page 10.

EV0000268

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

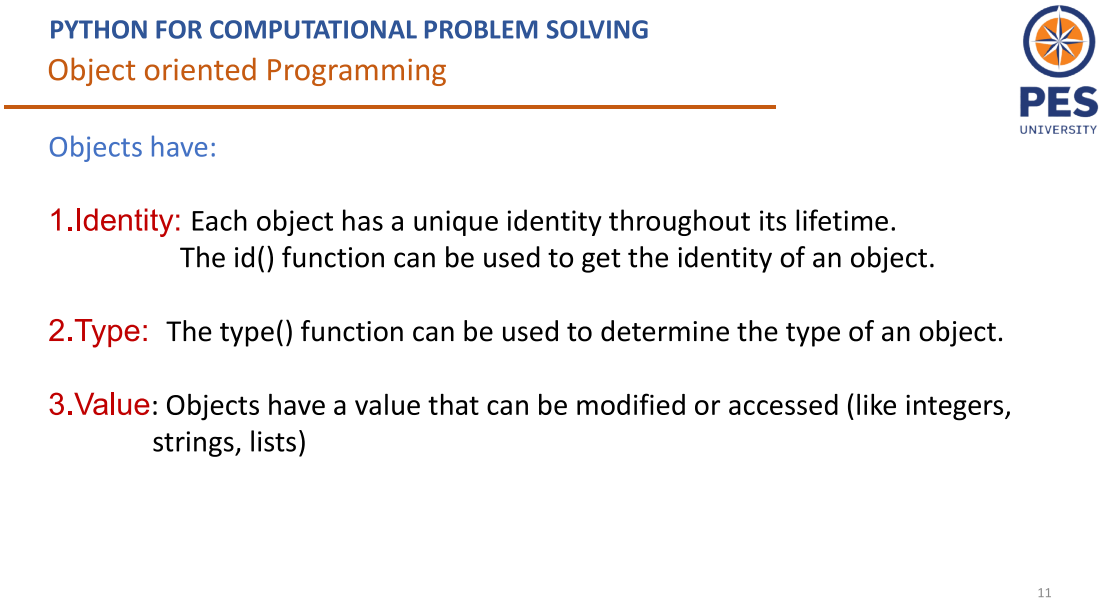


Figure G000057. UE24CS151A PCPS Unit 4 SindhuRPai Class 8 9 OOP · source slide page 11.

EV0000271

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

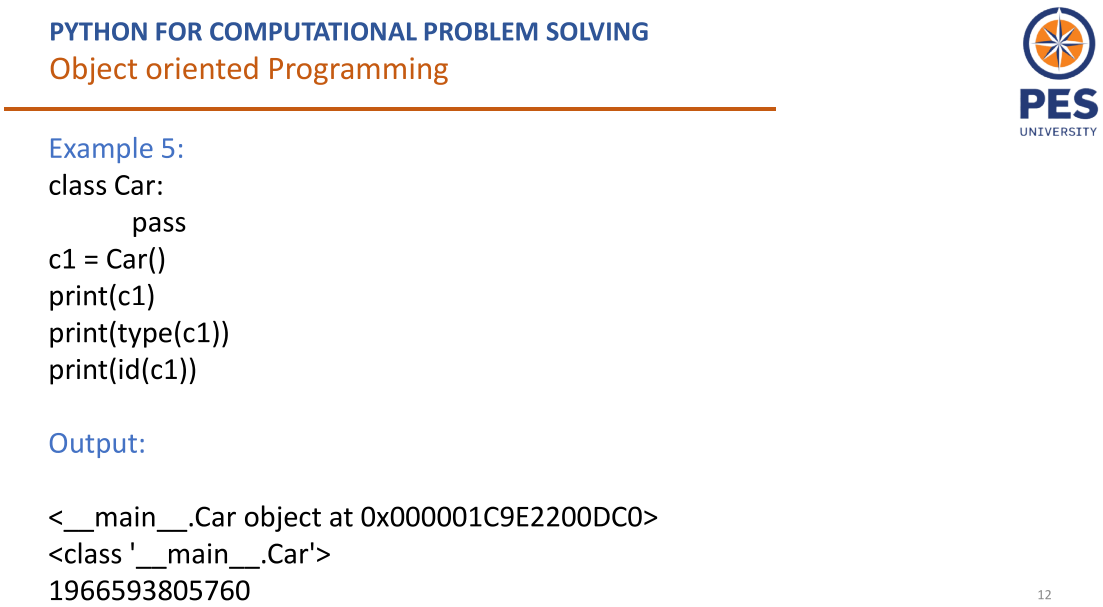


Figure G000058. UE24CS151A PCPS Unit 4 SindhuRPai Class 8 9 OOP · source slide page 12.

EV0000274

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Object oriented Programming

PES UNIVERSITY

Instantiation

- The existence of a class does not create the object.
- Object must be created explicitly
- To create an object of the class, use the function which has the same name as class.

  Ex: c1= Car() will create an object(c1) of class Car
- (.)Dot operator notation can be used to access attributes of the class.

  Ex: c1.car_name

13

Figure G000059. UE24CS151A PCPS Unit 4 SindhuRPai Class 8 9 OOP · source slide page 13.

EV0000277

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Object oriented Programming

PES UNIVERSITY

Constructor

- It is a special function of the class which is called when an object is created
- The name of the this function (constructor) is **__init__**
- It is invoked automatically and implicitly when the object of the class is created.
- The constructor can be used to initialize the internal state of an object, and create instance variables

14

Figure G000060. UE24CS151A PCPS Unit 4 SindhuRPai Class 8 9 OOP · source slide page 14.

EV0000280

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Object oriented Programming

Destructor

- It is a special function within the class by name __del__, that performs the clean-up actions when an object is destroyed or deleted.
- The destructor is automatically called just before an object is removed from memory by the garbage collector.
- It is often used to release resources before an object is removed from the system.

15

Figure G000061. UE24CS151A PCPS Unit 4 SindhuRPai Class 8 9 OOP · source slide page 15.

EV0000283

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

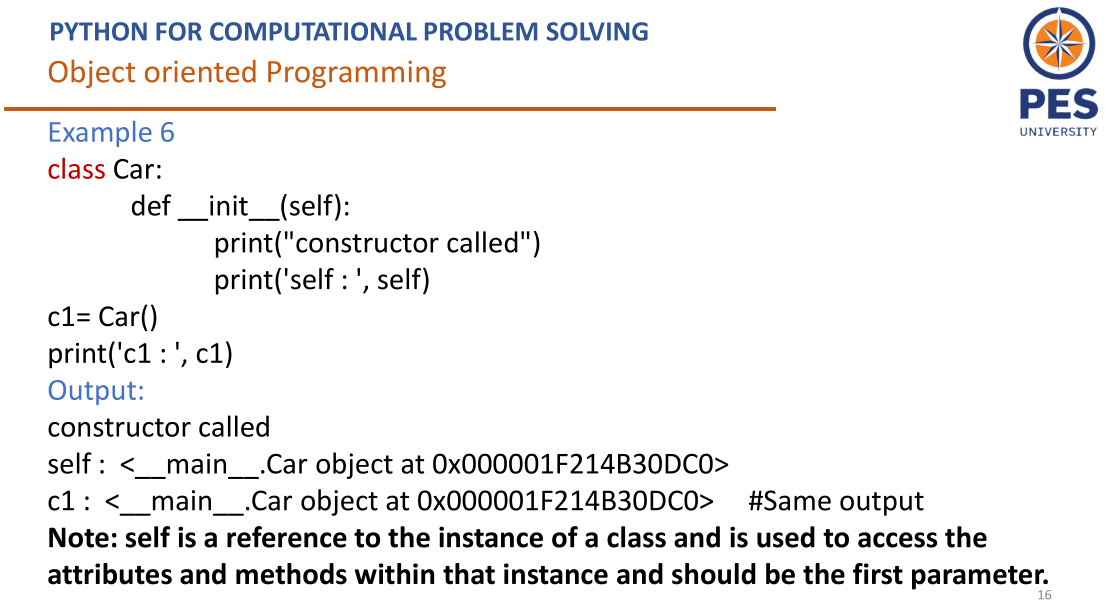


Figure G000062. UE24CS151A PCPS Unit 4 SindhuRPai Class 8 9 OOP · source slide page 16.

EV0000286

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

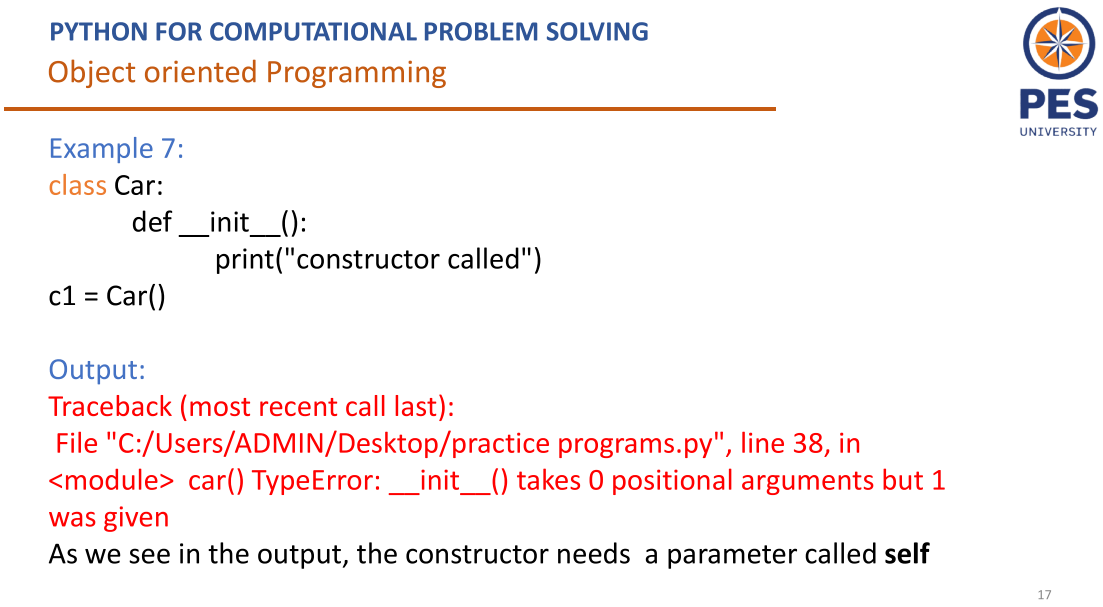


Figure G000063. UE24CS151A PCPS Unit 4 SindhuRPai Class 8 9 OOP · source slide page 17.

EV0000289

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

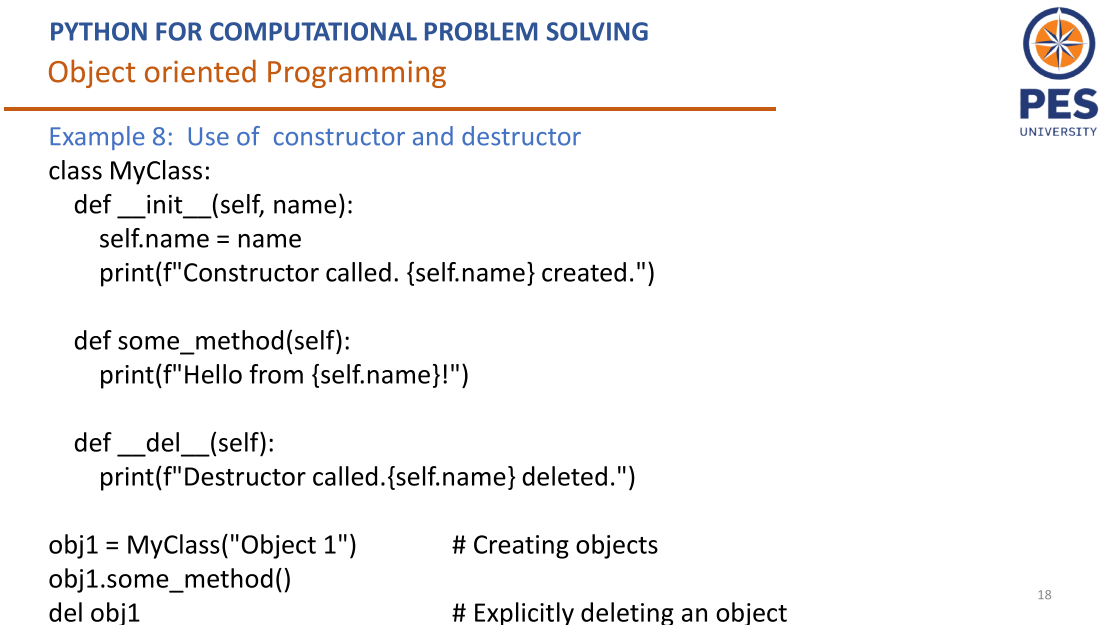


Figure G000064. UE24CS151A PCPS Unit 4 SindhuRPai Class 8 9 OOP · source slide page 18.

EV0000292

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Object oriented Programming

PES UNIVERSITY

Output

```
Constructor called. Object 1 created.
Hello from Object 1!
Destructor called. Object 1 deleted.
```

Note: The destructor is called when the object obj1 is explicitly deleted using “del obj1”

The module garbage collector automatically manages memory and calls destructors when objects are not needed.

19

Figure G000065. UE24CS151A PCPS Unit 4 SindhuRPai Class 8 9 OOP · source slide page 19.

EV0000295

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Object oriented Programming

PES UNIVERSITY

Types of Constructors :

1. Parameterized constructor
2. Non Parameterized constructor

Example 9: Non-parameterized constructors

```
#when no parameters are passed for invoking constructor
class Person:
        def __init__(self):
                print("Constructor without parameters")
p = Person()
```

Output:

```
Constructor without parameters
```

20

Figure G000066. UE24CS151A PCPS Unit 4 SindhuRPai Class 8 9 OOP · source slide page 20.

EV0000298

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

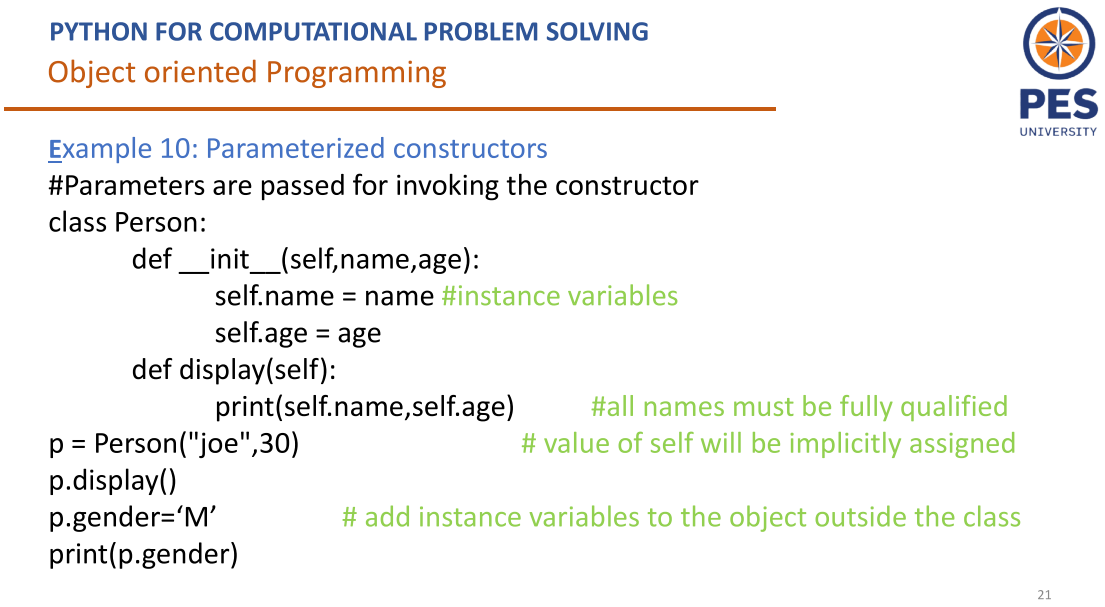


Figure G000067. UE24CS151A PCPS Unit 4 SindhuRPai Class 8 9 OOP · source slide page 21.

EV0000301

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Object oriented Programming(Practice Program)

PES UNIVERSITY

Example 11:Demonstration class & object with constructor and destructor.

```
class Rectangle:
    def __init__(self, length, width):
        self.length = length
        self.width = width

    def display(self):
        print(f"Rectangle dimensions: {self.length} x {self.width}")

    def __del__(self):
        print("Rectangle object destroyed.")

# Creating a Rectangle object
rect = Rectangle(5, 10)
 rect.display()
```

22

Figure G000068. UE24CS151A PCPS Unit 4 SindhuRPai Class 8 9 OOP · source slide page 22.

EV0000304

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Object oriented Programming

PES UNIVERSITY

Getter and Setter Method:

Getter:

- Used to retrieve the value of attribute of a class without directly exposing it.

Setter:

- Used to modify the value of attribute of a class.
- Allows controlled modification of the attribute's value by performing checks or validations before assigning the new value.

23

Figure G000069. UE24CS151A PCPS Unit 4 SindhuRPai Class 8 9 OOP · source slide page 23.

EV0000307

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Object oriented Programming

PES UNIVERSITY

Ex: Getter and setter using user defined functions

```
class getset:
   def __init__(self, name):
      self.name = name
   def get_name(self):
      return self.name
   def set_name(self,new_name):
      self.name=new_name

 obj = getset("Arun")
 print("Name:: before calling setter",obj.get_name())          # Output: Arun
obj.set_name("Ram")
print("Name:: after calling setter",obj.get_name())           # Output: Ram
```

24

Figure G000070. UE24CS151A PCPS Unit 4 SindhuRPai Class 8 9 OOP · source slide page 24.

EV0000310

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

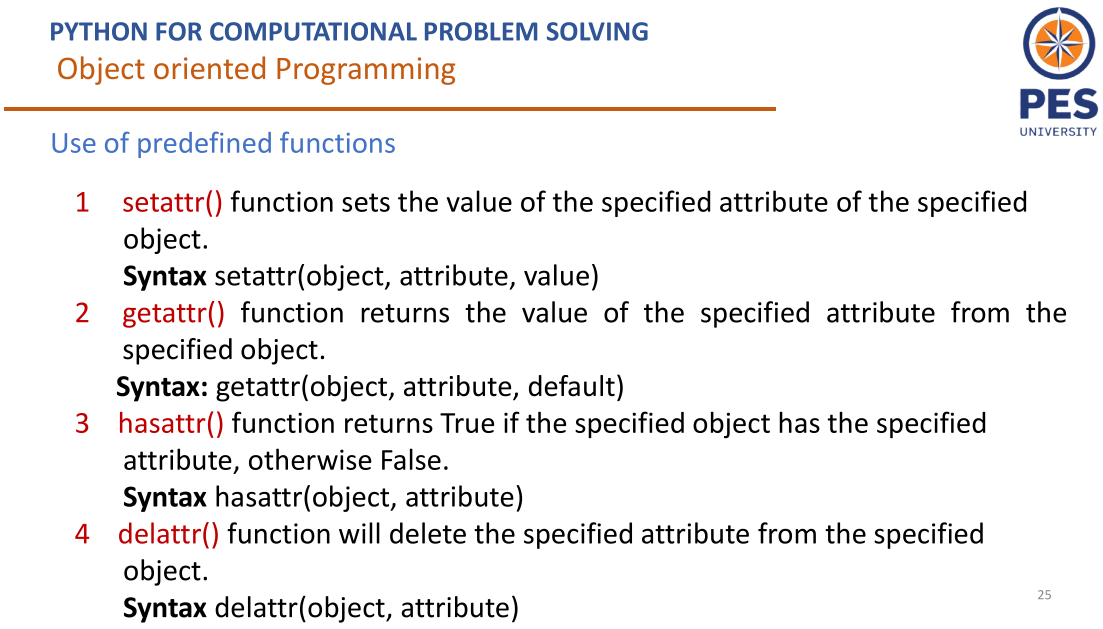


Figure G000071. UE24CS151A PCPS Unit 4 SindhuRPai Class 8 9 OOP · source slide page 25.

EV0000313

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

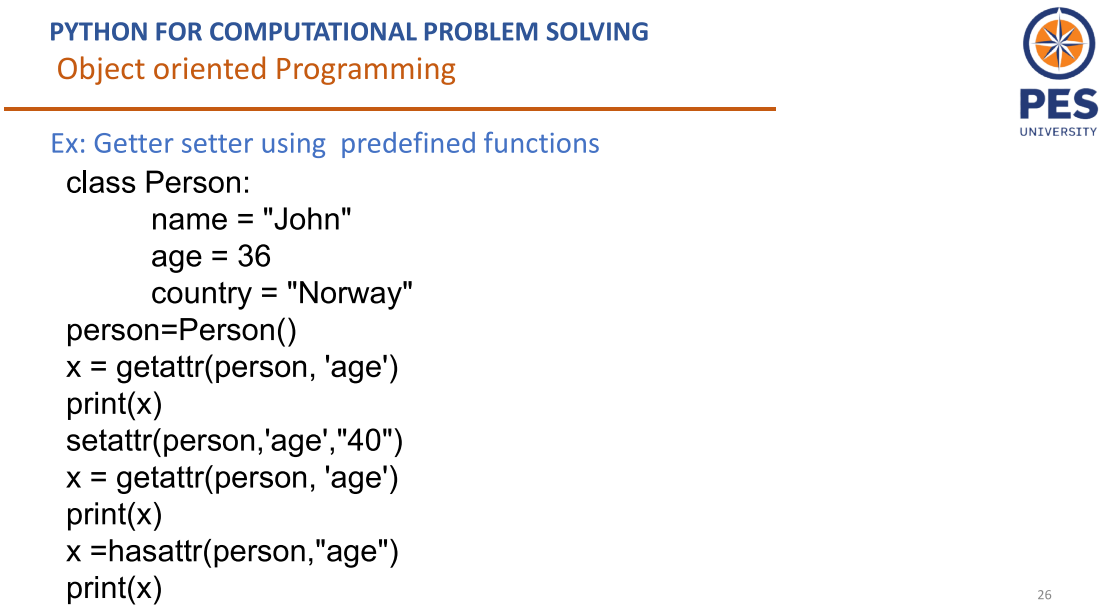


Figure G000072. UE24CS151A PCPS Unit 4 SindhuRPai Class 8 9 OOP · source slide page 26.

EV0000316

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Object oriented Programming

PES UNIVERSITY

Practice programs

1.Define a class called “candidate” which has the Registration number and Score as the data members.
Write a function Input() which allows the user to enter values for the above data members.
Write a function called Selection() which assigns remarks as per the score obtained by the candidate as follows.
If score>=60 then assign remarks=“Selected” else remarks= “Not selected”
Write a display() function to view the data members.

Test this created class for all its functionality by creating objects

27

Figure G000073. UE24CS151A PCPS Unit 4 SindhuRPai Class 8 9 OOP · source slide page 27.

EV0000319

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Object oriented Programming

PES UNIVERSITY

2. Define a class called Travel with the following descriptions:
Class members: Travel _Code, Place, Number _of _travelers, Number _of _buses

Define a constructor which initializes the above class members with the values 105, “Bombay”,15, and 5 respectively.
Take the input() from the user for all the data members.
Also assign the number of buses equal to 1 if number of travelers is greater or equal to 10 otherwise no bus is assigned.

Test this created class for all its functionality by creating objects

28

Figure G000074. UE24CS151A PCPS Unit 4 SindhuRPai Class 8 9 OOP · source slide page 28.

EV0000322

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Object oriented Programming

PES UNIVERSITY

3. Define a class called " Stock" with following data members
item _code, item _name, price, quantity , Discount.

Define a member function CalcDisc() to calculate the discount as per the following:
If quantity<=100 then discount=0
If quantity <=150 then discount is 5%
If quantity >150 then discount is 10%
Write a function Enter_Details() which allows the user to enter values to the data members and call CalcDisc() to calculate the discount.
Write a function Display() to view the contents of all the data members.

Test this created class for all its functionality by creating objects

29

Figure G000075. UE24CS151A PCPS Unit 4 SindhuRPai Class 8 9 OOP · source slide page 29.

EV0000325

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Object oriented Programming

PES UNIVERSITY

4.Define a class to represent the bank account of a customer with the information like Name of the depositor, Account number and type of the account(Savings, Current) and Balance amount.

Define separate methods for the following:

1. To initialize the data member
2. To Deposit the amount
3. To withdraw the amount after checking the balance amount
4. To display the data members

30

Figure G000076. UE24CS151A PCPS Unit 4 SindhuRPai Class 8 9 OOP · source slide page 30.

EV0000328

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

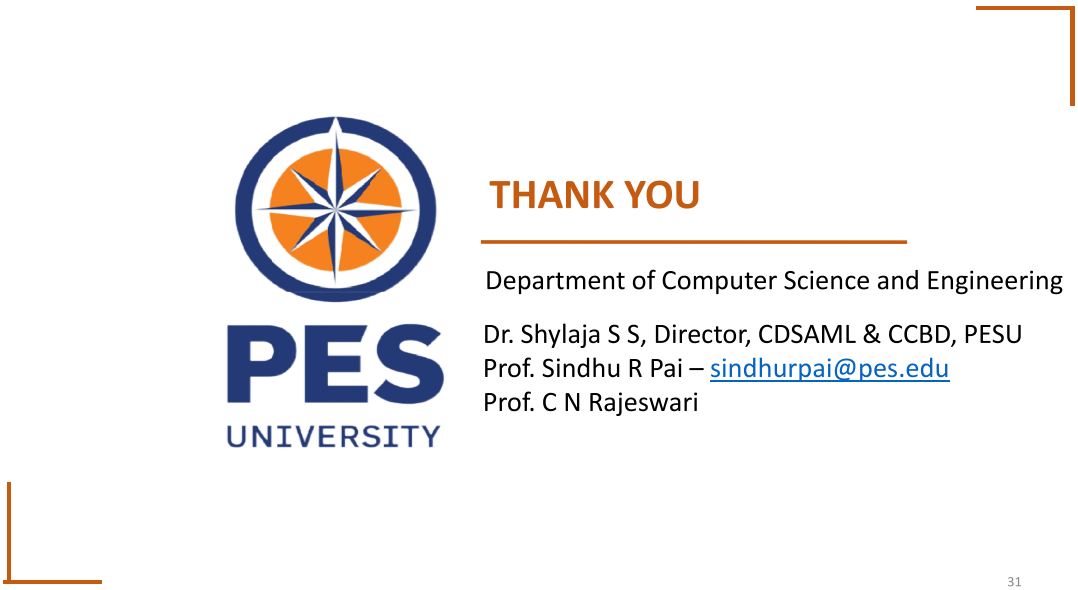


Figure G000077. UE24CS151A PCPS Unit 4 SindhuRPai Class 8 9 OOP · source slide page 31.

EV0000331

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Important points and why

1. Source slide page 1 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Object Oriented Programming Prof. Sindhu R Pai PCPS Theory Anchor - 2024 Department of Computer Science and Engineering EV0000240 · EV0000241 · EV0000242
2. Source slide page 2 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Object oriented Programming Programming Paradigm: Style or way or strategy using which we write the solution Procedure Oriented Programming (POP) • Focus is on procedures. • Procedure is a set of instructions used to accomplish a specific task. • It can be routines, sub-routines, functions etc. Examples : C, Pascal, Fortran, Cobol, Algol etc. 2 EV0000243 · EV0000244 · EV0000245
3. Source slide page 3 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Object oriented Programming Object Oriented Programming (OOP) Focus is on the data and the operations that manipulate the data. HelpsHelps inin organizingorganizing andand designingdesigning codecode byby representingrepresenting real-worldreal world entitiesentities asas objects. OOP is mainly useful to develop big and complex projects carried out by large teams. Ex: Java, C#, C++, Python etc. 3 EV0000246 · EV0000247 · EV0000248

4. Source slide page 4 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Object oriented Programming Features of OOP Data Abstraction: • The way you view an object(Ex: employee, library, customers etc) • Represents the essential features without background details. • Depending on the application, abstraction has to be implemented. Data encapsulation: • Binding of data and procedures as a single unit. • Encapsulation is a way to implement data abstraction. 4 EV0000249 · EV0000250 · EV0000251

5. Source slide page 5 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Object oriented Programming Data Hiding: • Its about who can access the data . • Implemented using access specifiers. Inheritance: • Capability of one class (child) to derive the capabilities of another class(parent) • Code reusability is the major benefit of inheritance. Polymorphism: • Capability of an object of a class to behave differently in response to the data. 5 EV0000252 · EV0000253 · EV0000254

6. Source slide page 6 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Object oriented Programming Key concepts in OOP: class: • It is a methodology to create an entity. • Blueprintl i or templatel ffor creatingi objects.bj • Defines attributes (variables) and methods (functions) that all objects of that class will have. • The class by itself is a type and implementation. • A class specifies the set of instance variables and methods that are “bundled 6 together” EV0000255 · EV0000256 · EV0000257

7. Source slide page 7 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Object oriented Programming Syntax: (To create a class) class ClassName: <statement-1> . <statement-N> Example 1 class Ex1: pass print(Ex1, type(Ex1)) Output: <class '__main__.Ex1'> <class 'type’> So, Ex1 is a type in the package __main__ 7 EV0000258 · EV0000259 · EV0000260

8. Source slide page 8 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Object oriented Programming Example 2: class with attributes (fields or variables) class Car: car_name="Benz" print(Car.car_name) Output: Benz Example 3: class with methods (behaviour) class Car: def fuel(): print("Petrol") Car.fuel() Output: Petrol 8 EV0000261 · EV0000262 · EV0000263

9. Source slide page 9 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Object oriented Programming Example 4: Class having data and a method. class Car: car_name="Benz" def display(): print("This is a Petrol Car") print(Car.car_name) Car.display() Output: Benz This is a Petrol Car 9 EV0000264 · EV0000265 · EV0000266

10. Source slide page 10 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Object oriented Programming Objects (Instances): • It is a a physical instance of a class. • Represents the real-world entities • Have theirh own attributesb ((classl variables/instancebl / variables)bl) andd methods(class functions), as defined by the class. 10 EV0000267 · EV0000268 · EV0000269

11. Source slide page 11 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Object oriented Programming Objects have: 1.Identity: Each object has a unique identity throughout its lifetime. The id() function can be used to get the identity of an object. 2.Type: The type() function can be used to determine the type of an object. 3.Value: Objects have a value that can be modified or accessed (like integers, strings, lists) 11 EV0000270 · EV0000271 · EV0000272

12. Source slide page 12 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Object oriented Programming Example 5: class Car: pass c1 = Car() print(c1)print(c) print(type(c1)) print(id(c1)) Output: <__main__.Car object at 0x000001C9E2200DC0> <class '__main__.Car'> 121966593805760 EV0000273 · EV0000274 · EV0000275
13. Source slide page 13 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Object oriented Programming Instantiation • The existence of a class does not create the object. • Object must be created explicitly •• ToTo createcreate anan objectobject ofof thethe classclass, useuse thethe functionfunction whichwhich hashas thethe samesame namename asas classclass. Ex: c1= Car() will create an object(c1) of class Car • (.)Dot operator notation can be used to access attributes of the class. Ex: c1.car_name 13 EV0000276 · EV0000277 · EV0000278
14. Source slide page 14 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Object oriented Programming Constructor • It is a special function of the class which is called when an object is created • The name of the this function (constructor) is __init__ • It is invoked automatically and implicitly when the object of the class is created. • The constructor can be used to initialize the internal state of an object, and create instance variables 14 EV0000279 · EV0000280 · EV0000281
15. Source slide page 15 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Object oriented Programming Destructor • It is a special function within the class by name __del__, that performs the clean- up actions when an object is destroyed or deleted. • The destructor is automatically called just before an object is removed from memory by the garbage collector. • It is often used to release resources before an object is removed from the system. 15 EV0000282 · EV0000283 · EV0000284
16. Source slide page 16 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Object oriented Programming Example 6 class Car: def __init__(self): print("constructor called") print('self : ', self) c1= Car()() print('c1 : ', c1) Output: constructor called self : <__main__.Car object at 0x000001F214B30DC0> c1 : <__main__.Car object at 0x000001F214B30DC0> #Same output Note: self is a reference to the instance of a class and is used to access the attributes and methods within that instance and should be the first parameter. 16 EV0000285 · EV0000286 · EV0000287
17. Source slide page 17 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Object oriented Programming Example 7: class Car: def __init__(): print("constructor called") c1c = Car()Car() Output: Traceback (most recent call last): File "C:/Users/ADMIN/Desktop/practice programs.py", line 38, in <module> car() TypeError: __init__() takes 0 positional arguments but 1 was given As we see in the output, the constructor needs a parameter called self 17 EV0000288 · EV0000289 · EV0000290
18. Source slide page 18 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Object oriented Programming Example 8: Use of constructor and destructor class MyClass: def __init__(self, name): self.name = name print(f"Constructor called. {self.name} created.") def some_method(self): print(f"Hello from {self.name}!") def __del__(self): print(f"Destructor called. {self.name} deleted.") obj1 = MyClass("Object 1") # Creating objects obj1.some_method() 18 del obj1 # Explicitly deleting an object EV0000291 · EV0000292 · EV0000293

19. Source slide page 19 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Object oriented Programming Output Constructor called. Object 1 created. Hello from Object 1! Destructor called. Object 1 deleted. Note: The destructor is called when the object obj1 is explicitly deleted using “del obj1” The module garbage collector automatically manages memory and calls destructors when objects are not needed. 19 EV0000294 · EV0000295 · EV0000296

20. Source slide page 20 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Object oriented Programming Types of Constructors : 1. Parameterized constructor 2. Non Parameterized constructor ExampleExample 9:9: Non-parameterizedNon parameterized constructorsconstructors #when no parameters are passed for invoking constructor class Person: def __init__(self): print("Constructor without parameters") p = Person() Output: 20Constructor without parameters EV0000297 · EV0000298 · EV0000299

21. Source slide page 21 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Object oriented Programming Example 10: Parameterized constructors #Parameters are passed for invoking the constructor class Person: def __init__(self,name,age): self.nameself.name = namename #instance#instance variablesvariables self.age = age def display(self): print(self.name,self.age) #all names must be fully qualified p = Person("joe",30) # value of self will be implicitly assigned p.display() p.gender=‘M’ # add instance variables to the object outside the class print(p.gender) 21 EV0000300 · EV0000301 · EV0000302

22. Source slide page 22 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Object oriented Programming(Practice Program) Example 11:Demonstration class & object with constructor and destructor. class Rectangle: def __init__(self, length, width): self.length = length self.width = width def display(self): print(f"Rectangle dimensions: {self.length} x {self.width}") def __del__(self): print("Rectangle object destroyed.") # Creating a Rectangle object rect = Rectangle(5, 10) 22 rect.display() EV0000303 · EV0000304 · EV0000305

23. Source slide page 23 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Object oriented Programming Getter and Setter Method: Getter: • Used to retrieve the value of attribute of a class without directly exposing it. Setter: • Used to modify the value of attribute of a class. • Allows controlled modification of the attribute's value by performing checks or validations before assigning the new value. 23 EV0000306 · EV0000307 · EV0000308

24. Source slide page 24 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Object oriented Programming Ex: Getter and setter using user defined functions class getset: def __init__(self, name): self.name = name def get_name(self): returnt self.namelf def set_name(self,new_name): self.name=new_name obj = getset("Arun") print("Name:: before calling setter",obj.get_name()) # Output: Arun obj.set_name("Ram") print("Name:: after calling setter",obj.get_name()) # Output: Ram 24 EV0000309 · EV0000310 · EV0000311

25. Source slide page 25 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Object oriented Programming Use of predefined functions 1 setattr() function sets the value of the specified attribute of the specified object. Syntax setattr(object, attribute, value) 22 getattr()t tt () ffunctionti returnst ththe valuel off ththe specifiedifi d attributett ib t ffrom ththe specified object. Syntax: getattr(object, attribute, default) 3 hasattr() function returns True if the specified object has the specified attribute, otherwise False. Syntax hasattr(object, attribute) 4 delattr() function will delete the specified attribute from the specified object. Syntax delattr(object, attribute) 25 EV0000312 · EV0000313 · EV0000314

26. Source slide page 26 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Object oriented Programming Ex: Getter setter using predefined functions class Person: name = "John" age = 36 countryy = "Norway"y person=Person() x = getattr(person, 'age') print(x) setattr(person,'age',"40") x = getattr(person, 'age') print(x) x =hasattr(person,"age") print(x) 26 EV0000315 · EV0000316 · EV0000317

27. Source slide page 27 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Object oriented Programming Practice programs 1.Define a class called “candidate” which has the Registration number and Score as the data members. WriteWrite aa functionfunction Input()Input() whichwhich allowsallows thethe useruser toto enterenter valuesvalues forfor thethe aboveabove data members. Write a function called Selection() which assigns remarks as per the score obtained by the candidate as follows. If score>=60 then assign remarks=“Selected” else remarks= “Not selected” Write a display() function to view the data members. Test this created class for all its functionality by creating objects 27 EV0000318 · EV0000319 · EV0000320

28. Source slide page 28 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Object oriented Programming 2. Define a class called Travel with the following descriptions: Class members: Travel _Code, Place, Number _of _travelers, Number _of _buses Define a constructor which initializes the above class members with the values 105, “Bombay”Bombay ,15,15 andand 55 respectivelyrespectively. Take the input() from the user for all the data members. Also assign the number of buses equal to 1 if number of travelers is greater or equal to 10 otherwise no bus is assigned. Test this created class for all its functionality by creating objects 28 EV0000321 · EV0000322 · EV0000323

29. Source slide page 29 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Object oriented Programming 3. Define a class called “ Stock” with following data members item _code, item _name, price, quantity , Discount. Define a member function CalcDisc() to calculate the discount as per the following: If quantity<=100 then discount=0 If quantity <=150 then discount is 5% If quantity >150 then discount is 10% Write a function Enter_Details() which allows the user to enter values to the data members and call CalcDisc() to calculate the discount. Write a function Display() to view the contents of all the data members. Test this created class for all its functionality by creating objects 29 EV0000324 · EV0000325 · EV0000326

30. Source slide page 30 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Object oriented Programming 4.Define a class to represent the bank account of a customer with the information like Name of the depositor, Account number and type of the account(Savings, Current) and Balance amount. DefineDefine separateseparate methodsmethods forfor thethe following:following: 1. To initialize the data member 2. To Deposit the amount 3. To withdraw the amount after checking the balance amount 4. To display the data members 30 EV0000327 · EV0000328 · EV0000329

31. Source slide page 31 (extracted text): THANK YOU Departmentp of Computerp Science and Engineeringg g Dr. Shylaja S S, Director, CDSAML & CCBD, PESU Prof. Sindhu R Pai – sindhurpai@pes.edu Prof. C N Rajeswari 31 EV0000330 · EV0000331 · EV0000332

SHORT-NOTE RECAP

Source deck preserved: UE24CS151A PCPS Unit 4 SindhuRPai Class 8 9 OOP. Its extracted page text is listed as searchable points and all rendered source pages are attached as figures. Match to a specific video is score-based; this index does not claim a manual content review.

EV0000240

9 Source slide deck: UE24CS151A PCPS Unit 4 SindhuRPai Class 11 Polymorphism

This entry preserves the complete slide deck titled UE24CS151A PCPS Unit 4 SindhuRPai Class 11 Polymorphism as searchable extracted page text and page-faithful rendered images. Its association with a particular video is not confirmed; it remains a separate topic under Python for computational Problem Solving, Unit_04, to avoid an unsupported match. EV0000411 · EV0000412

Graphs and diagrams

Figure G000104. UE24CS151A PCPS Unit 4 SindhuRPai Class 11 Polymorphism · source slide page 1.

EV0000412

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Polymorphism

PES UNIVERSITY

Introduction

- Polymorphism refers to having multiple forms.
- refers to the use of the same function name, but with different signatures
- allows objects of different classes to be treated as objects of a common superclass.
- This concept enables a single interface to be used for entities of different types.

2

Figure G000105. UE24CS151A PCPS Unit 4 SindhuRPai Class 11 Polymorphism · source slide page 2.

EV0000415

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Polymorphism

PES UNIVERSITY

Runtime Polymorphism

- Python supports runtime polymorphism by using techniques like method overloading and method overriding
- Runtime polymorphism is the ability of an object to behave differently based on its actual type during program execution
- It is also known as dynamic polymorphism
- It enables the same method name to behave differently based on the specific class instance at runtime.

3

Figure G000106. UE24CS151A PCPS Unit 4 SindhuRPai Class 11 Polymorphism · source slide page 3.

EV0000418

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

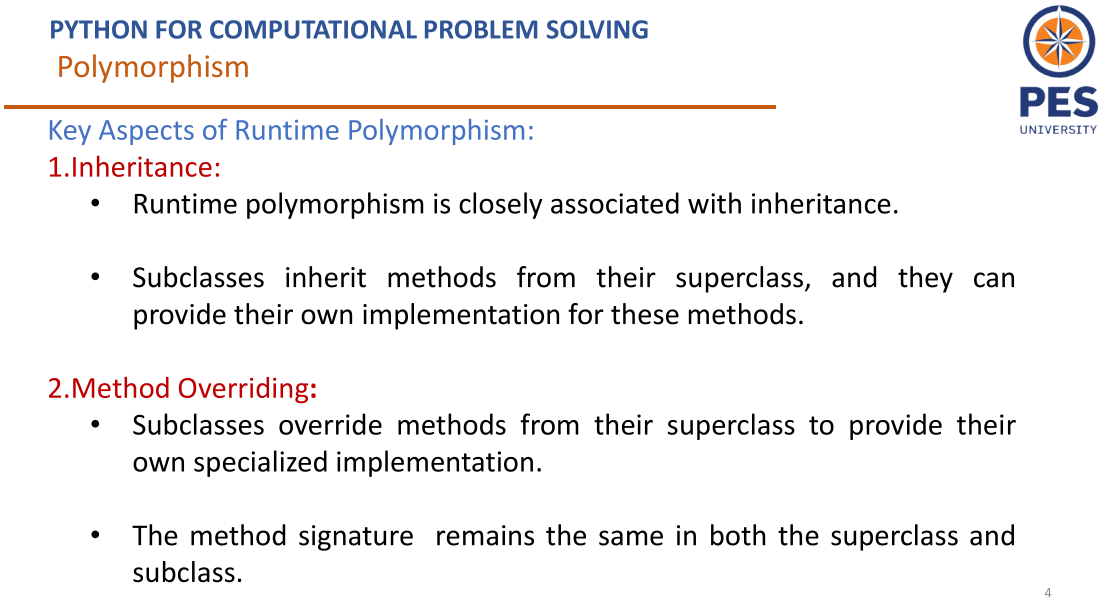


Figure G000107. UE24CS151A PCPS Unit 4 SindhuRPai Class 11 Polymorphism · source slide page 4.

EV0000421

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Polymorphism

PES UNIVERSITY

3.Dynamic Binding:

- During runtime, the appropriate method to execute is dynamically determined
- It is based on the actual type of the object invoking the method

4.Common Interface:

- Different subclasses sharing a common superclass interface
- Exhibits different behaviors based on their specific implementations.

5

Figure G000108. UE24CS151A PCPS Unit 4 SindhuRPai Class 11 Polymorphism · source slide page 5.

EV0000424

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Polymorphism

The following example (Refer next slide) demonstrates all these key features of runtime polymorphism.

In this example ‘Rectangle ‘ and ‘circle’ subclasses override the ‘calculate_area’ method from the ‘shape’ superclass to provide their specific area calculation.

The method ‘calculate_area’ is called on objects of different types(‘rect’ or ‘circle’) . The appropriate overridden method is executed based on the object’s actual type.

This happens at the runtime and method execution is dynamically bound to the object’s specific implementation.

6

Figure G000109. UE24CS151A PCPS Unit 4 SindhuRPai Class 11 Polymorphism · source slide page 6.

EV0000427

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

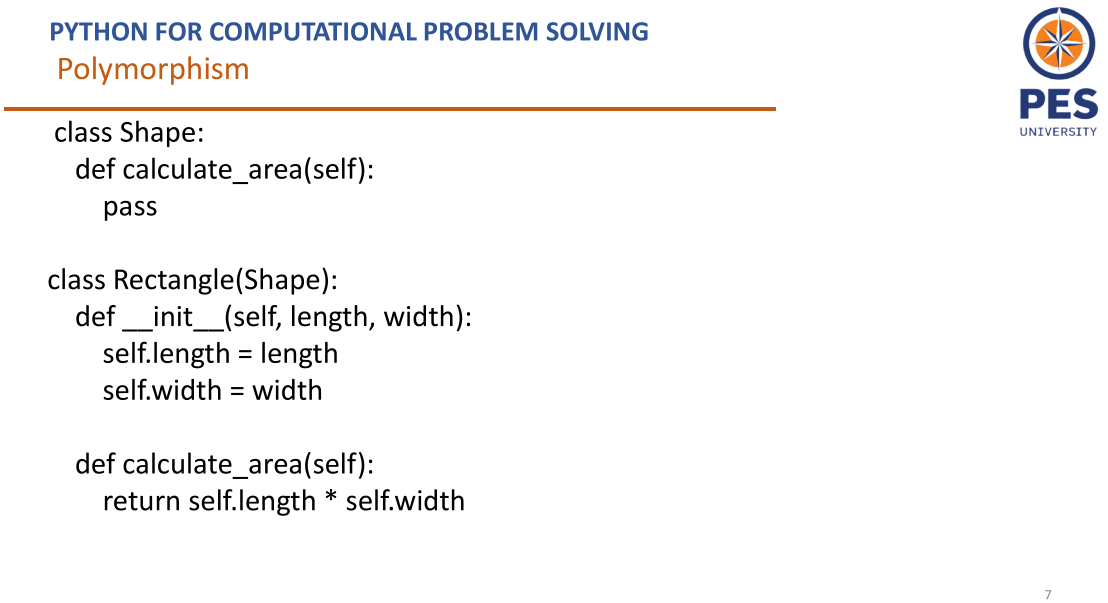


Figure G000110. UE24CS151A PCPS Unit 4 SindhuRPai Class 11 Polymorphism · source slide page 7.

EV0000430

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

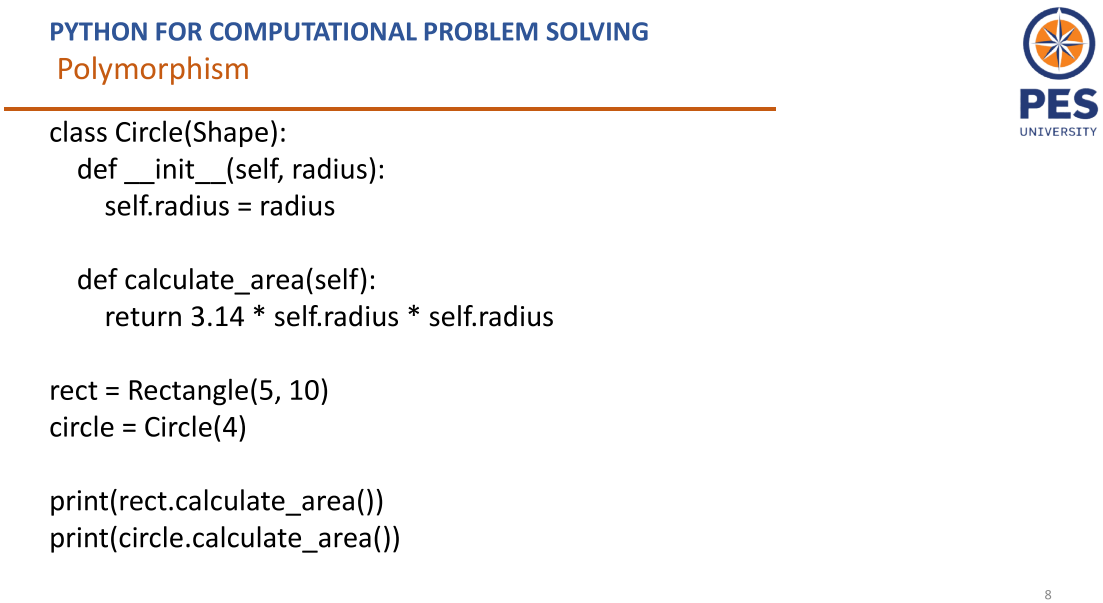


Figure G000111. UE24CS151A PCPS Unit 4 SindhuRPai Class 11 Polymorphism · source slide page 8.

EV0000433

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

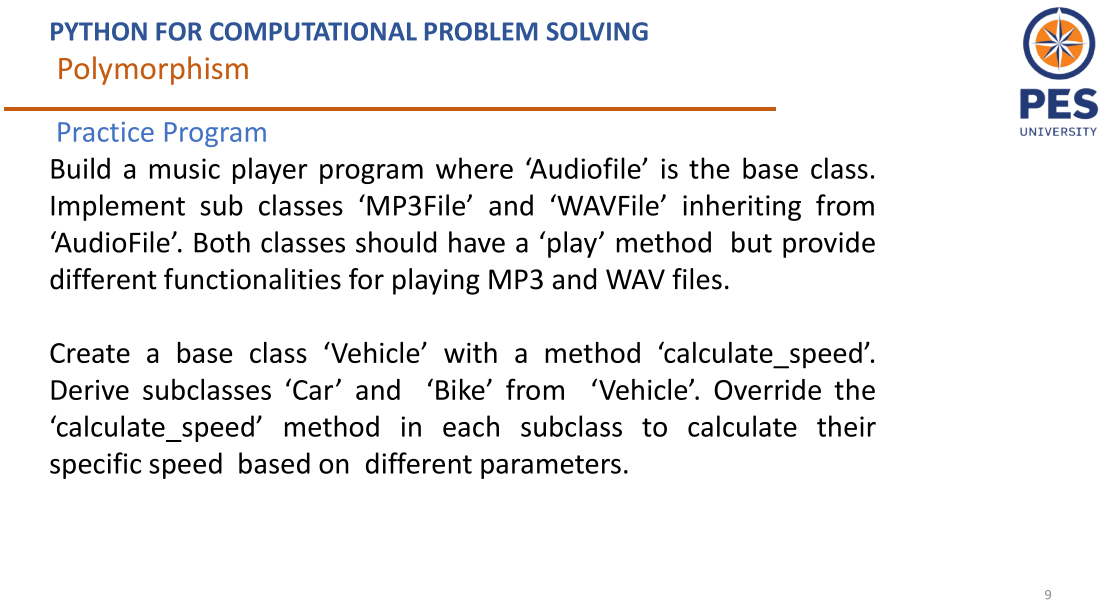


Figure G000112. UE24CS151A PCPS Unit 4 SindhuRPai Class 11 Polymorphism · source slide page 9.

EV0000436

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

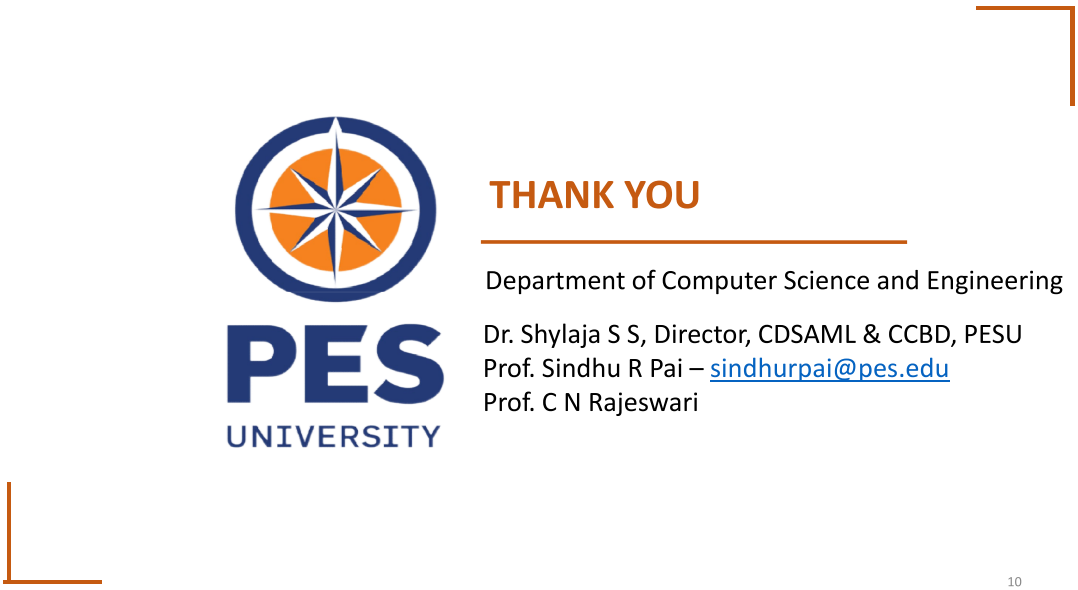


Figure G000113. UE24CS151A PCPS Unit 4 SindhuRPai Class 11 Polymorphism · source slide page 10.

EV0000439

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Important points and why

1. Source slide page 1 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Polymorphism Prof. Sindhu R Pai PCPS Theory Anchor - 2024 Department of Computer Science and Engineering EV0000411 · EV0000412 · EV0000413
2. Source slide page 2 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Polymorphism Introduction • Polymorphism refers to having multiple forms. • refersf to the use off the same ffunction name, but with differentff signatures • allows objects of different classes to be treated as objects of a common superclass. • This concept enables a single interface to be used for entities of different types. 2 EV0000414 · EV0000415 · EV0000416

3. Source slide page 3 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Polymorphism Runtime Polymorphism • Python supports runtime polymorphism by using techniques like method overloading and method overriding •• RuntimeRuntime polymorphismpolymorphism isis thethe abilityability ofof anan objectobject toto behavebehave differentlydifferently basedbased on its actual type during program execution • It is also known as dynamic polymorphism • It enables the same method name to behave differently based on the specific class instance at runtime. 3 EV0000417 · EV0000418 · EV0000419
4. Source slide page 4 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Polymorphism Key Aspects of Runtime Polymorphism: 1.Inheritance: • Runtime polymorphism is closely associated with inheritance. • Subclasses inherit methods from their superclass, and they can provided theirh own implementationl ffor theseh methods.h d 2.Method Overriding: • Subclasses override methods from their superclass to provide their own specialized implementation. • The method signature remains the same in both the superclass and subclass. 4 EV0000420 · EV0000421 · EV0000422
5. Source slide page 5 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Polymorphism 3.Dynamic Binding: • During runtime, the appropriate method to execute is dynamically determined • It is based on the actual type of the object invoking the method 4.Common Interface: • Different subclasses sharing a common superclass interface • Exhibits different behaviors based on their specific implementations. 5 EV0000423 · EV0000424 · EV0000425
6. Source slide page 6 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Polymorphism The following example (Refer next slide) demonstrates all these key features of runtime polymorphism. In this example ‘Rectangle ‘ and ‘circle’ subclasses override the ‘calculate_area’ method from the ‘shape’ superclass to provide their specific area calculation. The method ‘calculate_area’ is called on objects of different types(‘rect’ or ‘circle’) . The appropriate overridden method is executed based on the object’s actual type. This happens at the runtime and method execution is dynamically bound to the object’s specific implementation. 6 EV0000426 · EV0000427 · EV0000428
7. Source slide page 7 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Polymorphism class Shape: def calculate_area(self): pass class Rectangle(Shape): ddeff __init__(self,(lf llength,h width):d h) self.length = length self.width = width def calculate_area(self): return self.length * self.width 7 EV0000429 · EV0000430 · EV0000431
8. Source slide page 8 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Polymorphism class Circle(Shape): def __init__(self, radius): self.radius = radius def calculate_area(self): return 3.14 ** self.radiuslf d ** self.radiuslf d rect = Rectangle(5, 10) circle = Circle(4) print(rect.calculate_area()) print(circle.calculate_area()) 8 EV0000432 · EV0000433 · EV0000434
9. Source slide page 9 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Polymorphism Practice Program Build a music player program where ‘Audiofile’ is the base class. Implement sub classes ‘MP3File’ and ‘WAVFile’ inheriting from ‘AudioFile’. Both classes should have a ‘play’ method but provide different functionalities for playing MP3 and WAV files. Create a base class ‘Vehicle’ with a method ‘calculate_speed’. Derive subclasses ‘Car’ and ‘Bike’ from ‘Vehicle’. Override the ‘calculate_speed’ method in each subclass to calculate their specific speed based on different parameters. 9 EV0000435 · EV0000436 · EV0000437

10. Source slide page 10 (extracted text): THANK YOU Departmentp of Computerp Science and Engineeringg g Dr. Shylaja S S, Director, CDSAML & CCBD, PESU Prof. Sindhu R Pai – sindhurpai@pes.edu Prof. C N Rajeswari 10 EV0000438 · EV0000439 · EV0000440

SHORT-NOTE RECAP

Source deck preserved: UE24CS151A PCPS Unit 4 SindhuRPai Class 11 Polymorphism. Its extracted page text is listed as searchable points and all rendered source pages are attached as figures. Match to a specific video is score-based; this index does not claim a manual content review. EV0000411

10 Source slide deck: UE24CS151A PCPS Unit 4 SindhuRPai Class 12 Iterators

This entry preserves the complete slide deck titled UE24CS151A PCPS Unit 4 SindhuRPai Class 12 Iterators as searchable extracted page text and page-faithful rendered images. Its association with a particular video is not confirmed; it remains a separate topic under Python for computational Problem Solving, Unit_04, to avoid an unsupported match. EV0000441 · EV0000442

Graphs and diagrams

Figure G000114. UE24CS151A PCPS Unit 4 SindhuRPai Class 12 Iterators · source slide page 1.

EV0000442

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Introduction

PES UNIVERSITY

An iterator is an object that allows iteration through a sequence of elements, one at a time. It implements two main methods: **__iter__()** and **__next__()**.

__iter__():: returns the iterator object itself and is called when the iterator is initialized.

__next__():: returns the next item in the sequence.

When there are no more elements to return, it raises the **StopIteration** exception.

2

Figure G000115. UE24CS151A PCPS Unit 4 SindhuRPai Class 12 Iterators · source slide page 2.

EV0000445

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Introduction

PES UNIVERSITY

Iterators lazy object or Eager object?

Lazy Evaluation: Iterators follow the principle of lazy evaluation, meaning they generate values on-demand rather than computing all values at once.

This can be memory-efficient when dealing with large datasets as it only retrieves elements as needed.

3

Figure G000116. UE24CS151A PCPS Unit 4 SindhuRPai Class 12 Iterators · source slide page 3.

EV0000448

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Custom Iterable Objects

PES UNIVERSITY

The container class (like list) should support a function

1. __iter__(callable as iter(container-object)) which returns an object of a class called an iterator.

2. __next__(callable as next(iterator_object))

These two functions are interfaces which can be implemented by traversing through a container

Ex. we may visit only elements in odd position or elements satisfying a boolean condition – like elements greater than 100

4

Figure G000117. UE24CS151A PCPS Unit 4 SindhuRPai Class 12 Iterators · source slide page 4.

EV0000451

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

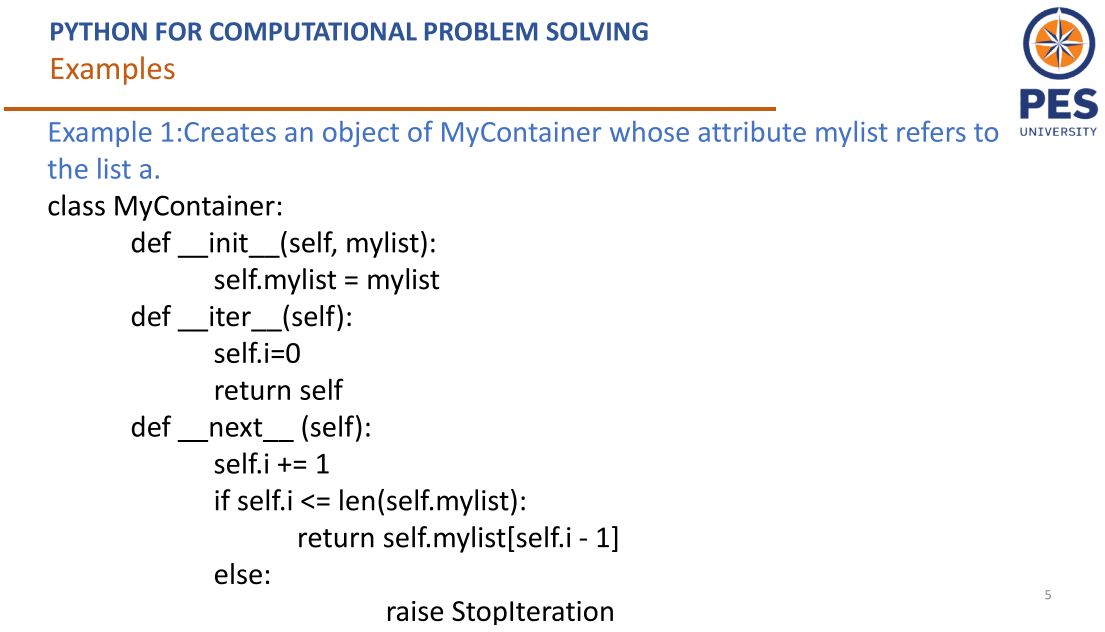


Figure G000118. UE24CS151A PCPS Unit 4 SindhuRPai Class 12 Iterators · source slide page 5.

EV0000454

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

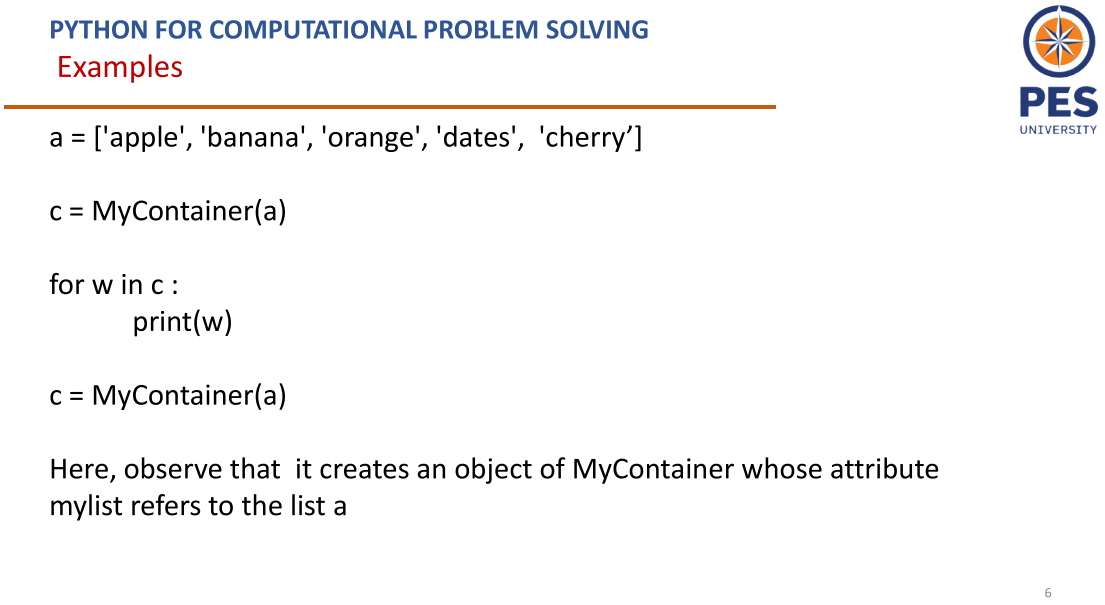


Figure G000119. UE24CS151A PCPS Unit 4 SindhuRPai Class 12 Iterators · source slide page 6.

EV0000457

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Working of iterators

PES UNIVERSITY

The for statement calls iter(c) which is changed to MyContainer.__iter__(c)

This __iter__ function adds a position attribute i to the object and then returns the MyContainer object itself as the iterator object.

The for statement keeps calling next on this iterable object.

The __next__ function has the logic to return the next element from the list and update the position and also raise the exception stop iteration when the end of the list is reached.

7

Figure G000120. UE24CS151A PCPS Unit 4 SindhuRPai Class 12 Iterators · source slide page 7.

EV0000460

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

PYTHON FOR COMPUTATIONAL PROBLEM SOLVING

Examples

PES UNIVERSITY

Example 2

```
class SquareNum:
    def __init__(self, n):
        self.n = n
        self.current = 0

    def __iter__(self):
        return self
    def __next__(self):
        if self.current >= self.n:
            raise StopIteration
        square = self.current ** 2
        self.current += 1
        return square
```

```
squares = SquareNum(5)
# Using the iterable class

for num in squares:
    print(num)
```

SquareNum is a class that generates a sequence of squares of numbers from 0 to n-1
It has __iter__() and __next__() methods implemented, making it iterable.
When instance of this class in a for loop, it iterates through the sequence, printing the squares of the numbers from 0 to 4

8

Figure G000121. UE24CS151A PCPS Unit 4 SindhuRPai Class 12 Iterators · source slide page 8.

EV0000463

How to read it. Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

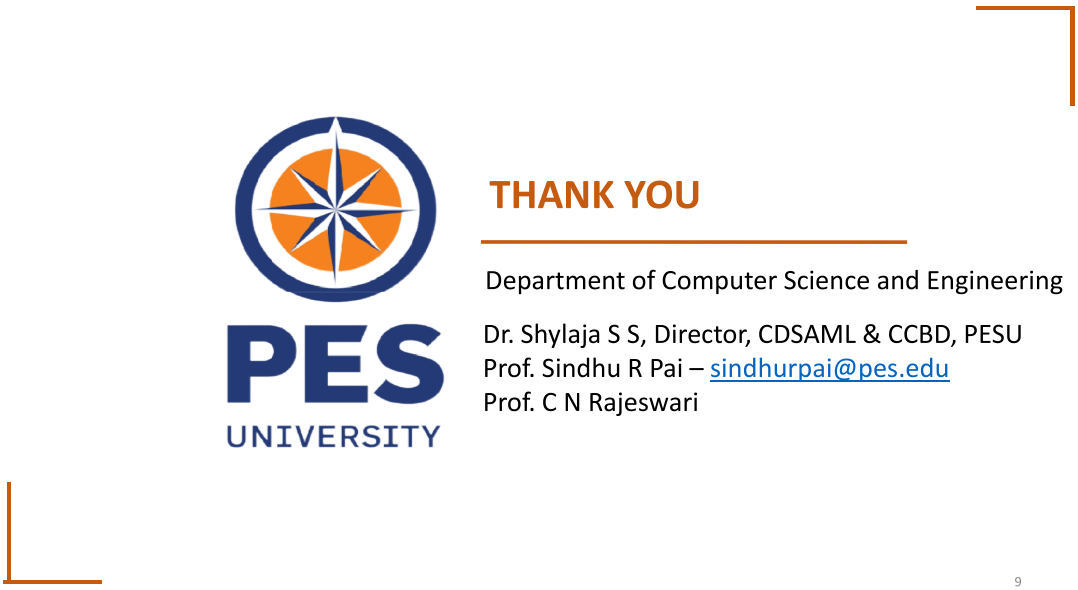


Figure G000122. UE24CS151A PCPS Unit 4 SindhuRPai Class 12 Iterators · source slide page 9. EV0000466

> ***How to read it.*** Use this original slide page to inspect formulas, diagrams, layout and any text extraction gaps. The original deck path and hash are recorded in its source metadata.

Important points and why

1. Source slide page 1 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Iterators in Python Prof. Sindhu R Pai PCPS Theory Anchor - 2024 Department of Computer Science and Engineering EV0000441 · EV0000442 · EV0000443
2. Source slide page 2 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Introduction An iterator is an object that allows iteration through a sequence of elements, one at a time. It implements two main methods: __iter__() and __next__(). __iter__()::iter ():: returnsreturns thethe iteratoriterator objectobject itselfitself andand isis calledcalled whenwhen thethe iteratoriterator is initialized. __next__():: returns the next item in the sequence. When there are no more elements to return, it raises the StopIteration exception. 2 EV0000444 · EV0000445 · EV0000446
3. Source slide page 3 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Introduction Iterators lazy object or Eager object? Lazyy Evaluation: Iterators follow the principlep p of lazyy evaluation,, meaningg they generate values on-demand rather than computing all values at once. This can be memory-efficient when dealing with large datasets as it only retrieves elements as needed. 3 EV0000447 · EV0000448 · EV0000449

4. Source slide page 4 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Custom Iterable Objects The container class (like list) should support a function 1. __iter__(callable as iter(container-object)) which returns an object of a class called an iterator. 2. __next__(callable(ll bl as next(iterator_object))(b)) These two functions are interfaces which can be implemented by traversing through a container Ex. we may visit only elements in odd position or elements satisfying a boolean condition – like elements greater than 100 4 EV0000450 · EV0000451 · EV0000452

5. Source slide page 5 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Examples Example 1:Creates an object of MyContainer whose attribute mylist refers to the list a. class MyContainer: def __init__(self, mylist): self.mylist = mylist ddeff __iter__(self):(lf) self.i=0 return self def __next__ (self): self.i += 1 if self.i <= len(self.mylist): return self.mylist[self.i - 1] else: 5 raise StopIteration EV0000453 · EV0000454 · EV0000455

6. Source slide page 6 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Examples a = ['apple', 'banana', 'orange', 'dates', 'cherry'] c = MyContainer(a) for w in c : print(w) c = MyContainer(a) Here, observe that it creates an object of MyContainer whose attribute mylist refers to the list a 6 EV0000456 · EV0000457 · EV0000458

7. Source slide page 7 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Working of iterators The for statement calls iter(c) which is changed to MyContainer.__iter__(c) This __iter__ function adds a position attribute i to the object and then returns the MyContainer object itself as the iterator object. The for statement keeps calling next on this iterable object. The __next__ function has the logic to return the next element from the list and update the position and also raise the exception stop iteration when the end of the list is reached. 7 EV0000459 · EV0000460 · EV0000461

8. Source slide page 8 (extracted text): PYTHON FOR COMPUTATIONAL PROBLEM SOLVING Examples Example 2 squares = SquareNum(5)class SquareNum: # Using the iterable class def __init__(self, n): self.n = n for num in squares: self.current = 0 print(num)print(num) def __iter__(self): SquareNum is a class that generates a sequence of return self squares of numbers from 0 to n-1 def __next__(self): It has __iter__() and __next__() methods if self.current >= self.n: implemented, making it iterable. raise StopIteration When instance of this class in a for loop, it iterates square = self.current ** 2 through the sequence, printing the squares of the self.current += 1 8 numbers from 0 to 4 return square EV0000462 · EV0000463 · EV0000464

9. Source slide page 9 (extracted text): THANK YOU Departmentp of Computerp Science and Engineeringg g Dr. Shylaja S S, Director, CDSAML & CCBD, PESU Prof. Sindhu R Pai – sindhurpai@pes.edu Prof. C N Rajeswari 9 EV0000465 · EV0000466 · EV0000467

SHORT-NOTE RECAP

Source deck preserved: UE24CS151A PCPS Unit 4 SindhuRPai Class 12 Iterators. Its extracted page text is listed as searchable points and all rendered source pages are attached as figures. Match to a specific video is score-based; this index does not claim a manual content review.

EV0000441

A Symbol glossary

Symbol	Name	Meaning	Appears in

B Sources, coverage and gaps

Source	Path	Use
Official unit catalogue	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/units.json	Official course/unit title and index.
Pytest, doctest, pdb transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/AV_Summaries/001_Pytest_doctest_pdb__9c2a04e725b61a85.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Pytest, doctest, pdb transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/AV_Summaries/001_Pytest_doctest_pdb__9c2a04e725b61a85.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Pytest, doctest, pdb transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/AV_Summaries/001_Pytest_doctest_pdb__9c2a04e725b61a85.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9991; only the JSON segmentation supplies the displayed timestamps.
Pytest, doctest, pdb transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/AV_Summaries/001_Pytest_doctest_pdb__9c2a04e725b61a85.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Pytest, doctest, pdb transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/AV_Summaries/001_Pytest_doctest_pdb__b3f1d7a13157fe7e.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.

Source	Path	Use
Pytest, doctest, pdb transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/AV_Summaries/001_Pytest_doctest_pdb__b3f1d7a13157fe7e.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Pytest, doctest, pdb transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/AV_Summaries/001_Pytest_doctest_pdb__b3f1d7a13157fe7e.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9982; only the JSON segmentation supplies the displayed timestamps.
Pytest, doctest, pdb transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/AV_Summaries/001_Pytest_doctest_pdb__b3f1d7a13157fe7e.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Python Classes and Objects transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/AV_Summaries/008_Python_Classes_and_Objects__67559daa97bde626.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Python Classes and Objects transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/AV_Summaries/008_Python_Classes_and_Objects__67559daa97bde626.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Python Classes and Objects transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/AV_Summaries/008_Python_Classes_and_Objects__67559daa97bde626.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9994; only the JSON segmentation supplies the displayed timestamps.

Source	Path	Use
Python Classes and Objects transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/AV_Summaries/008_Python_Classes_and_Objects__67559daa97bde626.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Inheritance transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/AV_Summaries/010_Inheritance__a6ea033571f65071.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Inheritance transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/AV_Summaries/010_Inheritance__a6ea033571f65071.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Inheritance transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/AV_Summaries/010_Inheritance__a6ea033571f65071.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9997; only the JSON segmentation supplies the displayed timestamps.
Inheritance transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/AV_Summaries/010_Inheritance__a6ea033571f65071.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Exceptions transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/AV_Summaries/015_Exceptions__524460b511d1f7c5.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Exceptions transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/AV_Summaries/015_Exceptions__524460b511d1f7c5.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.

Source	Path	Use
Exceptions transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/AV_Summaries/015_Exceptions__524460b511d1f7c5.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9999; only the JSON segmentation supplies the displayed timestamps.
Exceptions transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/AV_Summaries/015_Exceptions__524460b511d1f7c5.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Slide page text: UE24CS151A PCPS Unit 3 Sindhu Class 17 18 Pytest doctest pdb	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__17__18_Pytest_doctest_pdb__3cc021de2b8d71a0.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: UE24CS151A PCPS Unit 3 Sindhu Class 17 18 Pytest doctest pdb	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__17__18_Pytest_doctest_pdb__3cc021de2b8d71a0.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for UE24CS151A PCPS Unit 3 Sindhu Class 17 18 Pytest doctest pdb (converted PDF; original hash b78c970f591c88f9854f3ff5a639483d06dc76a8e2c1a5800c4e488b827de3db)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__17__18_Pytest_doctest_pdb__3cc021de2b8d71a0.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.
Slide page text: Virtual Environments	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/003_Virtual_Environments__ad787f97fa57d161.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.

Source	Path	Use
Alternate extracted slide page text: Virtual Environments	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/003_Virtual_Environments__ad787f97fa57d161.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for Virtual Environments (converted PDF; original hash 2dcb096e16cfd6f48004073b4b2115bc1fe9ca4aa6ba1ded8e075f9094ba2996)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/003_Virtual_Environments__ad787f97fa57d161.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.
Slide page text: UE24CS151A PCPS Unit 4 SindhuRPai Class 8 9 OOP	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/007_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__8__9_OOP__b4c4c4e7c25b5c27.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: UE24CS151A PCPS Unit 4 SindhuRPai Class 8 9 OOP	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/007_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__8__9_OOP__b4c4c4e7c25b5c27.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for UE24CS151A PCPS Unit 4 SindhuRPai Class 8 9 OOP (converted PDF; original hash 2965d957c0ddb606e258caa68f6cac39a01c30a87266fe53bf99981a0ca48ebb)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/007_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__8__9_OOP__b4c4c4e7c25b5c27.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.
Slide page text: UE24CS151A PCPS Unit 4 SindhuRPai Class 10 Inheritance	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/010_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__10_Inheritance__4dcdb777b0cfaae4.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.

Source	Path	Use
Alternate extracted slide page text: UE24CS151A PCPS Unit 4 SindhuRPai Class 10 Inheritance	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/010_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__10_Inheritance__4dcdb777b0cfaae4.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for UE24CS151A PCPS Unit 4 SindhuRPai Class 10 Inheritance (converted PDF; original hash 3cacb6bd7bcc912a6b169e4c3cbea336ae046494f1df0c927a54e282f5c19985)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/010_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__10_Inheritance__4dcdb777b0cfaae4.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.
Slide page text: UE24CS151A PCPS Unit 4 SindhuRPai Class 11 Polymorphism	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/011_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__11_Polymorphism__d914ed6dbb4c52c6.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: UE24CS151A PCPS Unit 4 SindhuRPai Class 11 Polymorphism	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/011_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__11_Polymorphism__d914ed6dbb4c52c6.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for UE24CS151A PCPS Unit 4 SindhuRPai Class 11 Polymorphism (converted PDF; original hash b68cc8fed979095224217876dccf47c02e508dddfdea2526dda58ed734b0f77a)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/011_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__11_Polymorphism__d914ed6dbb4c52c6.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.

Source	Path	Use
Slide page text: UE24CS151A PCPS Unit 4 SindhuRPai Class 12 Iterators	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/014_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__12_Iterators__a61be1414e88ae54.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: UE24CS151A PCPS Unit 4 SindhuRPai Class 12 Iterators	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/014_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__12_Iterators__a61be1414e88ae54.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for UE24CS151A PCPS Unit 4 SindhuRPai Class 12 Iterators (converted PDF; original hash e5e40e2fa3025f5bc0853935d4d4ae59522fb43ef182f9666f00317f7e9340d1)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/014_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__12_Iterators__a61be1414e88ae54.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.
Slide page text: UE24CS151A PCPS Unit 4 SindhuRPai Class 14 15 Exceptional Handling	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/015_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__14__15_Exceptional_Handling__147207e8e0b93741.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: UE24CS151A PCPS Unit 4 SindhuRPai Class 14 15 Exceptional Handling	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/015_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__14__15_Exceptional_Handling__147207e8e0b93741.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.

Source	Path	Use
Page-render source for UE24CS151A PCPS Unit 4 SindhuRPai Class 14 15 Exceptional Handling (converted PDF; original hash 55483b0bdaee29f016a9549096737614f28ed3e32a991169622b4b12a20ab12a)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/015_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__14__15_Exceptional_Handling__147207e8e0b93741.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.

Historical question-source excerpts are shown only when a page has a strong text-based unit match. They are source references, not new solutions or exam predictions.

Known gaps, conflicts and corrections

1. No historical archive question page met the unit-match candidate threshold; all subject-level and ambiguous pages remain in datasets/question-archive.json.
2. 1 video-to-slide associations were not promoted to a match. Candidate rankings are listed in the transcript corpus and video-slide-matches.csv; the slide decks appear separately.
3. 10 transcript segments have low ASR confidence; they are flagged with timestamps in the transcript blocks and indexed in the corpus.
4. This is a visibly labelled source-preservation draft. Human page-level semantic and visual review is not complete; no unsupported subject summary or formula correction is claimed.
5. The 19.26 GiB MP4 files remain hosted on Hugging Face; the full video path/hash and transcript coverage are indexed. The full audio/video was not watched end-to-end.
6. Formula candidates are machine-extracted text with source locators; use the adjacent original slide page and the formula index before treating a candidate as an authoritative equation.

Coverage

Every inventoried source unit has an explicit coverage decision. Covered means reviewed and mapped to content; exclusions and gaps remain visible.

Evidence	Source unit	Status	Decision
EV0000001	SRC00001/catalogue	covered	Unit title and index copied from the official units.json catalogue; human content review is pending.
EV0000002	SRC00002/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV00000003	SRC00003/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000004	SRC00004/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare at similarity 0.9991; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000005	SRC00005/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000006	SRC00002/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0021 to s0040. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000007	SRC00003/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0021 to s0040. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000008	SRC00004/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0021 to s0040. Sidecar variants compare at similarity 0.9991; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000009	SRC00005/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0021 to s0040. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV0000010	SRC00002/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0041 to s0042. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000011	SRC00003/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0041 to s0042. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000012	SRC00004/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0041 to s0042. Sidecar variants compare at similarity 0.9991; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000013	SRC00005/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0041 to s0042. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000014	SRC00006/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0019. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000015	SRC00007/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0019. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000016	SRC00008/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0019. Sidecar variants compare at similarity 0.9982; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV0000017	SRC00009/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0019. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000018	SRC00006/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0020 to s0021. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000019	SRC00007/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0020 to s0021. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000020	SRC00008/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0020 to s0021. Sidecar variants compare at similarity 0.9982; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000021	SRC00009/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0020 to s0021. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000022	SRC00010/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000023	SRC00011/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV0000024	SRC00012/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare at similarity 0.9994; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000025	SRC00013/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000026	SRC00010/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0022 to s0041. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000027	SRC00011/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0022 to s0041. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000028	SRC00012/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0022 to s0041. Sidecar variants compare at similarity 0.9994; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000029	SRC00013/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0022 to s0041. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000030	SRC00010/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0042 to s0060. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV0000031	SRC00011/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0042 to s0060. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000032	SRC00012/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0042 to s0060. Sidecar variants compare at similarity 0.9994; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000033	SRC00013/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0042 to s0060. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000034	SRC00010/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0061 to s0080. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000035	SRC00011/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0061 to s0080. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000036	SRC00012/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0061 to s0080. Sidecar variants compare at similarity 0.9994; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000037	SRC00013/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0061 to s0080. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV0000038	SRC00010/t001200-001499	covered	Timestamp interval t001200-001499; canonical segment IDs s0081 to s0100. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000039	SRC00011/t001200-001499	covered	Timestamp interval t001200-001499; canonical segment IDs s0081 to s0100. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000040	SRC00012/t001200-001499	covered	Timestamp interval t001200-001499; canonical segment IDs s0081 to s0100. Sidecar variants compare at similarity 0.9994; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000041	SRC00013/t001200-001499	covered	Timestamp interval t001200-001499; canonical segment IDs s0081 to s0100. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000042	SRC00010/t001500-001799	covered	Timestamp interval t001500-001799; canonical segment IDs s0101 to s0102. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000043	SRC00011/t001500-001799	covered	Timestamp interval t001500-001799; canonical segment IDs s0101 to s0102. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000044	SRC00012/t001500-001799	covered	Timestamp interval t001500-001799; canonical segment IDs s0101 to s0102. Sidecar variants compare at similarity 0.9994; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV0000045	SRC00013/t001500-001799	covered	Timestamp interval t001500-001799; canonical segment IDs s0101 to s0102. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000046	SRC00014/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000047	SRC00015/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000048	SRC00016/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare at similarity 0.9997; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000049	SRC00017/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000050	SRC00014/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0022 to s0041. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000051	SRC00015/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0022 to s0041. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV0000052	SRC00016/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0022 to s0041. Sidecar variants compare at similarity 0.9997; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000053	SRC00017/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0022 to s0041. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000054	SRC00014/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0042 to s0060. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000055	SRC00015/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0042 to s0060. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000056	SRC00016/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0042 to s0060. Sidecar variants compare at similarity 0.9997; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000057	SRC00017/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0042 to s0060. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000058	SRC00014/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0061 to s0064. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV0000059	SRC00015/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0061 to s0064. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000060	SRC00016/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0061 to s0064. Sidecar variants compare at similarity 0.9997; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000061	SRC00017/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0061 to s0064. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000062	SRC00018/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000063	SRC00019/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000064	SRC00020/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare at similarity 0.9999; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000065	SRC00021/t000000-000299	covered	Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV0000066	SRC00018/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0022 to s0042. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000067	SRC00019/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0022 to s0042. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000068	SRC00020/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0022 to s0042. Sidecar variants compare at similarity 0.9999; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000069	SRC00021/t000300-000599	covered	Timestamp interval t000300-000599; canonical segment IDs s0022 to s0042. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000070	SRC00018/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0043 to s0063. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000071	SRC00019/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0043 to s0063. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000072	SRC00020/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0043 to s0063. Sidecar variants compare at similarity 0.9999; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV0000073	SRC00021/t000600-000899	covered	Timestamp interval t000600-000899; canonical segment IDs s0043 to s0063. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000074	SRC00018/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0064 to s0083. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000075	SRC00019/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0064 to s0083. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000076	SRC00020/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0064 to s0083. Sidecar variants compare at similarity 0.9999; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000077	SRC00021/t000900-001199	covered	Timestamp interval t000900-001199; canonical segment IDs s0064 to s0083. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000078	SRC00018/t001200-001499	covered	Timestamp interval t001200-001499; canonical segment IDs s0084 to s0097. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000079	SRC00019/t001200-001499	covered	Timestamp interval t001200-001499; canonical segment IDs s0084 to s0097. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV00000080	SRC00020/t001200-001499	covered	Timestamp interval t001200-001499; canonical segment IDs s0084 to s0097. Sidecar variants compare at similarity 0.9999; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000081	SRC00021/t001200-001499	covered	Timestamp interval t001200-001499; canonical segment IDs s0084 to s0097. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000082	SRC00018/t001500-001799	covered	Timestamp interval t001500-001799; canonical segment IDs s0098 to s0112. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000083	SRC00019/t001500-001799	covered	Timestamp interval t001500-001799; canonical segment IDs s0098 to s0112. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000084	SRC00020/t001500-001799	covered	Timestamp interval t001500-001799; canonical segment IDs s0098 to s0112. Sidecar variants compare at similarity 0.9999; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000085	SRC00021/t001500-001799	covered	Timestamp interval t001500-001799; canonical segment IDs s0098 to s0112. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000086	SRC00018/t001800-002099	covered	Timestamp interval t001800-002099; canonical segment IDs s0113 to s0131. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV00000087	SRC00019/t001800-002099	covered	Timestamp interval t001800-002099; canonical segment IDs s0113 to s0131. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000088	SRC00020/t001800-002099	covered	Timestamp interval t001800-002099; canonical segment IDs s0113 to s0131. Sidecar variants compare at similarity 0.9999; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000089	SRC00021/t001800-002099	covered	Timestamp interval t001800-002099; canonical segment IDs s0113 to s0131. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000090	SRC00018/t002100-002399	covered	Timestamp interval t002100-002399; canonical segment IDs s0132 to s0150. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000091	SRC00019/t002100-002399	covered	Timestamp interval t002100-002399; canonical segment IDs s0132 to s0150. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000092	SRC00020/t002100-002399	covered	Timestamp interval t002100-002399; canonical segment IDs s0132 to s0150. Sidecar variants compare at similarity 0.9999; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV00000093	SRC00021/t002100-002399	covered	Timestamp interval t002100-002399; canonical segment IDs s0132 to s0150. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV0000094	SRC00018/t002400-002699	covered	Timestamp interval t002400-002699; canonical segment IDs s0151 to s0169. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000095	SRC00019/t002400-002699	covered	Timestamp interval t002400-002699; canonical segment IDs s0151 to s0169. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000096	SRC00020/t002400-002699	covered	Timestamp interval t002400-002699; canonical segment IDs s0151 to s0169. Sidecar variants compare at similarity 0.9999; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000097	SRC00021/t002400-002699	covered	Timestamp interval t002400-002699; canonical segment IDs s0151 to s0169. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000098	SRC00018/t002700-002999	covered	Timestamp interval t002700-002999; canonical segment IDs s0170 to s0184. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000099	SRC00019/t002700-002999	covered	Timestamp interval t002700-002999; canonical segment IDs s0170 to s0184. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000100	SRC00020/t002700-002999	covered	Timestamp interval t002700-002999; canonical segment IDs s0170 to s0184. Sidecar variants compare at similarity 0.9999; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.

Evidence	Source unit	Status	Decision
EV0000101	SRC00021/t002700-002999	covered	Timestamp interval t002700-002999; canonical segment IDs s0170 to s0184. Sidecar variants compare at similarity 1.0000; verbatim transcript text is included in the linked note. This is a source-coverage check, not an audiovisual fact check.
EV0000102	SRC00022/p1	covered	Extracted slide page 1 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__17__18_Pytest_doctest_pdb__3cc021de2b8d71a0.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000103	SRC00024/p1	covered	Original slide page 1 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__17__18_Pytest_doctest_pdb__3cc021de2b8d71a0.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000104	SRC00023/p1	covered	Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human comparison is pending.
EV0000105	SRC00022/p2	covered	Extracted slide page 2 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__17__18_Pytest_doctest_pdb__3cc021de2b8d71a0.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000106	SRC00024/p2	covered	Original slide page 2 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__17__18_Pytest_doctest_pdb__3cc021de2b8d71a0.pdf; visual page included as vector figure. Human layout/content review is pending.

Evidence	Source unit	Status	Decision
EV0000107	SRC00023/p2	covered	Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human comparison is pending.
EV0000108	SRC00022/p3	covered	Extracted slide page 3 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__17__18_Pytest_doctest_pdb__3cc021de2b8d71a0.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000109	SRC00024/p3	covered	Original slide page 3 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__17__18_Pytest_doctest_pdb__3cc021de2b8d71a0.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000110	SRC00023/p3	covered	Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human comparison is pending.
EV0000111	SRC00022/p4	covered	Extracted slide page 4 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__17__18_Pytest_doctest_pdb__3cc021de2b8d71a0.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000112	SRC00024/p4	covered	Original slide page 4 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__17__18_Pytest_doctest_pdb__3cc021de2b8d71a0.pdf; visual page included as vector figure. Human layout/content review is pending.

Evidence	Source unit	Status	Decision
EV0000113	SRC00023/p4	covered	Alternate extracted slide text page 4; normalized text matches the .slides.md page. Human comparison is pending.
EV0000114	SRC00022/p5	covered	Extracted slide page 5 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__17__18_Pytest_doctest_pdb__3cc021de2b8d71a0.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000115	SRC00024/p5	covered	Original slide page 5 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__17__18_Pytest_doctest_pdb__3cc021de2b8d71a0.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000116	SRC00023/p5	covered	Alternate extracted slide text page 5; normalized text matches the .slides.md page. Human comparison is pending.
EV0000117	SRC00022/p6	covered	Extracted slide page 6 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__17__18_Pytest_doctest_pdb__3cc021de2b8d71a0.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000118	SRC00024/p6	covered	Original slide page 6 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__17__18_Pytest_doctest_pdb__3cc021de2b8d71a0.pdf; visual page included as vector figure. Human layout/content review is pending.

Evidence	Source unit	Status	Decision
EV0000119	SRC00023/p6	covered	Alternate extracted slide text page 6; normalized text matches the .slides.md page. Human comparison is pending.
EV0000120	SRC00022/p7	covered	Extracted slide page 7 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__17__18_Pytest_doctest_pdb__3cc021de2b8d71a0.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000121	SRC00024/p7	covered	Original slide page 7 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__17__18_Pytest_doctest_pdb__3cc021de2b8d71a0.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000122	SRC00023/p7	covered	Alternate extracted slide text page 7; normalized text matches the .slides.md page. Human comparison is pending.
EV0000123	SRC00022/p8	covered	Extracted slide page 8 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__17__18_Pytest_doctest_pdb__3cc021de2b8d71a0.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000124	SRC00024/p8	covered	Original slide page 8 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__17__18_Pytest_doctest_pdb__3cc021de2b8d71a0.pdf; visual page included as vector figure. Human layout/content review is pending.

Evidence	Source unit	Status	Decision
EV0000125	SRC00023/p8	covered	Alternate extracted slide text page 8; normalized text matches the .slides.md page. Human comparison is pending.
EV0000126	SRC00022/p9	covered	Extracted slide page 9 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__17__18_Pytest_doctest_pdb__3cc021de2b8d71a0.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000127	SRC00024/p9	covered	Original slide page 9 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__17__18_Pytest_doctest_pdb__3cc021de2b8d71a0.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000128	SRC00023/p9	covered	Alternate extracted slide text page 9; normalized text matches the .slides.md page. Human comparison is pending.
EV0000129	SRC00022/p10	covered	Extracted slide page 10 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__17__18_Pytest_doctest_pdb__3cc021de2b8d71a0.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000130	SRC00024/p10	covered	Original slide page 10 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__17__18_Pytest_doctest_pdb__3cc021de2b8d71a0.pdf; visual page included as vector figure. Human layout/content review is pending.

Evidence	Source unit	Status	Decision
EV0000131	SRC00023/p10	covered	Alternate extracted slide text page 10; normalized text matches the .slides.md page. Human comparison is pending.
EV0000132	SRC00022/p11	covered	Extracted slide page 11 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__17__18_Pytest_doctest_pdb__3cc021de2b8d71a0.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000133	SRC00024/p11	covered	Original slide page 11 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__17__18_Pytest_doctest_pdb__3cc021de2b8d71a0.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000134	SRC00023/p11	covered	Alternate extracted slide text page 11; normalized text matches the .slides.md page. Human comparison is pending.
EV0000135	SRC00022/p12	covered	Extracted slide page 12 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__17__18_Pytest_doctest_pdb__3cc021de2b8d71a0.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000136	SRC00024/p12	covered	Original slide page 12 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__17__18_Pytest_doctest_pdb__3cc021de2b8d71a0.pdf; visual page included as vector figure. Human layout/content review is pending.

Evidence	Source unit	Status	Decision
EV0000137	SRC00023/p12	covered	Alternate extracted slide text page 12; normalized text matches the .slides.md page. Human comparison is pending.
EV0000138	SRC00022/p13	covered	Extracted slide page 13 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__17__18_Pytest_doctest_pdb__3cc021de2b8d71a0.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000139	SRC00024/p13	covered	Original slide page 13 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__17__18_Pytest_doctest_pdb__3cc021de2b8d71a0.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000140	SRC00023/p13	covered	Alternate extracted slide text page 13; normalized text matches the .slides.md page. Human comparison is pending.
EV0000141	SRC00022/p14	covered	Extracted slide page 14 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__17__18_Pytest_doctest_pdb__3cc021de2b8d71a0.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000142	SRC00024/p14	covered	Original slide page 14 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__17__18_Pytest_doctest_pdb__3cc021de2b8d71a0.pdf; visual page included as vector figure. Human layout/content review is pending.

Evidence	Source unit	Status	Decision
EV0000143	SRC00023/p14	covered	Alternate extracted slide text page 14; normalized text matches the .slides.md page. Human comparison is pending.
EV0000144	SRC00022/p15	covered	Extracted slide page 15 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__17__18_Pytest_doctest_pdb__3cc021de2b8d71a0.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000145	SRC00024/p15	covered	Original slide page 15 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__17__18_Pytest_doctest_pdb__3cc021de2b8d71a0.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000146	SRC00023/p15	covered	Alternate extracted slide text page 15; normalized text matches the .slides.md page. Human comparison is pending.
EV0000147	SRC00022/p16	covered	Extracted slide page 16 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__17__18_Pytest_doctest_pdb__3cc021de2b8d71a0.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000148	SRC00024/p16	covered	Original slide page 16 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__17__18_Pytest_doctest_pdb__3cc021de2b8d71a0.pdf; visual page included as vector figure. Human layout/content review is pending.

Evidence	Source unit	Status	Decision
EV0000149	SRC00023/p16	covered	Alternate extracted slide text page 16; normalized text matches the .slides.md page. Human comparison is pending.
EV0000150	SRC00022/p17	covered	Extracted slide page 17 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__17__18_Pytest_doctest_pdb__3cc021de2b8d71a0.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000151	SRC00024/p17	covered	Original slide page 17 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__17__18_Pytest_doctest_pdb__3cc021de2b8d71a0.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000152	SRC00023/p17	covered	Alternate extracted slide text page 17; normalized text matches the .slides.md page. Human comparison is pending.
EV0000153	SRC00022/p18	covered	Extracted slide page 18 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__17__18_Pytest_doctest_pdb__3cc021de2b8d71a0.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000154	SRC00024/p18	covered	Original slide page 18 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__17__18_Pytest_doctest_pdb__3cc021de2b8d71a0.pdf; visual page included as vector figure. Human layout/content review is pending.

Evidence	Source unit	Status	Decision
EV0000155	SRC00023/p18	covered	Alternate extracted slide text page 18; normalized text matches the .slides.md page. Human comparison is pending.
EV0000156	SRC00022/p19	covered	Extracted slide page 19 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__17__18_Pytest_doctest_pdb__3cc021de2b8d71a0.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000157	SRC00024/p19	covered	Original slide page 19 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__17__18_Pytest_doctest_pdb__3cc021de2b8d71a0.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000158	SRC00023/p19	covered	Alternate extracted slide text page 19; normalized text matches the .slides.md page. Human comparison is pending.
EV0000159	SRC00022/p20	covered	Extracted slide page 20 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__17__18_Pytest_doctest_pdb__3cc021de2b8d71a0.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000160	SRC00024/p20	covered	Original slide page 20 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__17__18_Pytest_doctest_pdb__3cc021de2b8d71a0.pdf; visual page included as vector figure. Human layout/content review is pending.

Evidence	Source unit	Status	Decision
EV0000161	SRC00023/p20	covered	Alternate extracted slide text page 20; normalized text matches the .slides.md page. Human comparison is pending.
EV0000162	SRC00022/p21	covered	Extracted slide page 21 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__17__18_Pytest_doctest_pdb__3cc021de2b8d71a0.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000163	SRC00024/p21	covered	Original slide page 21 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__17__18_Pytest_doctest_pdb__3cc021de2b8d71a0.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000164	SRC00023/p21	covered	Alternate extracted slide text page 21; normalized text matches the .slides.md page. Human comparison is pending.
EV0000165	SRC00022/p22	covered	Extracted slide page 22 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__17__18_Pytest_doctest_pdb__3cc021de2b8d71a0.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000166	SRC00024/p22	covered	Original slide page 22 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__17__18_Pytest_doctest_pdb__3cc021de2b8d71a0.pdf; visual page included as vector figure. Human layout/content review is pending.

Evidence	Source unit	Status	Decision
EV0000167	SRC00023/p22	covered	Alternate extracted slide text page 22; normalized text matches the .slides.md page. Human comparison is pending.
EV0000168	SRC00022/p23	covered	Extracted slide page 23 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__17__18_Pytest_doctest_pdb__3cc021de2b8d71a0.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000169	SRC00024/p23	covered	Original slide page 23 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__17__18_Pytest_doctest_pdb__3cc021de2b8d71a0.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000170	SRC00023/p23	covered	Alternate extracted slide text page 23; normalized text matches the .slides.md page. Human comparison is pending.
EV0000171	SRC00022/p24	covered	Extracted slide page 24 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__17__18_Pytest_doctest_pdb__3cc021de2b8d71a0.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000172	SRC00024/p24	covered	Original slide page 24 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__17__18_Pytest_doctest_pdb__3cc021de2b8d71a0.pdf; visual page included as vector figure. Human layout/content review is pending.

Evidence	Source unit	Status	Decision
EV0000173	SRC00023/p24	covered	Alternate extracted slide text page 24; normalized text matches the .slides.md page. Human comparison is pending.
EV0000174	SRC00022/p25	covered	Extracted slide page 25 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__17__18_Pytest_doctest_pdb__3cc021de2b8d71a0.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000175	SRC00024/p25	covered	Original slide page 25 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__17__18_Pytest_doctest_pdb__3cc021de2b8d71a0.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000176	SRC00023/p25	covered	Alternate extracted slide text page 25; normalized text matches the .slides.md page. Human comparison is pending.
EV0000177	SRC00022/p26	covered	Extracted slide page 26 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__17__18_Pytest_doctest_pdb__3cc021de2b8d71a0.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000178	SRC00024/p26	covered	Original slide page 26 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__17__18_Pytest_doctest_pdb__3cc021de2b8d71a0.pdf; visual page included as vector figure. Human layout/content review is pending.

Evidence	Source unit	Status	Decision
EV0000179	SRC00023/p26	covered	Alternate extracted slide text page 26; normalized text matches the .slides.md page. Human comparison is pending.
EV0000180	SRC00022/p27	covered	Extracted slide page 27 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__17__18_Pytest_doctest_pdb__3cc021de2b8d71a0.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000181	SRC00024/p27	covered	Original slide page 27 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__17__18_Pytest_doctest_pdb__3cc021de2b8d71a0.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000182	SRC00023/p27	covered	Alternate extracted slide text page 27; normalized text matches the .slides.md page. Human comparison is pending.
EV0000183	SRC00022/p28	covered	Extracted slide page 28 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__17__18_Pytest_doctest_pdb__3cc021de2b8d71a0.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000184	SRC00024/p28	covered	Original slide page 28 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__17__18_Pytest_doctest_pdb__3cc021de2b8d71a0.pdf; visual page included as vector figure. Human layout/content review is pending.

Evidence	Source unit	Status	Decision
EV0000185	SRC00023/p28	covered	Alternate extracted slide text page 28; normalized text matches the .slides.md page. Human comparison is pending.
EV0000186	SRC00025/p1	covered	Extracted slide page 1 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/003_Virtual_Environments__ad787f97fa57d161.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000187	SRC00027/p1	covered	Original slide page 1 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/003_Virtual_Environments__ad787f97fa57d161.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000188	SRC00026/p1	covered	Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human comparison is pending.
EV0000189	SRC00025/p2	covered	Extracted slide page 2 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/003_Virtual_Environments__ad787f97fa57d161.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000190	SRC00027/p2	covered	Original slide page 2 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/003_Virtual_Environments__ad787f97fa57d161.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000191	SRC00026/p2	covered	Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000192	SRC00025/p3	covered	Extracted slide page 3 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/003_Virtual_Environments__ad787f97fa57d161.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000193	SRC00027/p3	covered	Original slide page 3 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/003_Virtual_Environments__ad787f97fa57d161.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000194	SRC00026/p3	covered	Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human comparison is pending.
EV0000195	SRC00025/p4	covered	Extracted slide page 4 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/003_Virtual_Environments__ad787f97fa57d161.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000196	SRC00027/p4	covered	Original slide page 4 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/003_Virtual_Environments__ad787f97fa57d161.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000197	SRC00026/p4	covered	Alternate extracted slide text page 4; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000198	SRC00025/p5	covered	Extracted slide page 5 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/003_Virtual_Environments__ad787f97fa57d161.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000199	SRC00027/p5	covered	Original slide page 5 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/003_Virtual_Environments__ad787f97fa57d161.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000200	SRC00026/p5	covered	Alternate extracted slide text page 5; normalized text matches the .slides.md page. Human comparison is pending.
EV0000201	SRC00025/p6	covered	Extracted slide page 6 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/003_Virtual_Environments__ad787f97fa57d161.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000202	SRC00027/p6	covered	Original slide page 6 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/003_Virtual_Environments__ad787f97fa57d161.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000203	SRC00026/p6	covered	Alternate extracted slide text page 6; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000204	SRC00025/p7	covered	Extracted slide page 7 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/003_Virtual_Environments__ad787f97fa57d161.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000205	SRC00027/p7	covered	Original slide page 7 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/003_Virtual_Environments__ad787f97fa57d161.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000206	SRC00026/p7	covered	Alternate extracted slide text page 7; normalized text matches the .slides.md page. Human comparison is pending.
EV0000207	SRC00025/p8	covered	Extracted slide page 8 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/003_Virtual_Environments__ad787f97fa57d161.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000208	SRC00027/p8	covered	Original slide page 8 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/003_Virtual_Environments__ad787f97fa57d161.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000209	SRC00026/p8	covered	Alternate extracted slide text page 8; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000210	SRC00025/p9	covered	Extracted slide page 9 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/003_Virtual_Environments__ad787f97fa57d161.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000211	SRC00027/p9	covered	Original slide page 9 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/003_Virtual_Environments__ad787f97fa57d161.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000212	SRC00026/p9	covered	Alternate extracted slide text page 9; normalized text matches the .slides.md page. Human comparison is pending.
EV0000213	SRC00025/p10	covered	Extracted slide page 10 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/003_Virtual_Environments__ad787f97fa57d161.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000214	SRC00027/p10	covered	Original slide page 10 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/003_Virtual_Environments__ad787f97fa57d161.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000215	SRC00026/p10	covered	Alternate extracted slide text page 10; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000216	SRC00025/p11	covered	Extracted slide page 11 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/003_Virtual_Environments__ad787f97fa57d161.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000217	SRC00027/p11	covered	Original slide page 11 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/003_Virtual_Environments__ad787f97fa57d161.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000218	SRC00026/p11	covered	Alternate extracted slide text page 11; normalized text matches the .slides.md page. Human comparison is pending.
EV0000219	SRC00025/p12	covered	Extracted slide page 12 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/003_Virtual_Environments__ad787f97fa57d161.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000220	SRC00027/p12	covered	Original slide page 12 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/003_Virtual_Environments__ad787f97fa57d161.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000221	SRC00026/p12	covered	Alternate extracted slide text page 12; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000222	SRC00025/p13	covered	Extracted slide page 13 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/003_Virtual_Environments__ad787f97fa57d161.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000223	SRC00027/p13	covered	Original slide page 13 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/003_Virtual_Environments__ad787f97fa57d161.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000224	SRC00026/p13	covered	Alternate extracted slide text page 13; normalized text matches the .slides.md page. Human comparison is pending.
EV0000225	SRC00025/p14	covered	Extracted slide page 14 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/003_Virtual_Environments__ad787f97fa57d161.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000226	SRC00027/p14	covered	Original slide page 14 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/003_Virtual_Environments__ad787f97fa57d161.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000227	SRC00026/p14	covered	Alternate extracted slide text page 14; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000228	SRC00025/p15	covered	Extracted slide page 15 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/003_Virtual_Environments__ad787f97fa57d161.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000229	SRC00027/p15	covered	Original slide page 15 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/003_Virtual_Environments__ad787f97fa57d161.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000230	SRC00026/p15	covered	Alternate extracted slide text page 15; normalized text matches the .slides.md page. Human comparison is pending.
EV0000231	SRC00025/p16	covered	Extracted slide page 16 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/003_Virtual_Environments__ad787f97fa57d161.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000232	SRC00027/p16	covered	Original slide page 16 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/003_Virtual_Environments__ad787f97fa57d161.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000233	SRC00026/p16	covered	Alternate extracted slide text page 16; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000234	SRC00025/p17	covered	Extracted slide page 17 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/003_Virtual_Environments__ad787f97fa57d161.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000235	SRC00027/p17	covered	Original slide page 17 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/003_Virtual_Environments__ad787f97fa57d161.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000236	SRC00026/p17	covered	Alternate extracted slide text page 17; normalized text matches the .slides.md page. Human comparison is pending.
EV0000237	SRC00025/p18	covered	Extracted slide page 18 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/003_Virtual_Environments__ad787f97fa57d161.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000238	SRC00027/p18	covered	Original slide page 18 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/003_Virtual_Environments__ad787f97fa57d161.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000239	SRC00026/p18	covered	Alternate extracted slide text page 18; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000240	SRC00028/p1	covered	Extracted slide page 1 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/007_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__8__9_OOP__b4c4c4e7c25b5c27.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000241	SRC00030/p1	covered	Original slide page 1 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/007_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__8__9_OOP__b4c4c4e7c25b5c27.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000242	SRC00029/p1	covered	Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human comparison is pending.
EV0000243	SRC00028/p2	covered	Extracted slide page 2 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/007_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__8__9_OOP__b4c4c4e7c25b5c27.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000244	SRC00030/p2	covered	Original slide page 2 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/007_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__8__9_OOP__b4c4c4e7c25b5c27.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000245	SRC00029/p2	covered	Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000246	SRC00028/p3	covered	Extracted slide page 3 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/007_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__8__9_OOP__b4c4c4e7c25b5c27.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000247	SRC00030/p3	covered	Original slide page 3 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/007_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__8__9_OOP__b4c4c4e7c25b5c27.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000248	SRC00029/p3	covered	Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human comparison is pending.
EV0000249	SRC00028/p4	covered	Extracted slide page 4 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/007_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__8__9_OOP__b4c4c4e7c25b5c27.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000250	SRC00030/p4	covered	Original slide page 4 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/007_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__8__9_OOP__b4c4c4e7c25b5c27.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000251	SRC00029/p4	covered	Alternate extracted slide text page 4; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000252	SRC00028/p5	covered	Extracted slide page 5 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/007_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__8__9_OOP__b4c4c4e7c25b5c27.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000253	SRC00030/p5	covered	Original slide page 5 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/007_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__8__9_OOP__b4c4c4e7c25b5c27.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000254	SRC00029/p5	covered	Alternate extracted slide text page 5; normalized text matches the .slides.md page. Human comparison is pending.
EV0000255	SRC00028/p6	covered	Extracted slide page 6 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/007_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__8__9_OOP__b4c4c4e7c25b5c27.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000256	SRC00030/p6	covered	Original slide page 6 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/007_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__8__9_OOP__b4c4c4e7c25b5c27.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000257	SRC00029/p6	covered	Alternate extracted slide text page 6; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000258	SRC00028/p7	covered	Extracted slide page 7 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/007_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__8__9_OOP__b4c4c4e7c25b5c27.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000259	SRC00030/p7	covered	Original slide page 7 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/007_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__8__9_OOP__b4c4c4e7c25b5c27.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000260	SRC00029/p7	covered	Alternate extracted slide text page 7; normalized text matches the .slides.md page. Human comparison is pending.
EV0000261	SRC00028/p8	covered	Extracted slide page 8 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/007_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__8__9_OOP__b4c4c4e7c25b5c27.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000262	SRC00030/p8	covered	Original slide page 8 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/007_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__8__9_OOP__b4c4c4e7c25b5c27.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000263	SRC00029/p8	covered	Alternate extracted slide text page 8; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000264	SRC00028/p9	covered	Extracted slide page 9 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/007_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__8__9_OOP__b4c4c4e7c25b5c27.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000265	SRC00030/p9	covered	Original slide page 9 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/007_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__8__9_OOP__b4c4c4e7c25b5c27.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000266	SRC00029/p9	covered	Alternate extracted slide text page 9; normalized text matches the .slides.md page. Human comparison is pending.
EV0000267	SRC00028/p10	covered	Extracted slide page 10 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/007_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__8__9_OOP__b4c4c4e7c25b5c27.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000268	SRC00030/p10	covered	Original slide page 10 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/007_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__8__9_OOP__b4c4c4e7c25b5c27.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000269	SRC00029/p10	covered	Alternate extracted slide text page 10; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000270	SRC00028/p11	covered	Extracted slide page 11 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/007_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__8__9_OOP__b4c4c4e7c25b5c27.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000271	SRC00030/p11	covered	Original slide page 11 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/007_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__8__9_OOP__b4c4c4e7c25b5c27.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000272	SRC00029/p11	covered	Alternate extracted slide text page 11; normalized text matches the .slides.md page. Human comparison is pending.
EV0000273	SRC00028/p12	covered	Extracted slide page 12 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/007_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__8__9_OOP__b4c4c4e7c25b5c27.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000274	SRC00030/p12	covered	Original slide page 12 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/007_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__8__9_OOP__b4c4c4e7c25b5c27.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000275	SRC00029/p12	covered	Alternate extracted slide text page 12; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000276	SRC00028/p13	covered	Extracted slide page 13 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/007_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__8__9_OOP__b4c4c4e7c25b5c27.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000277	SRC00030/p13	covered	Original slide page 13 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/007_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__8__9_OOP__b4c4c4e7c25b5c27.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000278	SRC00029/p13	covered	Alternate extracted slide text page 13; normalized text matches the .slides.md page. Human comparison is pending.
EV0000279	SRC00028/p14	covered	Extracted slide page 14 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/007_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__8__9_OOP__b4c4c4e7c25b5c27.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000280	SRC00030/p14	covered	Original slide page 14 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/007_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__8__9_OOP__b4c4c4e7c25b5c27.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000281	SRC00029/p14	covered	Alternate extracted slide text page 14; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000282	SRC00028/p15	covered	Extracted slide page 15 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/007_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__8__9_OOP__b4c4c4e7c25b5c27.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000283	SRC00030/p15	covered	Original slide page 15 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/007_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__8__9_OOP__b4c4c4e7c25b5c27.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000284	SRC00029/p15	covered	Alternate extracted slide text page 15; normalized text matches the .slides.md page. Human comparison is pending.
EV0000285	SRC00028/p16	covered	Extracted slide page 16 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/007_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__8__9_OOP__b4c4c4e7c25b5c27.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000286	SRC00030/p16	covered	Original slide page 16 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/007_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__8__9_OOP__b4c4c4e7c25b5c27.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000287	SRC00029/p16	covered	Alternate extracted slide text page 16; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000288	SRC00028/p17	covered	Extracted slide page 17 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/007_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__8__9_OOP__b4c4c4e7c25b5c27.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000289	SRC00030/p17	covered	Original slide page 17 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/007_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__8__9_OOP__b4c4c4e7c25b5c27.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000290	SRC00029/p17	covered	Alternate extracted slide text page 17; normalized text matches the .slides.md page. Human comparison is pending.
EV0000291	SRC00028/p18	covered	Extracted slide page 18 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/007_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__8__9_OOP__b4c4c4e7c25b5c27.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000292	SRC00030/p18	covered	Original slide page 18 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/007_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__8__9_OOP__b4c4c4e7c25b5c27.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000293	SRC00029/p18	covered	Alternate extracted slide text page 18; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000294	SRC00028/p19	covered	Extracted slide page 19 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/007_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__8__9_OOP__b4c4c4e7c25b5c27.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000295	SRC00030/p19	covered	Original slide page 19 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/007_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__8__9_OOP__b4c4c4e7c25b5c27.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000296	SRC00029/p19	covered	Alternate extracted slide text page 19; normalized text matches the .slides.md page. Human comparison is pending.
EV0000297	SRC00028/p20	covered	Extracted slide page 20 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/007_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__8__9_OOP__b4c4c4e7c25b5c27.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000298	SRC00030/p20	covered	Original slide page 20 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/007_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__8__9_OOP__b4c4c4e7c25b5c27.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000299	SRC00029/p20	covered	Alternate extracted slide text page 20; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000300	SRC00028/p21	covered	Extracted slide page 21 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/007_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__8__9_OOP__b4c4c4e7c25b5c27.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000301	SRC00030/p21	covered	Original slide page 21 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/007_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__8__9_OOP__b4c4c4e7c25b5c27.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000302	SRC00029/p21	covered	Alternate extracted slide text page 21; normalized text matches the .slides.md page. Human comparison is pending.
EV0000303	SRC00028/p22	covered	Extracted slide page 22 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/007_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__8__9_OOP__b4c4c4e7c25b5c27.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000304	SRC00030/p22	covered	Original slide page 22 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/007_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__8__9_OOP__b4c4c4e7c25b5c27.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000305	SRC00029/p22	covered	Alternate extracted slide text page 22; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000306	SRC00028/p23	covered	Extracted slide page 23 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/007_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__8__9_OOP__b4c4c4e7c25b5c27.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000307	SRC00030/p23	covered	Original slide page 23 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/007_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__8__9_OOP__b4c4c4e7c25b5c27.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000308	SRC00029/p23	covered	Alternate extracted slide text page 23; normalized text matches the .slides.md page. Human comparison is pending.
EV0000309	SRC00028/p24	covered	Extracted slide page 24 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/007_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__8__9_OOP__b4c4c4e7c25b5c27.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000310	SRC00030/p24	covered	Original slide page 24 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/007_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__8__9_OOP__b4c4c4e7c25b5c27.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000311	SRC00029/p24	covered	Alternate extracted slide text page 24; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000312	SRC00028/p25	covered	Extracted slide page 25 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/007_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__8__9_OOP__b4c4c4e7c25b5c27.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000313	SRC00030/p25	covered	Original slide page 25 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/007_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__8__9_OOP__b4c4c4e7c25b5c27.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000314	SRC00029/p25	covered	Alternate extracted slide text page 25; normalized text matches the .slides.md page. Human comparison is pending.
EV0000315	SRC00028/p26	covered	Extracted slide page 26 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/007_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__8__9_OOP__b4c4c4e7c25b5c27.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000316	SRC00030/p26	covered	Original slide page 26 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/007_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__8__9_OOP__b4c4c4e7c25b5c27.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000317	SRC00029/p26	covered	Alternate extracted slide text page 26; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000318	SRC00028/p27	covered	Extracted slide page 27 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/007_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__8__9_OOP__b4c4c4e7c25b5c27.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000319	SRC00030/p27	covered	Original slide page 27 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/007_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__8__9_OOP__b4c4c4e7c25b5c27.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000320	SRC00029/p27	covered	Alternate extracted slide text page 27; normalized text matches the .slides.md page. Human comparison is pending.
EV0000321	SRC00028/p28	covered	Extracted slide page 28 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/007_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__8__9_OOP__b4c4c4e7c25b5c27.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000322	SRC00030/p28	covered	Original slide page 28 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/007_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__8__9_OOP__b4c4c4e7c25b5c27.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000323	SRC00029/p28	covered	Alternate extracted slide text page 28; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000324	SRC00028/p29	covered	Extracted slide page 29 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/007_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__8__9_OOP__b4c4c4e7c25b5c27.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000325	SRC00030/p29	covered	Original slide page 29 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/007_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__8__9_OOP__b4c4c4e7c25b5c27.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000326	SRC00029/p29	covered	Alternate extracted slide text page 29; normalized text matches the .slides.md page. Human comparison is pending.
EV0000327	SRC00028/p30	covered	Extracted slide page 30 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/007_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__8__9_OOP__b4c4c4e7c25b5c27.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000328	SRC00030/p30	covered	Original slide page 30 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/007_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__8__9_OOP__b4c4c4e7c25b5c27.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000329	SRC00029/p30	covered	Alternate extracted slide text page 30; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000330	SRC00028/p31	covered	Extracted slide page 31 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/007_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__8__9_OOP__b4c4c4e7c25b5c27.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000331	SRC00030/p31	covered	Original slide page 31 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/007_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__8__9_OOP__b4c4c4e7c25b5c27.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000332	SRC00029/p31	covered	Alternate extracted slide text page 31; normalized text matches the .slides.md page. Human comparison is pending.
EV0000333	SRC00031/p1	covered	Extracted slide page 1 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/010_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__10_Inheritance__4dcdb777b0cfaae4.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000334	SRC00033/p1	covered	Original slide page 1 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/010_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__10_Inheritance__4dcdb777b0cfaae4.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000335	SRC00032/p1	covered	Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000336	SRC00031/p2	covered	Extracted slide page 2 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/010_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__10_Inheritance__4dcdb777b0cfaae4.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000337	SRC00033/p2	covered	Original slide page 2 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/010_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__10_Inheritance__4dcdb777b0cfaae4.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000338	SRC00032/p2	covered	Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human comparison is pending.
EV0000339	SRC00031/p3	covered	Extracted slide page 3 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/010_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__10_Inheritance__4dcdb777b0cfaae4.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000340	SRC00033/p3	covered	Original slide page 3 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/010_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__10_Inheritance__4dcdb777b0cfaae4.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000341	SRC00032/p3	covered	Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000342	SRC00031/p4	covered	Extracted slide page 4 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/010_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__10_Inheritance__4dcdb777b0cfaae4.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000343	SRC00033/p4	covered	Original slide page 4 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/010_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__10_Inheritance__4dcdb777b0cfaae4.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000344	SRC00032/p4	covered	Alternate extracted slide text page 4; normalized text matches the .slides.md page. Human comparison is pending.
EV0000345	SRC00031/p5	covered	Extracted slide page 5 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/010_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__10_Inheritance__4dcdb777b0cfaae4.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000346	SRC00033/p5	covered	Original slide page 5 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/010_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__10_Inheritance__4dcdb777b0cfaae4.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000347	SRC00032/p5	covered	Alternate extracted slide text page 5; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000348	SRC00031/p6	covered	Extracted slide page 6 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/010_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__10_Inheritance__4dcdb777b0cfaae4.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000349	SRC00033/p6	covered	Original slide page 6 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/010_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__10_Inheritance__4dcdb777b0cfaae4.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000350	SRC00032/p6	covered	Alternate extracted slide text page 6; normalized text matches the .slides.md page. Human comparison is pending.
EV0000351	SRC00031/p7	covered	Extracted slide page 7 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/010_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__10_Inheritance__4dcdb777b0cfaae4.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000352	SRC00033/p7	covered	Original slide page 7 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/010_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__10_Inheritance__4dcdb777b0cfaae4.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000353	SRC00032/p7	covered	Alternate extracted slide text page 7; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000354	SRC00031/p8	covered	Extracted slide page 8 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/010_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__10_Inheritance__4dcdb777b0cfaae4.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000355	SRC00033/p8	covered	Original slide page 8 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/010_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__10_Inheritance__4dcdb777b0cfaae4.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000356	SRC00032/p8	covered	Alternate extracted slide text page 8; normalized text matches the .slides.md page. Human comparison is pending.
EV0000357	SRC00031/p9	covered	Extracted slide page 9 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/010_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__10_Inheritance__4dcdb777b0cfaae4.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000358	SRC00033/p9	covered	Original slide page 9 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/010_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__10_Inheritance__4dcdb777b0cfaae4.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000359	SRC00032/p9	covered	Alternate extracted slide text page 9; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000360	SRC00031/p10	covered	Extracted slide page 10 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/010_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__10_Inheritance__4dcdb777b0cfaae4.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000361	SRC00033/p10	covered	Original slide page 10 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/010_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__10_Inheritance__4dcdb777b0cfaae4.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000362	SRC00032/p10	covered	Alternate extracted slide text page 10; normalized text matches the .slides.md page. Human comparison is pending.
EV0000363	SRC00031/p11	covered	Extracted slide page 11 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/010_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__10_Inheritance__4dcdb777b0cfaae4.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000364	SRC00033/p11	covered	Original slide page 11 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/010_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__10_Inheritance__4dcdb777b0cfaae4.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000365	SRC00032/p11	covered	Alternate extracted slide text page 11; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000366	SRC00031/p12	covered	Extracted slide page 12 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/010_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__10_Inheritance__4dcdb777b0cfaae4.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000367	SRC00033/p12	covered	Original slide page 12 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/010_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__10_Inheritance__4dcdb777b0cfaae4.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000368	SRC00032/p12	covered	Alternate extracted slide text page 12; normalized text matches the .slides.md page. Human comparison is pending.
EV0000369	SRC00031/p13	covered	Extracted slide page 13 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/010_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__10_Inheritance__4dcdb777b0cfaae4.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000370	SRC00033/p13	covered	Original slide page 13 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/010_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__10_Inheritance__4dcdb777b0cfaae4.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000371	SRC00032/p13	covered	Alternate extracted slide text page 13; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000372	SRC00031/p14	covered	Extracted slide page 14 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/010_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__10_Inheritance__4dcdb777b0cfaae4.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000373	SRC00033/p14	covered	Original slide page 14 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/010_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__10_Inheritance__4dcdb777b0cfaae4.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000374	SRC00032/p14	covered	Alternate extracted slide text page 14; normalized text matches the .slides.md page. Human comparison is pending.
EV0000375	SRC00031/p15	covered	Extracted slide page 15 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/010_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__10_Inheritance__4dcdb777b0cfaae4.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000376	SRC00033/p15	covered	Original slide page 15 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/010_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__10_Inheritance__4dcdb777b0cfaae4.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000377	SRC00032/p15	covered	Alternate extracted slide text page 15; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000378	SRC00031/p16	covered	Extracted slide page 16 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/010_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__10_Inheritance__4dcdb777b0cfaae4.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000379	SRC00033/p16	covered	Original slide page 16 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/010_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__10_Inheritance__4dcdb777b0cfaae4.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000380	SRC00032/p16	covered	Alternate extracted slide text page 16; normalized text matches the .slides.md page. Human comparison is pending.
EV0000381	SRC00031/p17	covered	Extracted slide page 17 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/010_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__10_Inheritance__4dcdb777b0cfaae4.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000382	SRC00033/p17	covered	Original slide page 17 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/010_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__10_Inheritance__4dcdb777b0cfaae4.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000383	SRC00032/p17	covered	Alternate extracted slide text page 17; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000384	SRC00031/p18	covered	Extracted slide page 18 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/010_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__10_Inheritance__4dcdb777b0cfaae4.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000385	SRC00033/p18	covered	Original slide page 18 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/010_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__10_Inheritance__4dcdb777b0cfaae4.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000386	SRC00032/p18	covered	Alternate extracted slide text page 18; normalized text matches the .slides.md page. Human comparison is pending.
EV0000387	SRC00031/p19	covered	Extracted slide page 19 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/010_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__10_Inheritance__4dcdb777b0cfaae4.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000388	SRC00033/p19	covered	Original slide page 19 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/010_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__10_Inheritance__4dcdb777b0cfaae4.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000389	SRC00032/p19	covered	Alternate extracted slide text page 19; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000390	SRC00031/p20	covered	Extracted slide page 20 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/010_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__10_Inheritance__4dcdb777b0cfaae4.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000391	SRC00033/p20	covered	Original slide page 20 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/010_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__10_Inheritance__4dcdb777b0cfaae4.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000392	SRC00032/p20	covered	Alternate extracted slide text page 20; normalized text matches the .slides.md page. Human comparison is pending.
EV0000393	SRC00031/p21	covered	Extracted slide page 21 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/010_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__10_Inheritance__4dcdb777b0cfaae4.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000394	SRC00033/p21	covered	Original slide page 21 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/010_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__10_Inheritance__4dcdb777b0cfaae4.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000395	SRC00032/p21	covered	Alternate extracted slide text page 21; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000396	SRC00031/p22	covered	Extracted slide page 22 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/010_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__10_Inheritance__4dcdb777b0cfaae4.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000397	SRC00033/p22	covered	Original slide page 22 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/010_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__10_Inheritance__4dcdb777b0cfaae4.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000398	SRC00032/p22	covered	Alternate extracted slide text page 22; normalized text matches the .slides.md page. Human comparison is pending.
EV0000399	SRC00031/p23	covered	Extracted slide page 23 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/010_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__10_Inheritance__4dcdb777b0cfaae4.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000400	SRC00033/p23	covered	Original slide page 23 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/010_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__10_Inheritance__4dcdb777b0cfaae4.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000401	SRC00032/p23	covered	Alternate extracted slide text page 23; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000402	SRC00031/p24	covered	Extracted slide page 24 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/010_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__10_Inheritance__4dcdb777b0cfaae4.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000403	SRC00033/p24	covered	Original slide page 24 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/010_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__10_Inheritance__4dcdb777b0cfaae4.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000404	SRC00032/p24	covered	Alternate extracted slide text page 24; normalized text matches the .slides.md page. Human comparison is pending.
EV0000405	SRC00031/p25	covered	Extracted slide page 25 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/010_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__10_Inheritance__4dcdb777b0cfaae4.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000406	SRC00033/p25	covered	Original slide page 25 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/010_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__10_Inheritance__4dcdb777b0cfaae4.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000407	SRC00032/p25	covered	Alternate extracted slide text page 25; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000408	SRC00031/p26	covered	Extracted slide page 26 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/010_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__10_Inheritance__4dcdb777b0cfaae4.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000409	SRC00033/p26	covered	Original slide page 26 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/010_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__10_Inheritance__4dcdb777b0cfaae4.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000410	SRC00032/p26	covered	Alternate extracted slide text page 26; normalized text matches the .slides.md page. Human comparison is pending.
EV0000411	SRC00034/p1	covered	Extracted slide page 1 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/011_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__11_Polymorphism__d914ed6dbb4c52c6.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000412	SRC00036/p1	covered	Original slide page 1 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/011_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__11_Polymorphism__d914ed6dbb4c52c6.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000413	SRC00035/p1	covered	Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000414	SRC00034/p2	covered	Extracted slide page 2 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/011_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__11_Polymorphism__d914ed6dbb4c52c6.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000415	SRC00036/p2	covered	Original slide page 2 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/011_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__11_Polymorphism__d914ed6dbb4c52c6.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000416	SRC00035/p2	covered	Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human comparison is pending.
EV0000417	SRC00034/p3	covered	Extracted slide page 3 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/011_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__11_Polymorphism__d914ed6dbb4c52c6.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000418	SRC00036/p3	covered	Original slide page 3 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/011_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__11_Polymorphism__d914ed6dbb4c52c6.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000419	SRC00035/p3	covered	Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000420	SRC00034/p4	covered	Extracted slide page 4 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/011_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__11_Polymorphism__d914ed6dbb4c52c6.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000421	SRC00036/p4	covered	Original slide page 4 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/011_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__11_Polymorphism__d914ed6dbb4c52c6.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000422	SRC00035/p4	covered	Alternate extracted slide text page 4; normalized text matches the .slides.md page. Human comparison is pending.
EV0000423	SRC00034/p5	covered	Extracted slide page 5 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/011_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__11_Polymorphism__d914ed6dbb4c52c6.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000424	SRC00036/p5	covered	Original slide page 5 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/011_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__11_Polymorphism__d914ed6dbb4c52c6.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000425	SRC00035/p5	covered	Alternate extracted slide text page 5; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000426	SRC00034/p6	covered	Extracted slide page 6 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/011_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__11_Polymorphism__d914ed6dbb4c52c6.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000427	SRC00036/p6	covered	Original slide page 6 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/011_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__11_Polymorphism__d914ed6dbb4c52c6.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000428	SRC00035/p6	covered	Alternate extracted slide text page 6; normalized text matches the .slides.md page. Human comparison is pending.
EV0000429	SRC00034/p7	covered	Extracted slide page 7 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/011_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__11_Polymorphism__d914ed6dbb4c52c6.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000430	SRC00036/p7	covered	Original slide page 7 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/011_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__11_Polymorphism__d914ed6dbb4c52c6.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000431	SRC00035/p7	covered	Alternate extracted slide text page 7; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000432	SRC00034/p8	covered	Extracted slide page 8 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/011_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__11_Polymorphism__d914ed6dbb4c52c6.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000433	SRC00036/p8	covered	Original slide page 8 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/011_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__11_Polymorphism__d914ed6dbb4c52c6.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000434	SRC00035/p8	covered	Alternate extracted slide text page 8; normalized text matches the .slides.md page. Human comparison is pending.
EV0000435	SRC00034/p9	covered	Extracted slide page 9 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/011_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__11_Polymorphism__d914ed6dbb4c52c6.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000436	SRC00036/p9	covered	Original slide page 9 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/011_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__11_Polymorphism__d914ed6dbb4c52c6.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000437	SRC00035/p9	covered	Alternate extracted slide text page 9; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000438	SRC00034/p10	covered	Extracted slide page 10 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/011_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__11_Polymorphism__d914ed6dbb4c52c6.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000439	SRC00036/p10	covered	Original slide page 10 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/011_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__11_Polymorphism__d914ed6dbb4c52c6.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000440	SRC00035/p10	covered	Alternate extracted slide text page 10; normalized text matches the .slides.md page. Human comparison is pending.
EV0000441	SRC00037/p1	covered	Extracted slide page 1 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/014_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__12_Iterators__a61be1414e88ae54.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000442	SRC00039/p1	covered	Original slide page 1 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/014_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__12_Iterators__a61be1414e88ae54.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000443	SRC00038/p1	covered	Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000444	SRC00037/p2	covered	Extracted slide page 2 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/014_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__12_Iterators__a61be1414e88ae54.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000445	SRC00039/p2	covered	Original slide page 2 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/014_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__12_Iterators__a61be1414e88ae54.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000446	SRC00038/p2	covered	Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human comparison is pending.
EV0000447	SRC00037/p3	covered	Extracted slide page 3 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/014_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__12_Iterators__a61be1414e88ae54.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000448	SRC00039/p3	covered	Original slide page 3 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/014_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__12_Iterators__a61be1414e88ae54.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000449	SRC00038/p3	covered	Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000450	SRC00037/p4	covered	Extracted slide page 4 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/014_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__12_Iterators__a61be1414e88ae54.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000451	SRC00039/p4	covered	Original slide page 4 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/014_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__12_Iterators__a61be1414e88ae54.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000452	SRC00038/p4	covered	Alternate extracted slide text page 4; normalized text matches the .slides.md page. Human comparison is pending.
EV0000453	SRC00037/p5	covered	Extracted slide page 5 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/014_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__12_Iterators__a61be1414e88ae54.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000454	SRC00039/p5	covered	Original slide page 5 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/014_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__12_Iterators__a61be1414e88ae54.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000455	SRC00038/p5	covered	Alternate extracted slide text page 5; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000456	SRC00037/p6	covered	Extracted slide page 6 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/014_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__12_Iterators__a61be1414e88ae54.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000457	SRC00039/p6	covered	Original slide page 6 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/014_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__12_Iterators__a61be1414e88ae54.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000458	SRC00038/p6	covered	Alternate extracted slide text page 6; normalized text matches the .slides.md page. Human comparison is pending.
EV0000459	SRC00037/p7	covered	Extracted slide page 7 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/014_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__12_Iterators__a61be1414e88ae54.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000460	SRC00039/p7	covered	Original slide page 7 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/014_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__12_Iterators__a61be1414e88ae54.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000461	SRC00038/p7	covered	Alternate extracted slide text page 7; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000462	SRC00037/p8	covered	Extracted slide page 8 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/014_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__12_Iterators__a61be1414e88ae54.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000463	SRC00039/p8	covered	Original slide page 8 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/014_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__12_Iterators__a61be1414e88ae54.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000464	SRC00038/p8	covered	Alternate extracted slide text page 8; normalized text matches the .slides.md page. Human comparison is pending.
EV0000465	SRC00037/p9	covered	Extracted slide page 9 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/014_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__12_Iterators__a61be1414e88ae54.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000466	SRC00039/p9	covered	Original slide page 9 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/014_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__12_Iterators__a61be1414e88ae54.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000467	SRC00038/p9	covered	Alternate extracted slide text page 9; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000468	SRC00040/p1	covered	Extracted slide page 1 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/015_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__14__15_Exceptional_Handling__147207e8e0b93741.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000469	SRC00042/p1	covered	Original slide page 1 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/015_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__14__15_Exceptional_Handling__147207e8e0b93741.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000470	SRC00041/p1	covered	Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human comparison is pending.
EV0000471	SRC00040/p2	covered	Extracted slide page 2 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/015_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__14__15_Exceptional_Handling__147207e8e0b93741.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000472	SRC00042/p2	covered	Original slide page 2 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/015_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__14__15_Exceptional_Handling__147207e8e0b93741.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000473	SRC00041/p2	covered	Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000474	SRC00040/p3	covered	Extracted slide page 3 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/015_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__14__15_Exceptional_Handling__147207e8e0b93741.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000475	SRC00042/p3	covered	Original slide page 3 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/015_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__14__15_Exceptional_Handling__147207e8e0b93741.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000476	SRC00041/p3	covered	Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human comparison is pending.
EV0000477	SRC00040/p4	covered	Extracted slide page 4 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/015_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__14__15_Exceptional_Handling__147207e8e0b93741.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000478	SRC00042/p4	covered	Original slide page 4 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/015_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__14__15_Exceptional_Handling__147207e8e0b93741.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000479	SRC00041/p4	covered	Alternate extracted slide text page 4; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000480	SRC00040/p5	covered	Extracted slide page 5 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/015_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__14__15_Exceptional_Handling__147207e8e0b93741.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000481	SRC00042/p5	covered	Original slide page 5 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/015_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__14__15_Exceptional_Handling__147207e8e0b93741.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000482	SRC00041/p5	covered	Alternate extracted slide text page 5; normalized text matches the .slides.md page. Human comparison is pending.
EV0000483	SRC00040/p6	covered	Extracted slide page 6 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/015_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__14__15_Exceptional_Handling__147207e8e0b93741.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000484	SRC00042/p6	covered	Original slide page 6 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/015_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__14__15_Exceptional_Handling__147207e8e0b93741.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000485	SRC00041/p6	covered	Alternate extracted slide text page 6; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000486	SRC00040/p7	covered	Extracted slide page 7 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/015_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__14__15_Exceptional_Handling__147207e8e0b93741.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000487	SRC00042/p7	covered	Original slide page 7 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/015_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__14__15_Exceptional_Handling__147207e8e0b93741.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000488	SRC00041/p7	covered	Alternate extracted slide text page 7; normalized text matches the .slides.md page. Human comparison is pending.
EV0000489	SRC00040/p8	covered	Extracted slide page 8 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/015_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__14__15_Exceptional_Handling__147207e8e0b93741.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000490	SRC00042/p8	covered	Original slide page 8 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/015_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__14__15_Exceptional_Handling__147207e8e0b93741.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000491	SRC00041/p8	covered	Alternate extracted slide text page 8; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000492	SRC00040/p9	covered	Extracted slide page 9 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/015_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__14__15_Exceptional_Handling__147207e8e0b93741.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000493	SRC00042/p9	covered	Original slide page 9 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/015_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__14__15_Exceptional_Handling__147207e8e0b93741.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000494	SRC00041/p9	covered	Alternate extracted slide text page 9; normalized text matches the .slides.md page. Human comparison is pending.
EV0000495	SRC00040/p10	covered	Extracted slide page 10 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/015_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__14__15_Exceptional_Handling__147207e8e0b93741.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000496	SRC00042/p10	covered	Original slide page 10 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/015_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__14__15_Exceptional_Handling__147207e8e0b93741.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000497	SRC00041/p10	covered	Alternate extracted slide text page 10; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000498	SRC00040/p11	covered	Extracted slide page 11 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/015_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__14__15_Exceptional_Handling__147207e8e0b93741.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000499	SRC00042/p11	covered	Original slide page 11 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/015_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__14__15_Exceptional_Handling__147207e8e0b93741.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000500	SRC00041/p11	covered	Alternate extracted slide text page 11; normalized text matches the .slides.md page. Human comparison is pending.
EV0000501	SRC00040/p12	covered	Extracted slide page 12 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/015_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__14__15_Exceptional_Handling__147207e8e0b93741.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000502	SRC00042/p12	covered	Original slide page 12 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/015_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__14__15_Exceptional_Handling__147207e8e0b93741.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000503	SRC00041/p12	covered	Alternate extracted slide text page 12; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000504	SRC00040/p13	covered	Extracted slide page 13 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/015_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__14__15_Exceptional_Handling__147207e8e0b93741.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000505	SRC00042/p13	covered	Original slide page 13 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/015_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__14__15_Exceptional_Handling__147207e8e0b93741.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000506	SRC00041/p13	covered	Alternate extracted slide text page 13; normalized text matches the .slides.md page. Human comparison is pending.
EV0000507	SRC00040/p14	covered	Extracted slide page 14 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/015_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__14__15_Exceptional_Handling__147207e8e0b93741.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000508	SRC00042/p14	covered	Original slide page 14 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/015_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__14__15_Exceptional_Handling__147207e8e0b93741.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000509	SRC00041/p14	covered	Alternate extracted slide text page 14; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000510	SRC00040/p15	covered	Extracted slide page 15 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/015_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__14__15_Exceptional_Handling__147207e8e0b93741.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000511	SRC00042/p15	covered	Original slide page 15 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/015_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__14__15_Exceptional_Handling__147207e8e0b93741.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000512	SRC00041/p15	covered	Alternate extracted slide text page 15; normalized text matches the .slides.md page. Human comparison is pending.
EV0000513	SRC00040/p16	covered	Extracted slide page 16 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/015_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__14__15_Exceptional_Handling__147207e8e0b93741.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000514	SRC00042/p16	covered	Original slide page 16 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/015_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__14__15_Exceptional_Handling__147207e8e0b93741.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000515	SRC00041/p16	covered	Alternate extracted slide text page 16; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000516	SRC00040/p17	covered	Extracted slide page 17 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/015_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__14__15_Exceptional_Handling__147207e8e0b93741.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000517	SRC00042/p17	covered	Original slide page 17 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/015_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__14__15_Exceptional_Handling__147207e8e0b93741.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000518	SRC00041/p17	covered	Alternate extracted slide text page 17; normalized text matches the .slides.md page. Human comparison is pending.
EV0000519	SRC00040/p18	covered	Extracted slide page 18 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/015_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__14__15_Exceptional_Handling__147207e8e0b93741.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000520	SRC00042/p18	covered	Original slide page 18 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/015_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__14__15_Exceptional_Handling__147207e8e0b93741.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000521	SRC00041/p18	covered	Alternate extracted slide text page 18; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000522	SRC00040/p19	covered	Extracted slide page 19 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/015_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__14__15_Exceptional_Handling__147207e8e0b93741.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000523	SRC00042/p19	covered	Original slide page 19 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/015_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__14__15_Exceptional_Handling__147207e8e0b93741.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000524	SRC00041/p19	covered	Alternate extracted slide text page 19; normalized text matches the .slides.md page. Human comparison is pending.
EV0000525	SRC00040/p20	covered	Extracted slide page 20 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/015_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__14__15_Exceptional_Handling__147207e8e0b93741.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000526	SRC00042/p20	covered	Original slide page 20 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/015_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__14__15_Exceptional_Handling__147207e8e0b93741.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000527	SRC00041/p20	covered	Alternate extracted slide text page 20; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000528	SRC00040/p21	covered	Extracted slide page 21 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/015_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__14__15_Exceptional_Handling__147207e8e0b93741.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000529	SRC00042/p21	covered	Original slide page 21 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/015_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__14__15_Exceptional_Handling__147207e8e0b93741.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000530	SRC00041/p21	covered	Alternate extracted slide text page 21; normalized text matches the .slides.md page. Human comparison is pending.
EV0000531	SRC00040/p22	covered	Extracted slide page 22 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/015_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__14__15_Exceptional_Handling__147207e8e0b93741.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000532	SRC00042/p22	covered	Original slide page 22 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/015_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__14__15_Exceptional_Handling__147207e8e0b93741.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000533	SRC00041/p22	covered	Alternate extracted slide text page 22; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV0000534	SRC00040/p23	covered	Extracted slide page 23 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/015_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__14__15_Exceptional_Handling__147207e8e0b93741.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000535	SRC00042/p23	covered	Original slide page 23 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/015_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__14__15_Exceptional_Handling__147207e8e0b93741.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000536	SRC00041/p23	covered	Alternate extracted slide text page 23; normalized text matches the .slides.md page. Human comparison is pending.
EV0000537	SRC00040/p24	covered	Extracted slide page 24 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/015_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__14__15_Exceptional_Handling__147207e8e0b93741.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV0000538	SRC00042/p24	covered	Original slide page 24 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/015_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__14__15_Exceptional_Handling__147207e8e0b93741.pdf; visual page included as vector figure. Human layout/content review is pending.
EV0000539	SRC00041/p24	covered	Alternate extracted slide text page 24; normalized text matches the .slides.md page. Human comparison is pending.

Evidence	Source unit	Status	Decision
EV00005540	SRC00040/p25	covered	Extracted slide page 25 from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/015_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__14__15_Exceptional_Handling__147207e8e0b93741.slides.md; original page is also embedded when the source render is available. Human subject review is pending.
EV00005541	SRC00042/p25	covered	Original slide page 25 rendered from pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/015_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__14__15_Exceptional_Handling__147207e8e0b93741.pdf; visual page included as vector figure. Human layout/content review is pending.
EV00005542	SRC00041/p25	covered	Alternate extracted slide text page 25; normalized text matches the .slides.md page. Human comparison is pending.

C Revision and retrieval practice